

Statistics Lab Case Study

Team Members:-

507: Rajasi Barapatre

510: Sanika Chavan

547: Ketaki Paranjape

Dataset Used: Fatalities between Israel and Palestine

<https://www.kaggle.com/datasets/willianoliveiragibin/fatalities-in-the-israeli-palestinian>

```
import zipfile
import os
from google.colab import files
from google.colab import drive
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from scipy.stats import skew, kurtosis
from statsmodels.graphics.gofplots import qqplot
```

```
path="/content/fatalities_isr_pse_conflict_2000_to_2023.csv"
data=pd.read_csv(path)
```

```
data.head()
```

	name	date_of_event	age	citizenship	event_location	event_location_district	event_location_region	date_of_death	gender	took_part_in_the_hostilities	place_of_residence	place_of_residence_district	type_of_injury	ammunition	killed_by	notes	
0	'Abd a-Rahman Sulaiman Muhammad Abu Dughash	2023-08-24	32.0	Palestinian	Nur Shams R.C.	Tulkarm	West Bank	2023-08-24	M		NaN	Nur Shams R.C.	Tulkarm	gunfire	live ammunition	Israeli security forces	Fatally shot by Israeli forces while standing ...
1	Usayed Farhan Muhammad 'Ali Abu 'Ali	2023-08-24	21.0	Palestinian	Nur Shams R.C.	Tulkarm	West Bank	2023-08-24	M		NaN	Nur Shams R.C.	Tulkarm	gunfire	live ammunition	Israeli security forces	Fatally shot by Israeli forces while trying to...
2	'Abdallah 'Imad Sa'ed Abu Hassan	2023-08-22	16.0	Palestinian	Kfar Dan	Jenin	West Bank	2023-08-22	M		NaN	al-Yamun	Jenin	gunfire	live ammunition	Israeli security forces	Fatally shot by soldiers while firing at them ...
3	Durgham Muhammad Yhya al-Akhras	2023-08-20	19.0	Palestinian	'Aqbat Jaber R.C.	Jericho	West Bank	2023-08-20	M		NaN	'Aqbat Jaber R.C.	Jericho	gunfire	live ammunition	Israeli security forces	Shot in the head by Israeli forces while throw...
4	Rasafat 'Omar Ahmad Khameisah	2023-08-19	15.0	Palestinian	Jenin R.C.	Jenin	West Bank	2023-08-19	M		NaN	Jenin	Jenin	gunfire	live ammunition	Israeli security forces	Wounded by soldiers' gunfire after running awa...

Data.shape

(11124, 16)

Data.dtypes

```

name                object
date_of_event       object
age                 float64
citizenship         object
event_location      object
event_location_district  object
event_location_region  object
date_of_death       object
gender              object
took_part_in_the_hostilities  object
place_of_residence  object
place_of_residence_district  object
type_of_injury      object
ammunition          object
killed_by           object
notes               object
dtype: object

```

Data.columns

Index(['name', 'date_of_event', 'age', 'citizenship', 'event_location',

```
'event_location_district', 'event_location_region', 'date_of_death',
'gender', 'took_part_in_the_hostilities', 'place_of_residence',
'place_of_residence_district', 'type_of_injury', 'ammunition',
'killed_by', 'notes'],
dtype='object')
```

CHECKING NULL VALUES IN THE DATASET

```
data.isnull().sum().sort_values(ascending = False)
```

```
ammunition          5253
took_part_in_the_hostilities  1430
type_of_injury       291
notes               280
age                129
place_of_residence    68
place_of_residence_district  68
gender              20
name                0
date_of_event        0
citizenship          0
event_location       0
event_location_district  0
event_location_region  0
date_of_death        0
killed_by           0
```

CHECKING DATA TYPES

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 11124 entries, 0 to 11123
```

```
Data columns (total 16 columns):
```

#	Column	Non-Null Count	Dtype
0	name	11124 non-null	object
1	date_of_event	11124 non-null	object
2	age	10995 non-null	float64
3	citizenship	11124 non-null	object
4	event_location	11124 non-null	object

```

5 event_location_district    11124 non-null object
6 event_location_region      11124 non-null object
7 date_of_death              11124 non-null object
8 gender                     11104 non-null object
9 took_part_in_the_hostilities 9694 non-null object
10 place_of_residence         11056 non-null object
11 place_of_residence_district 11056 non-null object
12 type_of_injury             10833 non-null object
13 ammunition                 5871 non-null object
14 killed_by                  11124 non-null object
15 notes                      10844 non-null object

```

dtypes: float64(1), object(15)

memory usage: 1.4+ MB

```

missing_percentage = (data.isnull().sum().sort_values(ascending = False)/len(data))*100
Missing_percentage

```

```

ammunition           47.222222
took_part_in_the_hostilities 12.855088
type_of_injury        2.615965
notes                 2.517080
age                   1.159655
place_of_residence    0.611291
place_of_residence_district 0.611291
gender                0.179791
name                  0.000000
date_of_event         0.000000
citizenship           0.000000
event_location        0.000000
event_location_district 0.000000
event_location_region 0.000000
date_of_death         0.000000
killed_by             0.000000
dtype: float64

```

Filling null values

```
data['ammunition'].fillna('missile', inplace=True) # Replace with the mode or a default value
```

```
data['took_part_in_the_hostilities'].fillna('No', inplace=True)
```

```
data['type_of_injury'].fillna(data['type_of_injury'].mode().values[0], inplace=True)
```

```
data['age'].fillna(data['age'].median(), inplace=True)
```

```
data['place_of_residence'].fillna('Unknown', inplace=True)
```

```
data['place_of_residence_district'].fillna('Unknown', inplace=True)
```

```
data['gender'].fillna(data['gender'].mode().values[0], inplace=True)
```

```
data.drop(columns=['notes'], inplace=True)
```

```
data.isnull().sum().sort_values(ascending = False)
```

name	0
date_of_event	0
age	0
citizenship	0
event_location	0
event_location_district	0
event_location_region	0
date_of_death	0
gender	0
took_part_in_the_hostilities	0
place_of_residence	0
place_of_residence_district	0
type_of_injury	0
ammunition	0
killed_by	0

CHECKING UNIQUE VALUES AND THEIR COUNTS

```
data['ammunition'].value_counts()
```

missile	8130
live ammunition	1514
shell	675
explosive belt	326
bomb	249
mortar fire	51
knife	37
flechette shells	22
rubber-coated metal bullets	19

0.22-caliber bullets	16
phosphorus shell	16
Qassam rocket	15
car bomb	15
teargas canister	13
rocket	12
grad rocket	7
sponge rounds	2
grenade	2
flare bomb	1
stun grenade	1
rock	1

Name: ammunition, dtype: int64

data['name'].value_counts()

Name unknown to B'Tselem	23
Iyad Salah 'Abd al-Hafez Qafah	2
Muhammad Fouad Khalil Hijazi	2
Muhannad 'Omar Muhammad a-Nashasi	2
Tareq Rasmi Dufish	2
..	
Khadrah al-'Abed Salamah Abu Daqah	1
'Amru Ramadan Hassan Abu Daqah	1
Muhammad 'Abdallah Ahmad a-Zahuq	1
Husam Jamal Hasunah Shamalakh	1
Wichlav Zalsevsky	1

Name: name, Length: 11083, dtype: int64

data['citizenship'].value_counts()

Palestinian	10092
Israeli	1029
Jordanian	2
American	1

Name: citizenship, dtype: int64

data['event_location'].value_counts()

Gaza City	2232
Rafah	832
Khan Yunis	538

Jabalya R.C.	477
Beit Lahiya	471

...	
al-'Esawiyah	1
Elazar	1
Kibbutz Beit Govrin	1
Kh. Shuweika	1
Masha	1

Name: event_location, Length: 494, dtype: int64

data['event_location_district'].value_counts()

Gaza	2435
North Gaza	1910
Khan Yunis	1394
Rafah	1066
Deir al-Balah	854
Israel	679
Nablus	647
Jenin	512
Ramallah and al-Bira	350
Hebron	347
Tulkarm	254
Bethlehem	186
East Jerusalem	130
al-Quds	85
Gush Katif	70
Qalqiliya	65
Tubas	52
Jericho	48
Salfit	36
Gaza Strip	4

Name: event_location_district, dtype: int64

data['event_location_region'].value_counts()

Gaza Strip	7733
West Bank	2712
Israel	679

Name: event_location_region, dtype: int64

data['place_of_residence'].value_counts()

Gaza city	2283
Rafah	892

Khan Yunis	717
Jabalya R.C.	565
Beit Lahiya	479
...	
Shufah	1
Kibbutz Bror Hayil	1
Mishmar Hayarden	1
Um al-Ghanam	1
Banei Ayish	1

Name: place_of_residence, Length: 581, dtype: int64

```
data['place_of_residence_district'].value_counts()
```

Gaza	2453
North Gaza	1831
Khan Yunis	1367
Rafah	1097
Deir al-Balah	875
Israel	726
Nablus	621
Jenin	540
Hebron	380
Ramallah and al-Bira	295
Tulkarm	258
Bethlehem	167
East Jerusalem	121
al-Quds	102
Unknown	68
Qalqiliya	66
Tubas	56
Salfit	47
Jericho	34
Gush Katif	16
West Bank	4

Name: place_of_residence_district, dtype: int64

```
data['type_of_injury'].value_counts()
```

gunfire	10140
explosion	555
shelling	311
stabbing	48
house demolition	25


```

hit by a vehicle      18
beating               9
stones throwing       6
being bludgeoned with an axe  4
fire                  4
physically assaulted   2
physical assault      1
Strangulation         1
Name: type_of_injury, dtype: int64

```

```

data['killed_by'].value_counts()
Israeli security forces  10000
Palestinian civilians   1028
Israeli civilians       96
Name: killed_by, dtype: int64

```

UNIVARIATE ANALYSIS

Exploring age attribute

```

# Calculate basic descriptive statistics for the "age" column
mean_age = data['age'].mean()
median_age = data['age'].median()
std_age = data['age'].std()
mode_age = data['age'].mode().values[0]
variance_age = data['age'].var()

print(f"Mean Age: {mean_age}")
print(f"Median Age: {median_age}")
print(f"Mode Age: {mode_age}")
print(f"Standard Deviation of Age: {std_age}")
print(f"Variance: {variance_age}")
print("Range: ", data['age'].min(), " to ", data['age'].max())

```

```

Mean Age: 26.702265372168284
Median Age: 23.0

```

Mode Age: 23.0
Standard Deviation of Age: 13.70627225142466
Variance: 187.86189903017365
Range: 1.0 to 112.0

```
skewness = skew(data['age'])  
kurt = kurtosis(data['age'])  
print(f"Skewness: {skewness}")  
print(f"Kurtosis: {kurt}")
```

Skewness: 1.4104750564409838
Kurtosis: 2.7045637442543864

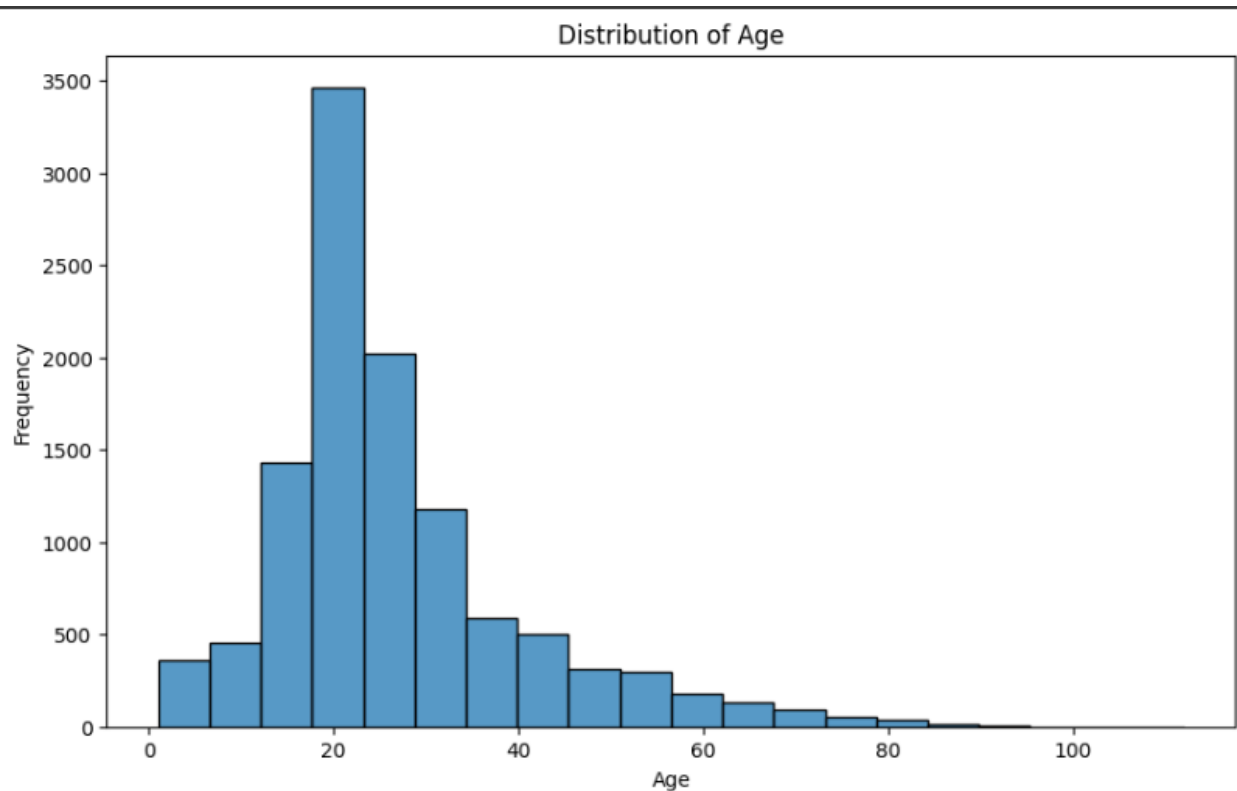
```
percentiles = np.percentile(data['age'], [25, 50, 75])  
print(f"25th Percentile: {percentiles[0]}")  
print(f"50th Percentile (Median): {percentiles[1]}")  
print(f"75th Percentile: {percentiles[2]}\n")
```

25th Percentile: 19.0
50th Percentile (Median): 23.0
75th Percentile: 31.0

```
Q1 = data['age'].quantile(0.25)  
Q3 = data['age'].quantile(0.75)  
IQR = Q3 - Q1  
lower_bound = Q1 - 1.5 * IQR  
upper_bound = Q3 + 1.5 * IQR  
outliers = data[(data['age'] < lower_bound) | (data['age'] > upper_bound)]  
print(f"Outliers: {outliers}\n")
```

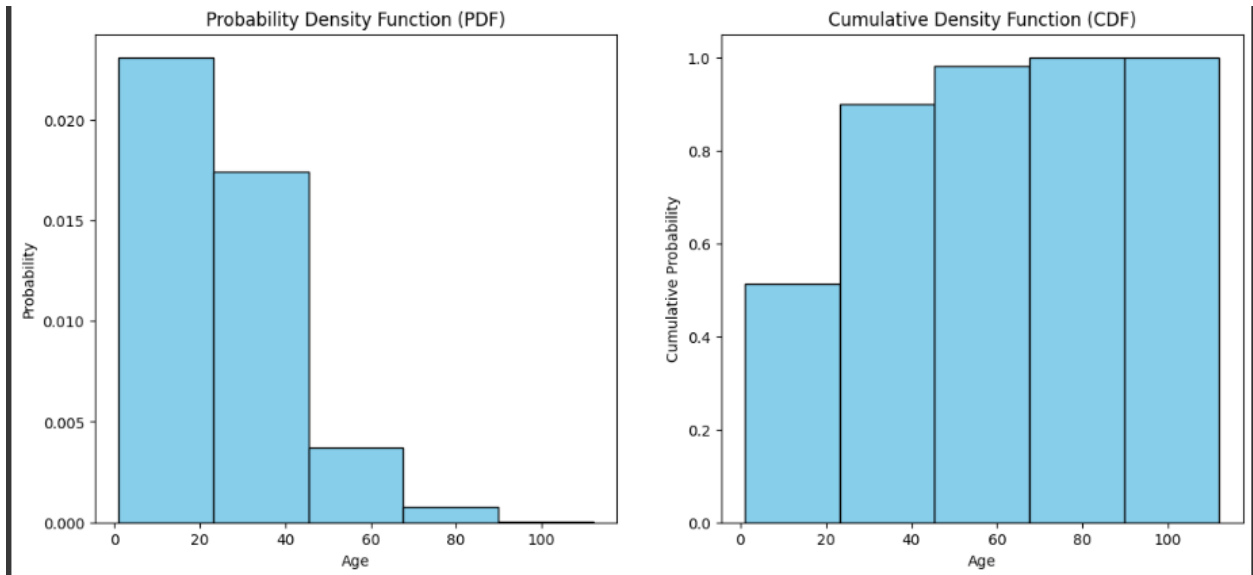
```
plt.figure(figsize=(10, 6))  
sns.histplot(data=data, x='age', bins=20)  
plt.title('Distribution of Age')  
plt.xlabel('Age')  
plt.ylabel('Frequency')
```

```
plt.show()
```



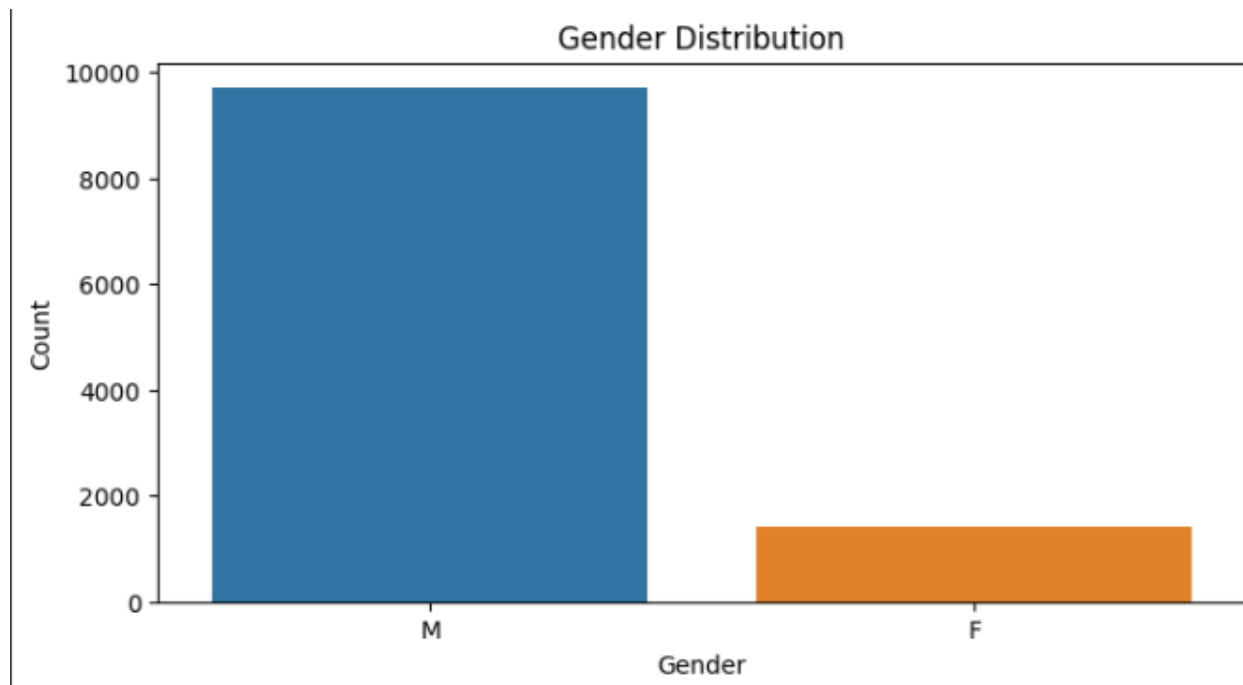
```
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.hist(data['age'], bins=5, density=True, color='skyblue', edgecolor='black')
plt.xlabel('Age')
plt.ylabel('Probability')
plt.title('Probability Density Function (PDF)')
```

```
plt.subplot(1, 2, 2)
plt.hist(data['age'], bins=5, density=True, cumulative=True, color='skyblue',
edgecolor='black')
plt.xlabel('Age')
plt.ylabel('Cumulative Probability')
plt.title('Cumulative Density Function (CDF)')
plt.show()
```



Exploring gender attribute

```
plt.figure(figsize=(8, 4))  
sns.countplot(data=data, x='gender')  
plt.title('Gender Distribution')  
plt.xlabel('Gender')  
plt.ylabel('Count')  
plt.show()
```



```
# Calculate the total number of casualties by gender
```

```
gender_casualties = data['gender'].value_counts()
```

```
print(gender_casualties)
```

```
M    9701
```

```
F     1423
```

```
Name: gender, dtype: int64
```

```
gender_percentage = data['gender'].value_counts(normalize=True)
```

```
mode_gender = data['gender'].mode().values[0]
```

```
print(f"Gender Percentage: {gender_percentage}")
```

```
print(f"Mode Gender: {mode_gender}")
```

```
Gender Percentage: M    0.872078
```

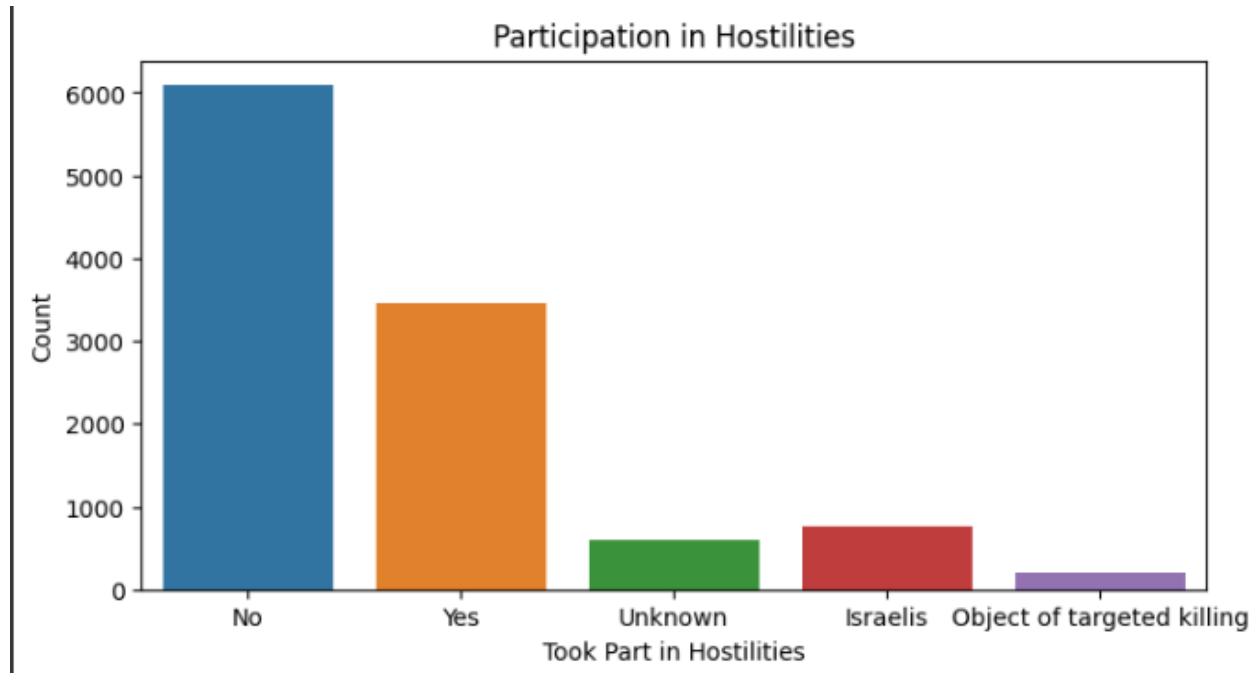
```
F    0.127922
```

```
Name: gender, dtype: float64
```

```
Mode Gender: M
```

Exploring took part in the hostilities attribute

```
plt.figure(figsize=(8, 4))
sns.countplot(data=data, x='took_part_in_the_hostilities')
plt.title('Participation in Hostilities')
plt.xlabel('Took Part in Hostilities')
plt.ylabel('Count')
plt.show()
```



```
# Calculate the proportion of events where individuals "took_part_in_the_hostilities"
hostilities_proportion =
data['took_part_in_the_hostilities'].value_counts(normalize=True)
print(hostilities_proportion)
```

```
No          0.546836
Yes          0.311668
Israelis     0.069310
Unknown      0.054207
Object of targeted killing  0.017979
Name: took_part_in_the_hostilities, dtype: float64
```

```

hostilities_percentage =
data['took_part_in_the_hostilities'].value_counts(normalize=True)
mode_hostilities = data['took_part_in_the_hostilities'].mode().values[0]
print(f"Hostilities Percentage: {hostilities_percentage}")
print(f"Mode Hostilities: {mode_hostilities}")

```

```

Hostilities Percentage: No          0.546836
Yes          0.311668
Israelis     0.069310
Unknown      0.054207
Object of targeted killing  0.017979
Name: took_part_in_the_hostilities, dtype: float64
Mode Hostilities: No

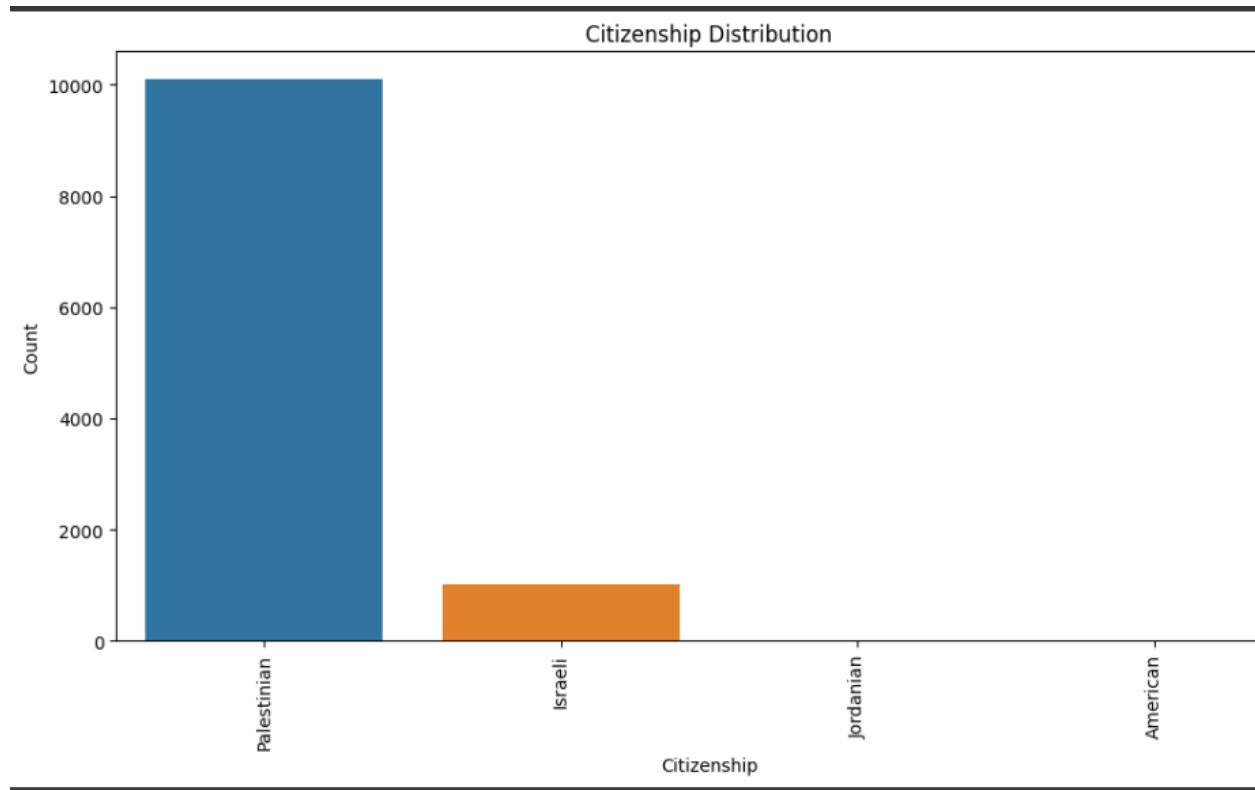
```

Exploring citizenship attribute

```

plt.figure(figsize=(12, 6))
sns.barplot(x=data['citizenship'].value_counts().index,
y=data['citizenship'].value_counts().values)
plt.title('Citizenship Distribution')
plt.xlabel('Citizenship')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.show()

```



```
citizenship_counts = data['citizenship'].value_counts()
print(citizenship_counts)
```

```
Palestinian    10092
Israeli         1029
Jordanian         2
American         1
Name: citizenship, dtype: int64
```

```
citizenship_percentage = data['citizenship'].value_counts(normalize=True)
mode_citizenship = data['citizenship'].mode().values[0]
print(f"Citizenship Percentage: {citizenship_percentage}")
print(f"Mode Citizenship: {mode_citizenship}")
```

```
Citizenship Percentage: Palestinian    0.907228
Israeli    0.092503
Jordanian    0.000180
American    0.000090
```


Name: citizenship, dtype: float64
Mode Citizenship: Palestinian

Exploring event_location_region attribute

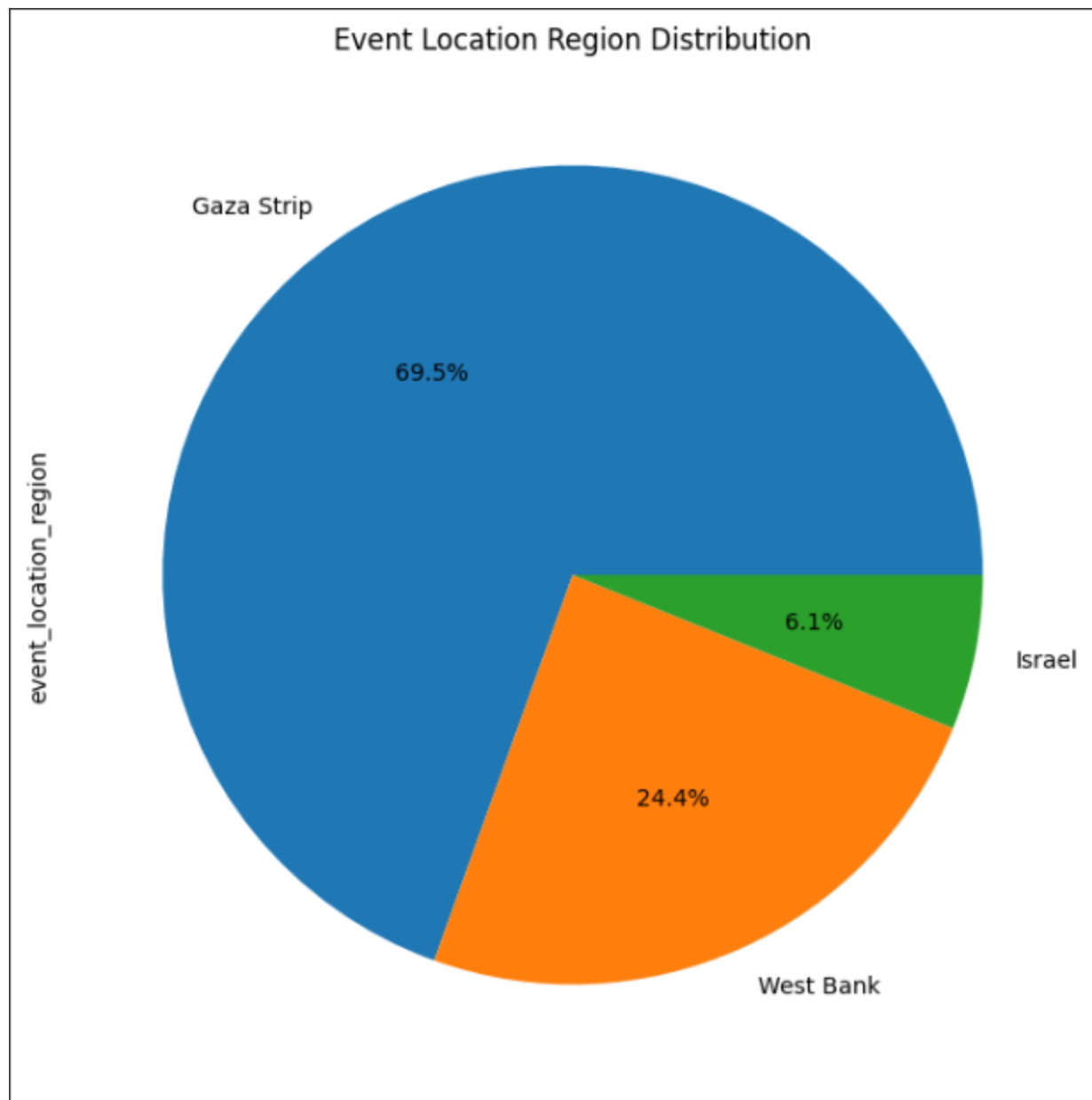
```
location_counts = data['event_location_region'].value_counts()  
print(location_counts)
```

```
Gaza Strip    7733  
West Bank    2712  
Israel        679  
Name: event_location_region, dtype: int64
```

```
region_percentage = data['event_location_region'].value_counts(normalize=True)  
mode_region = data['event_location_region'].mode().values[0]  
print(f"Region Percentage: {region_percentage}")  
print(f"Mode Region: {mode_region}")
```

```
Region Percentage: Gaza Strip    0.695164  
West Bank    0.243797  
Israel    0.061039  
Name: event_location_region, dtype: float64  
Mode Region: Gaza Strip
```

```
plt.figure(figsize=(8, 8))  
data['event_location_region'].value_counts().plot(kind='pie', autopct='%1.1f%%')  
plt.title('Event Location Region Distribution')  
plt.show()
```

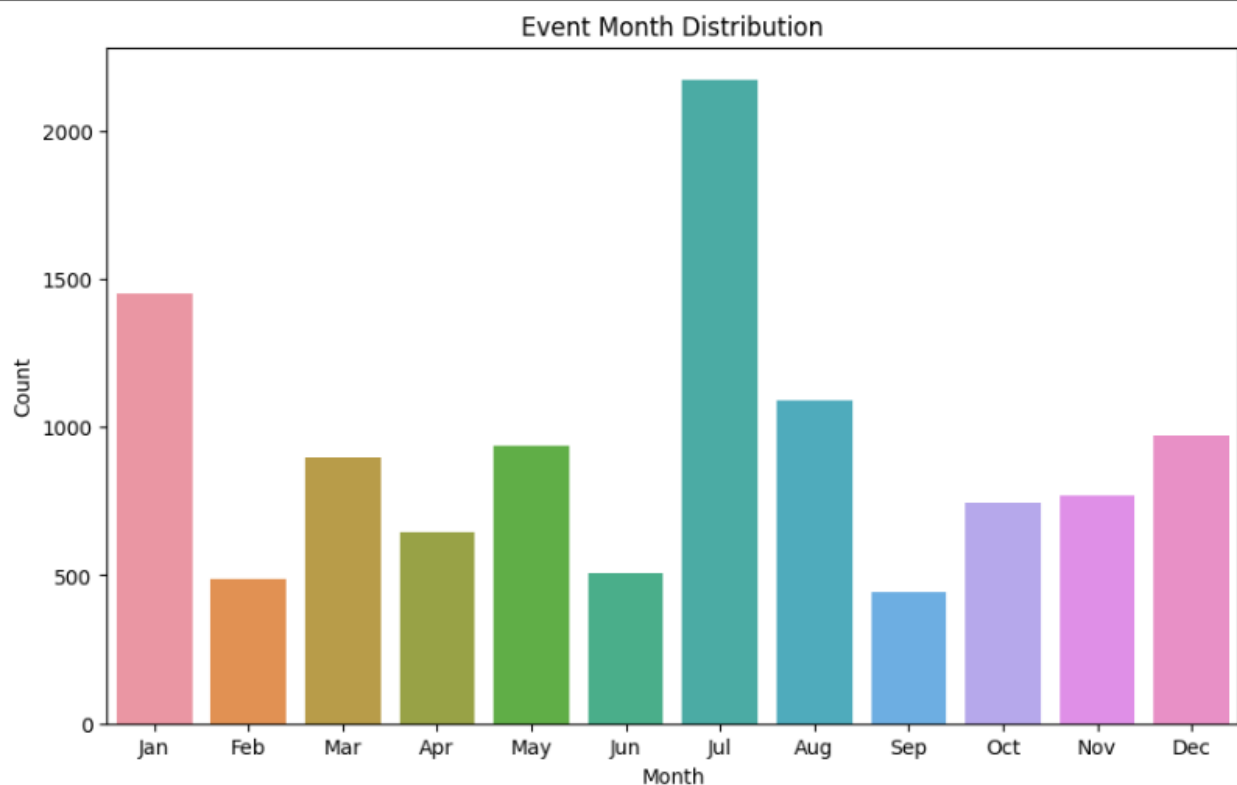


Exploring date_of_event attribute

```
data['date_of_event'] = pd.to_datetime(data['date_of_event'])  
data['event_month'] = data['date_of_event'].dt.month
```

```
plt.figure(figsize=(10, 6))  
sns.countplot(data=data, x='event_month')  
plt.title('Event Month Distribution')
```

```
plt.xlabel('Month')
plt.ylabel('Count')
plt.xticks(range(0, 12), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
plt.show()
```



```
mean_date = data['date_of_event'].mean()
median_date = data['date_of_event'].median()
mode_date = data['date_of_event'].mode().values[0]
date_range_start = data['date_of_event'].min()
date_range_end = data['date_of_event'].max()
print(f"Mean Date: {mean_date}")
print(f"Median Date: {median_date}")
print(f"Mode Date: {mode_date}")
print(f>Date Range: {date_range_start} to {date_range_end}")
```

Mean Date: 2009-10-31 01:25:18.446601984

Median Date: 2008-12-29 00:00:00
Mode Date: 2008-12-27T00:00:00.000000000
Date Range: 2000-10-02 00:00:00 to 2023-09-24 00:00:00

```
event_counts = data['date_of_event'].value_counts()  
print(event_counts)
```

```
2008-12-27    357  
2014-07-20    227  
2014-08-01    207  
2014-07-29    204  
2009-01-04    147
```

```
...  
2015-03-18     1  
2015-03-07     1  
2015-02-24     1  
2015-01-31     1  
2000-10-02     1
```

Name: date_of_event, Length: 2405, dtype: int64

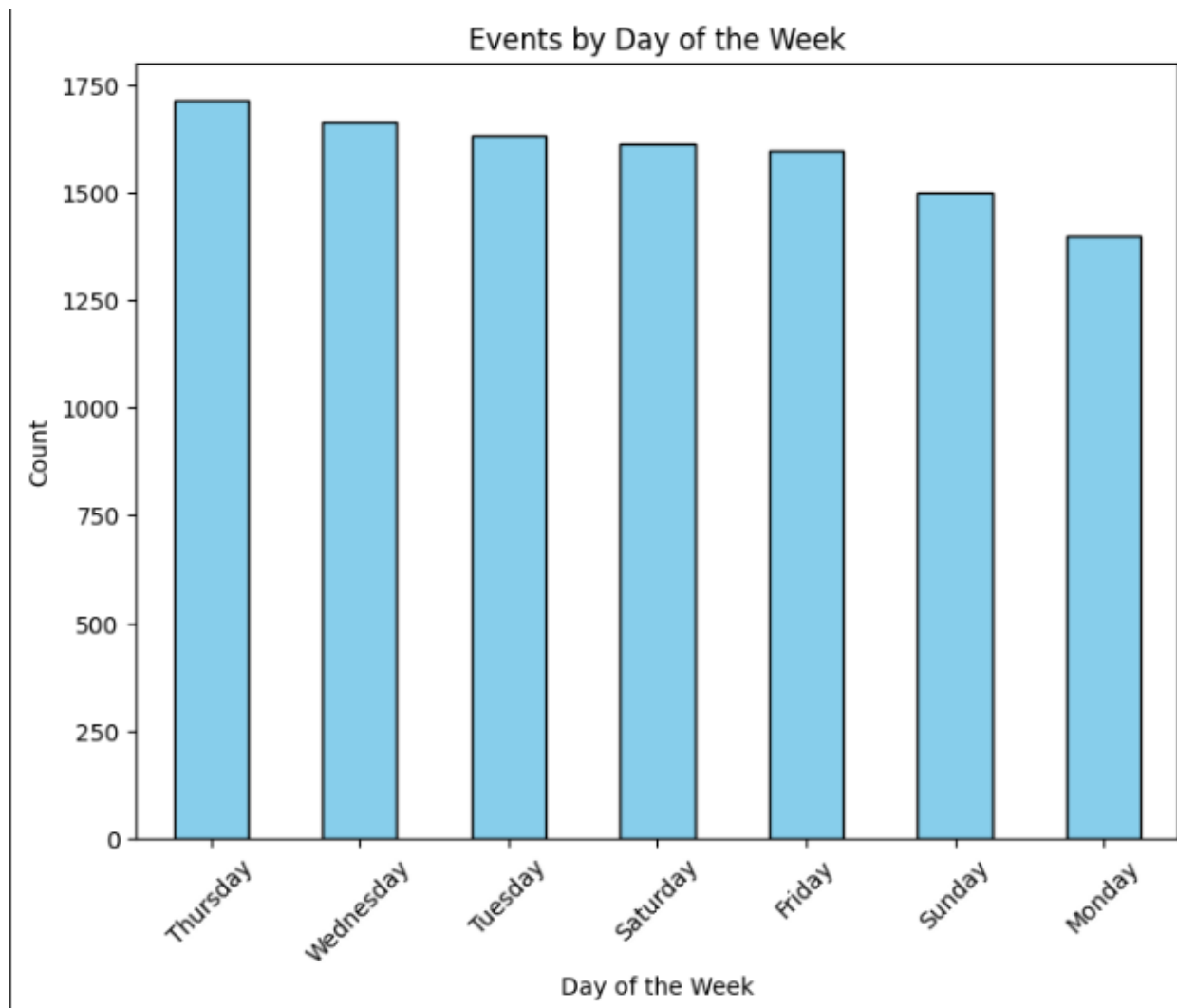
```
data['day_of_week'] = data['date_of_event'].dt.day_name()  
day_of_week_counts = data['day_of_week'].value_counts()  
print(data['day_of_week'])
```

```
0    Sunday  
1    Sunday  
2    Friday  
3  Wednesday  
4    Tuesday
```

```
...  
11119  Thursday  
11120   Tuesday  
11121   Saturday  
11122   Saturday  
11123    Monday
```

Name: day_of_week, Length: 11124, dtype: object

```
plt.figure(figsize=(8, 6))
day_of_week_counts.plot(kind='bar', color='skyblue', edgecolor='black')
plt.xlabel('Day of the Week')
plt.ylabel('Count')
plt.title('Events by Day of the Week')
plt.xticks(rotation=45)
plt.show()
```



Exploring date_of_death attribute

Count the number of events that occurred on each date

```
eventdeath_counts = data['date_of_death'].value_counts()
print(eventdeath_counts)
```

```
2008-12-27    341
2014-07-20    225
2014-08-01    197
2014-07-29    196
2014-07-30    131
```

```
...
2010-09-21     1
2010-09-25     1
2010-09-15     1
2010-09-17     1
2000-10-02     1
```

```
Name: date_of_death, Length: 2593, dtype: int64
```

```
data['date_of_death'] = pd.to_datetime(data['date_of_death'])
data['deathevent_month'] = data['date_of_death'].dt.month
```

```
# Create a count plot for the months
```

```
plt.figure(figsize=(10, 6))
```

```
sns.countplot(data=data, x='event_month')
```

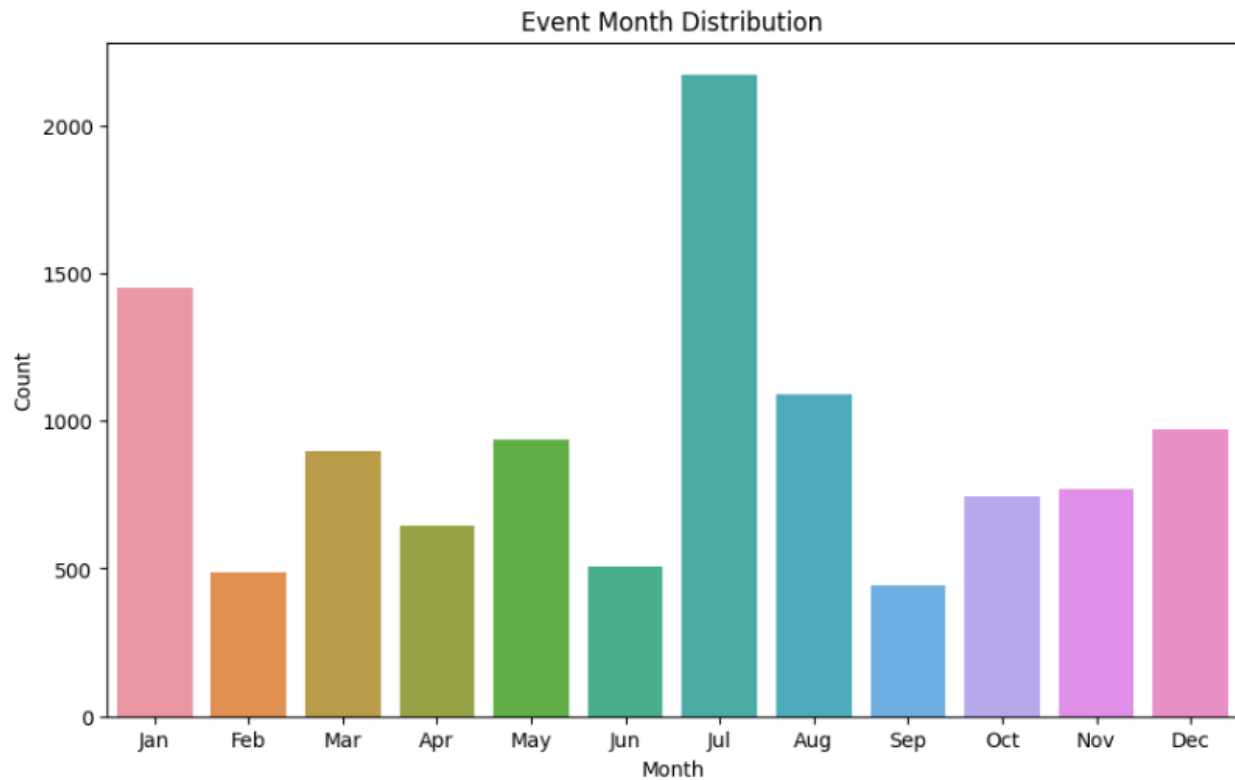
```
plt.title('Event Month Distribution')
```

```
plt.xlabel('Month')
```

```
plt.ylabel('Count')
```

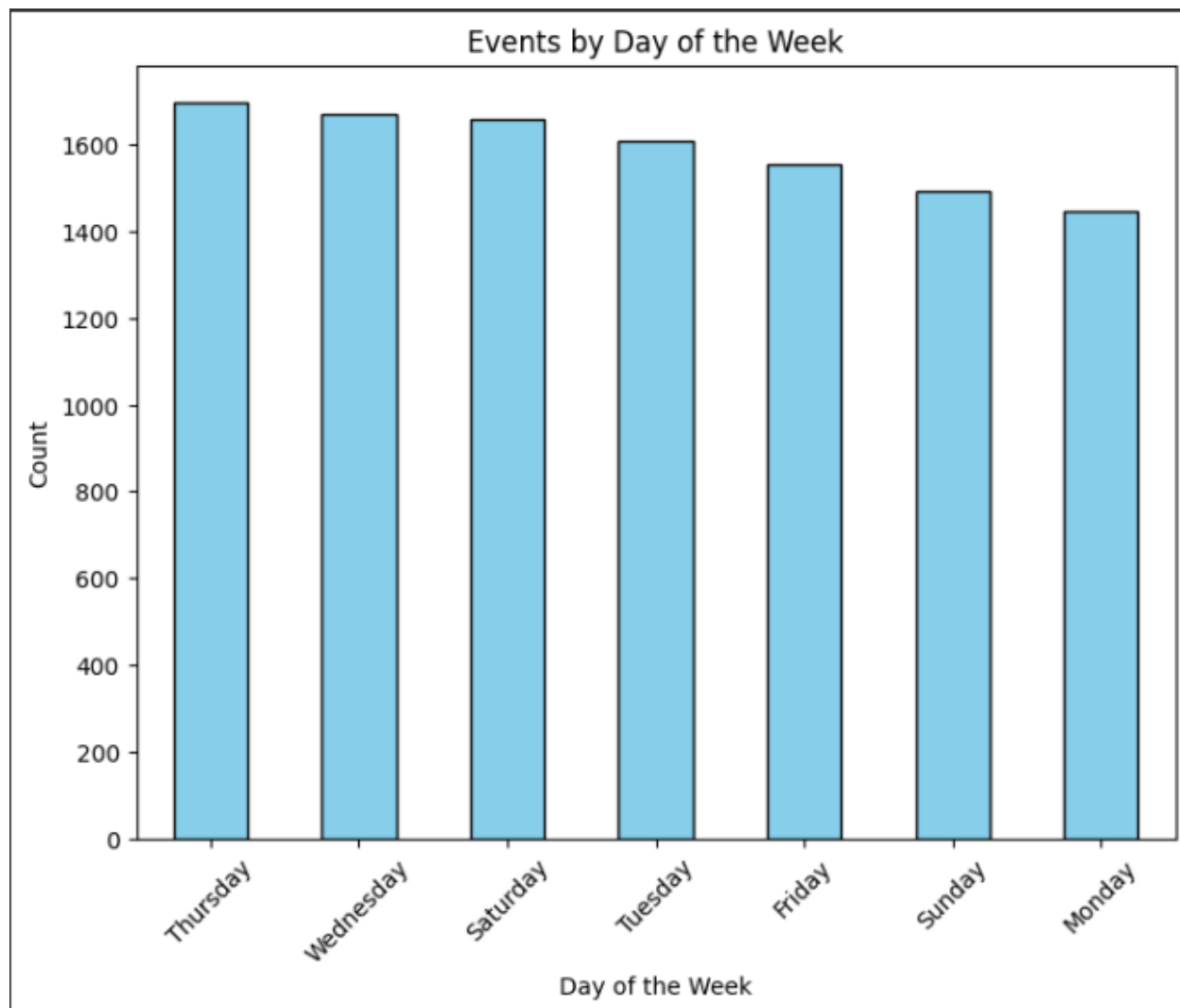
```
plt.xticks(range(0, 12), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
```

```
plt.show()
```



```
data['day_of_week_death'] = data['date_of_death'].dt.day_name()  
day_of_week_counts_death = data['day_of_week_death'].value_counts()
```

```
plt.figure(figsize=(8, 6))  
day_of_week_counts_death.plot(kind='bar', color='skyblue', edgecolor='black')  
plt.xlabel('Day of the Week')  
plt.ylabel('Count')  
plt.title('Events by Day of the Week')  
plt.xticks(rotation=45)  
plt.show()
```



```
data['year_death'] = data['date_of_death'].dt.year  
year_counts = data['year_death'].value_counts()
```

```
print(year_counts)
```

```
2014    2326  
2002    1326  
2009    1059  
2004     922  
2008     900  
2003     732  
2006     681
```


2007	400
2001	348
2021	324
2018	301
2012	262
2023	250
2005	240
2022	205
2015	179
2019	146
2011	130
2016	116
2010	89
2017	80
2013	42
2000	35
2020	31

Name: year_death, dtype: int64

```
data['year_event'] = data['date_of_event'].dt.year  
year_counts = data['year_event'].value_counts()
```

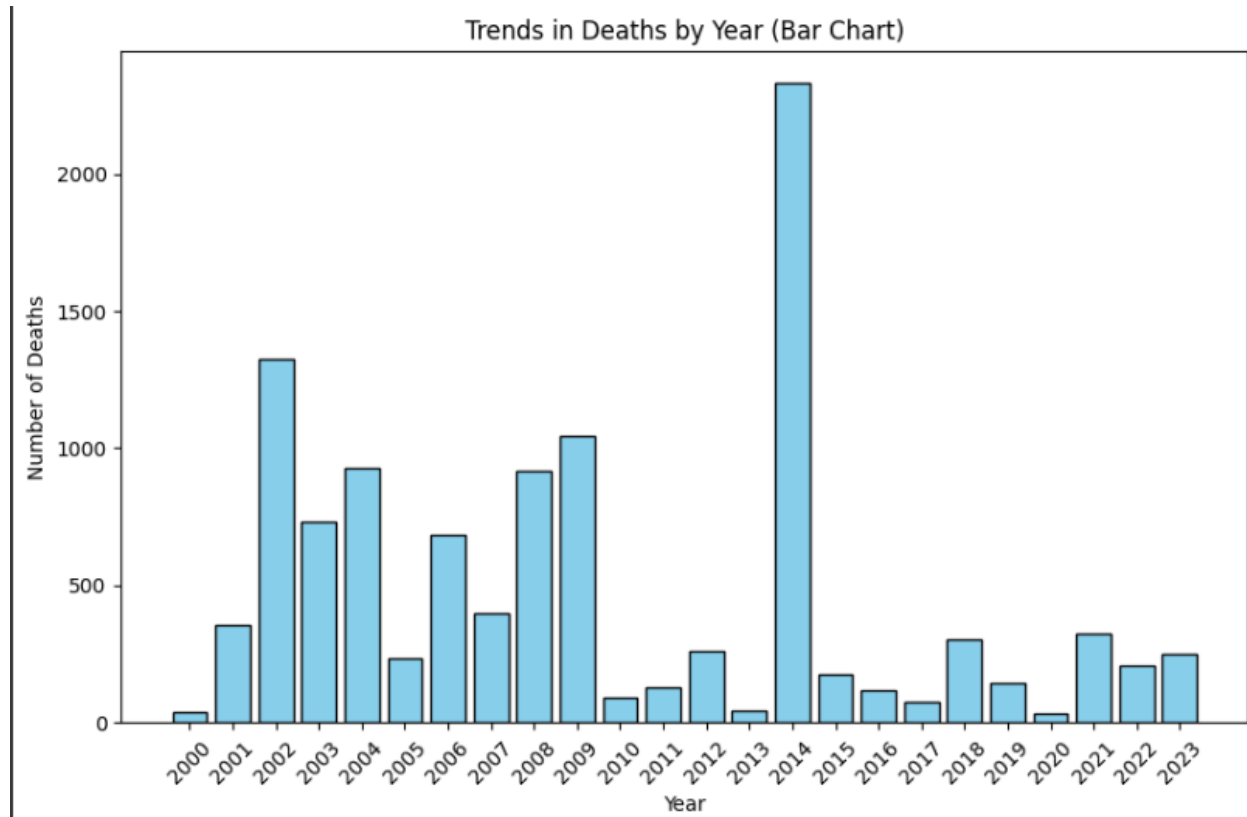
```
print(year_counts)
```

2014	2332
2002	1325
2009	1045
2004	928
2008	915
2003	733
2006	684
2007	395
2001	353
2021	325
2018	302
2012	261
2023	249
2005	234

2022	205
2015	177
2019	145
2011	129
2016	116
2010	89
2017	76
2013	41
2000	35
2020	30

Name: year_event, dtype: int64

```
plt.figure(figsize=(10, 6))
plt.bar(year_counts.index, year_counts.values, color='skyblue', edgecolor='black')
plt.xlabel('Year')
plt.ylabel('Number of Deaths')
plt.title('Trends in Deaths by Year (Bar Chart)')
plt.xticks(year_counts.index, rotation=45)
plt.show()
```



```
min_year = year_counts.idxmin()
max_year = year_counts.idxmax()
print(f"Notable Year with the Fewest Deaths: {min_year}")
print(f"Notable Year with the Most Deaths: {max_year}")
```

Notable Year with the Fewest Deaths: 2020

Notable Year with the Most Deaths: 2014

Exploring type of injury attribute

```
injury_percentage = data['type_of_injury'].value_counts(normalize=True)
mode_injury = data['type_of_injury'].mode().values[0]
print(f"Injury Type Percentage: {injury_percentage}")
print(f"Mode Injury Type: {mode_injury}")
```

Injury Type Percentage: gunfire	0.911543
explosion	0.049892
shelling	0.027958
stabbing	0.004315
house demolition	0.002247
hit by a vehicle	0.001618
beating	0.000809
stones throwing	0.000539
being bludgeoned with an axe	0.000360
fire	0.000360
physically assaulted	0.000180
physical assault	0.000090
Strangulation	0.000090

Name: type_of_injury, dtype: float64
Mode Injury Type: gunfire

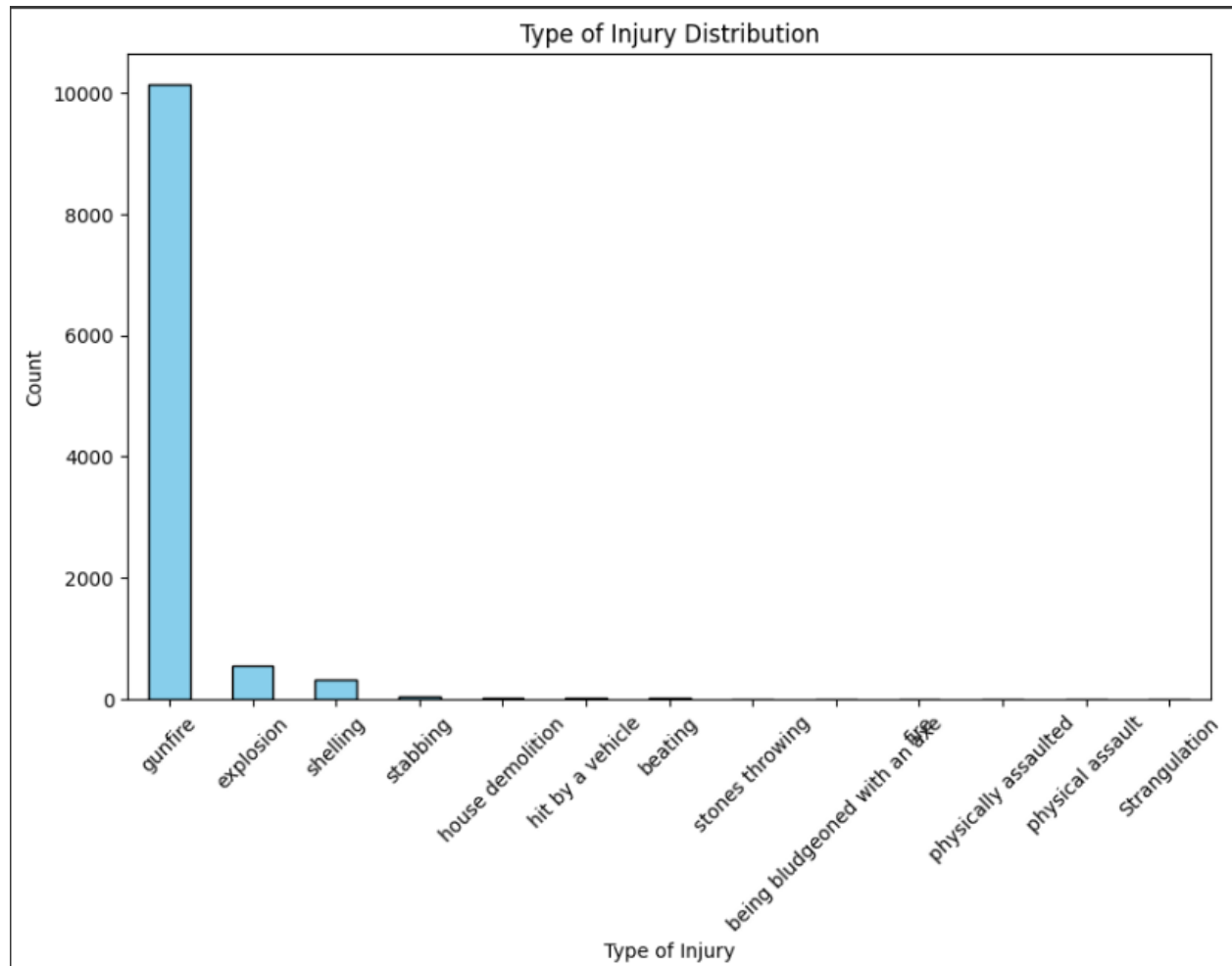
```
injury_counts = data['type_of_injury'].value_counts()
print(injury_counts)
```

gunfire	10140
explosion	555
shelling	311
stabbing	48
house demolition	25
hit by a vehicle	18
beating	9
stones throwing	6
being bludgeoned with an axe	4
fire	4
physically assaulted	2
physical assault	1
Strangulation	1

Name: type_of_injury, dtype: int64

```
plt.figure(figsize=(10, 6))
```

```
injury_counts.plot(kind='bar', color='skyblue', edgecolor='black')
plt.xlabel('Type of Injury')
plt.ylabel('Count')
plt.title('Type of Injury Distribution')
plt.xticks(rotation=45)
plt.show()
```



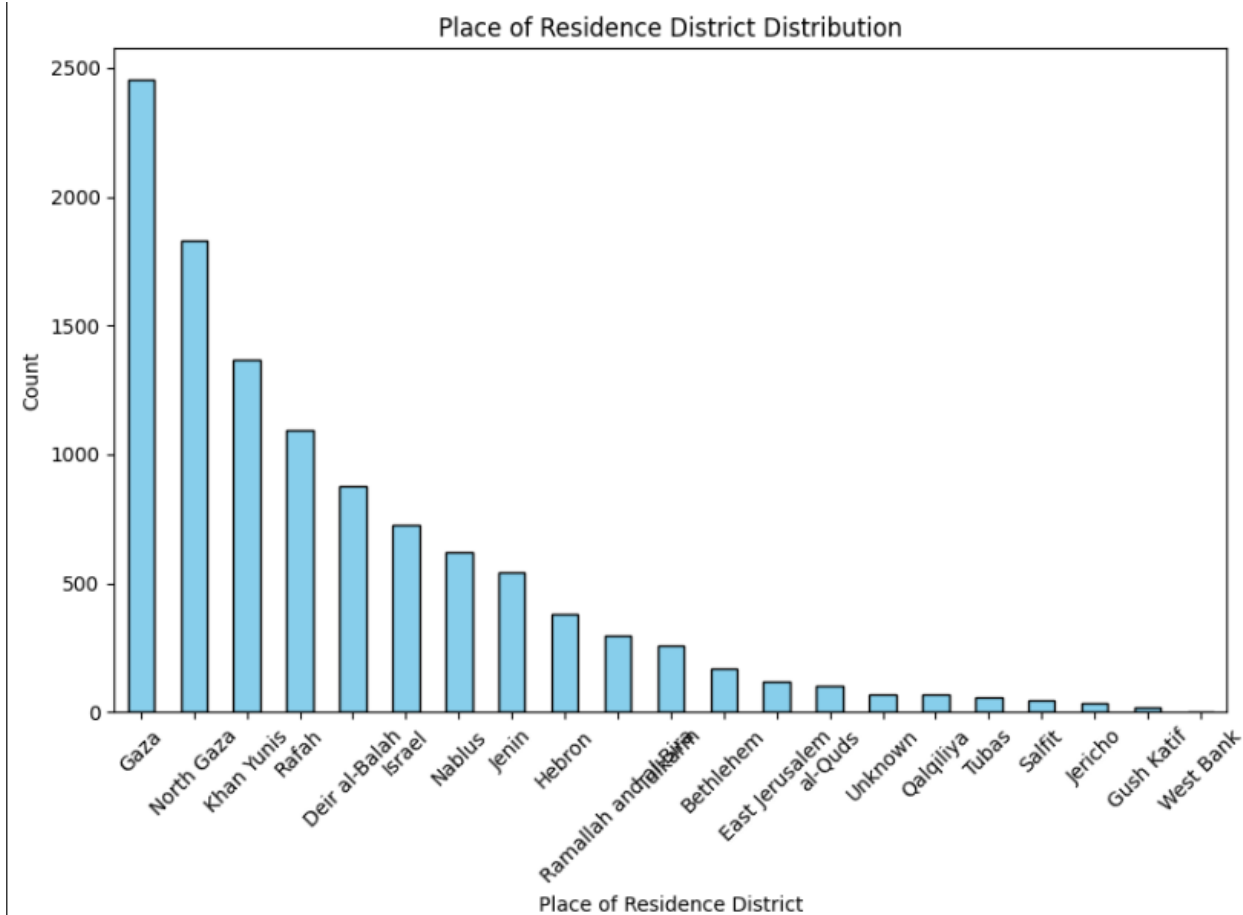
Exploring place of residence district attribute

```
district_counts = data['place_of_residence_district'].value_counts()
print(district_counts)
```

```
plt.figure(figsize=(10, 6))
district_counts.plot(kind='bar', color='skyblue', edgecolor='black')
plt.xlabel('Place of Residence District')
plt.ylabel('Count')
plt.title('Place of Residence District Distribution')
plt.xticks(rotation=45)
plt.show()
```

Gaza	2453
North Gaza	1831
Khan Yunis	1367
Rafah	1097
Deir al-Balah	875
Israel	726
Nablus	621
Jenin	540
Hebron	380
Ramallah and al-Bira	295
Tulkarm	258
Bethlehem	167
East Jerusalem	121
al-Quds	102
Unknown	68
Qalqiliya	66
Tubas	56
Salfit	47
Jericho	34
Gush Katif	16
West Bank	4

Name: place_of_residence_district, dtype: int64



```
district_percentage = data['place_of_residence_district'].value_counts(normalize=True)
mode_district = data['place_of_residence_district'].mode().values[0]
```

```
print(f"District Percentage: {district_percentage}")
print(f"Mode District: {mode_district}")
```

```
District Percentage: Gaza          0.220514
```

```
North Gaza          0.164599
```

```
Khan Yunis         0.122887
```

```
Rafah              0.098616
```

```
Deir al-Balah      0.078659
```

```
Israel              0.065264
```

```
Nablus              0.055825
```

```
Jenin               0.048544
```

```
Hebron              0.034160
```

```
Ramallah and al-Bira 0.026519
```

Tulkarm	0.023193
Bethlehem	0.015013
East Jerusalem	0.010877
al-Quds	0.009169
Unknown	0.006113
Qalqiliya	0.005933
Tubas	0.005034
Salfit	0.004225
Jericho	0.003056
Gush Katif	0.001438
West Bank	0.000360

Name: place_of_residence_district, dtype: float64
Mode District: Gaza

Exploring ammunition attribute

```
ammunition_percentage = data['ammunition'].value_counts(normalize=True)
mode_ammunition = data['ammunition'].mode().values[0]
```

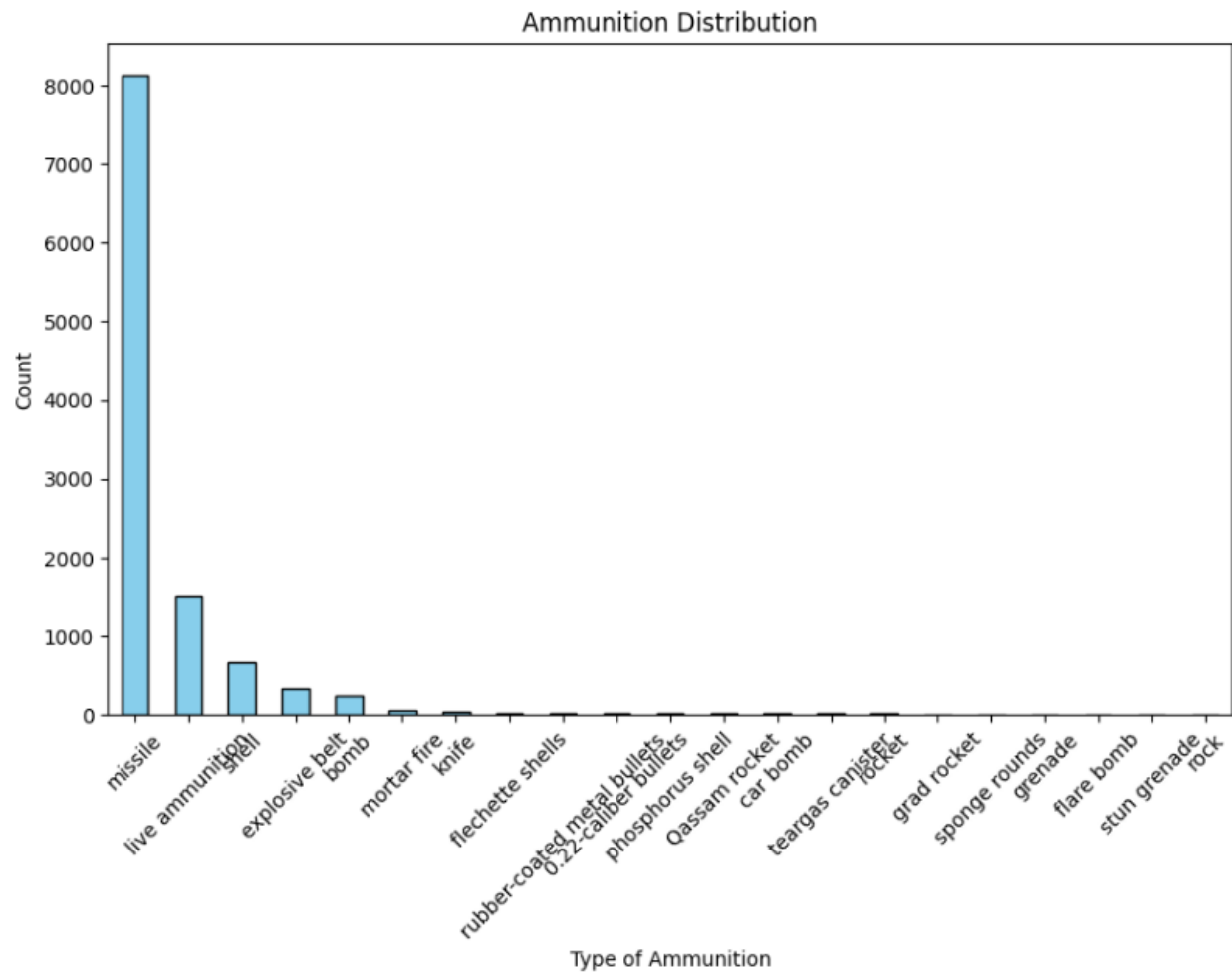
```
print(f"Ammunition Type Percentage: {ammunition_percentage}")
print(f"Mode Ammunition Type: {mode_ammunition}")
```

Ammunition Type Percentage: missile	0.730852
live ammunition	0.136102
shell	0.060680
explosive belt	0.029306
bomb	0.022384
mortar fire	0.004585
knife	0.003326
flechette shells	0.001978
rubber-coated metal bullets	0.001708
0.22-caliber bullets	0.001438
phosphorus shell	0.001438
Qassam rocket	0.001348
car bomb	0.001348
teargas canister	0.001169
rocket	0.001079
grad rocket	0.000629
sponge rounds	0.000180

grenade	0.000180
flare bomb	0.000090
stun grenade	0.000090
rock	0.000090

Name: ammunition, dtype: float64
Mode Ammunition Type: missile

```
ammunition_counts = data['ammunition'].value_counts()
plt.figure(figsize=(10, 6))
ammunition_counts.plot(kind='bar', color='skyblue', edgecolor='black')
plt.xlabel('Type of Ammunition')
plt.ylabel('Count')
plt.title('Ammunition Distribution')
plt.xticks(rotation=45)
plt.show()
```

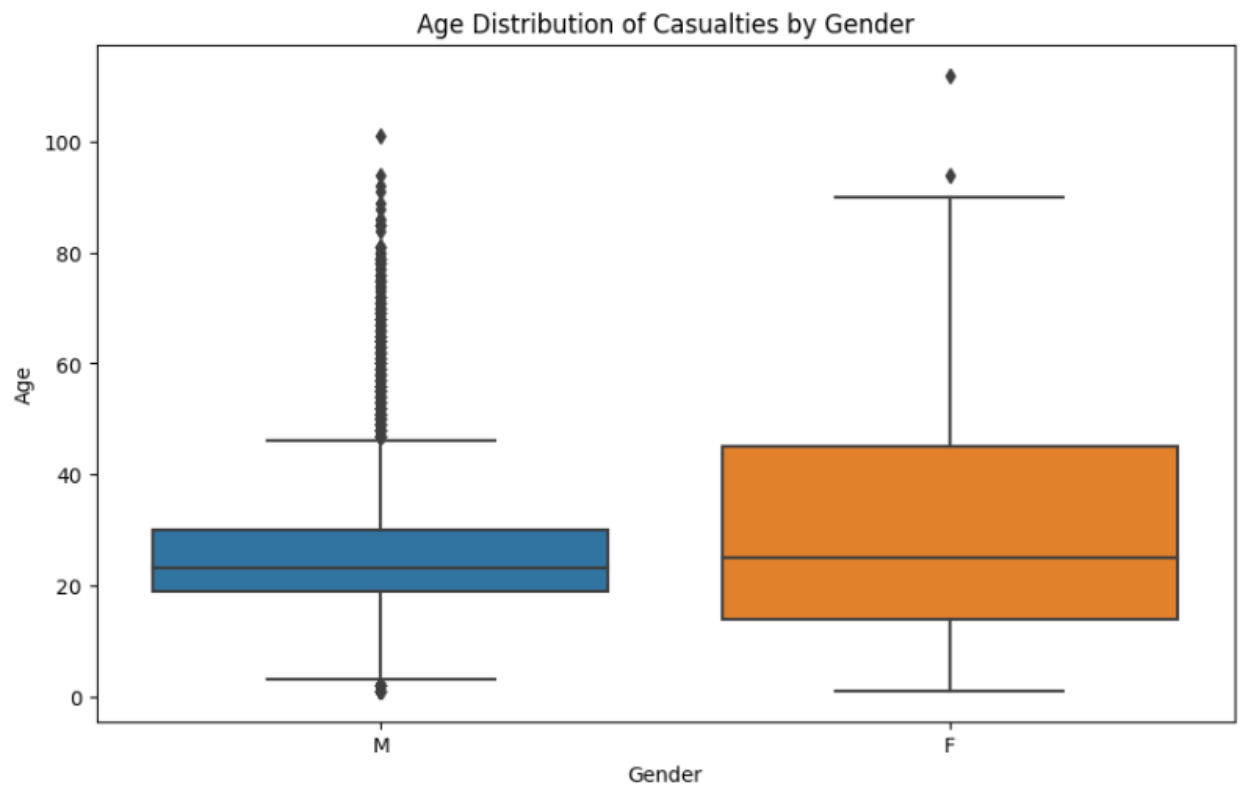


BIVARIATE ANALYSIS

AGE

```
age_by_gender = data.groupby('gender')['age'].describe()
print(age_by_gender)
```

	count	mean	std	min	25%	50%	75%	max
gender								
F	1423.0	30.036543	20.776422	1.0	14.0	25.0	45.0	112.0
M	9701.0	26.213174	12.258501	1.0	19.0	23.0	30.0	101.0



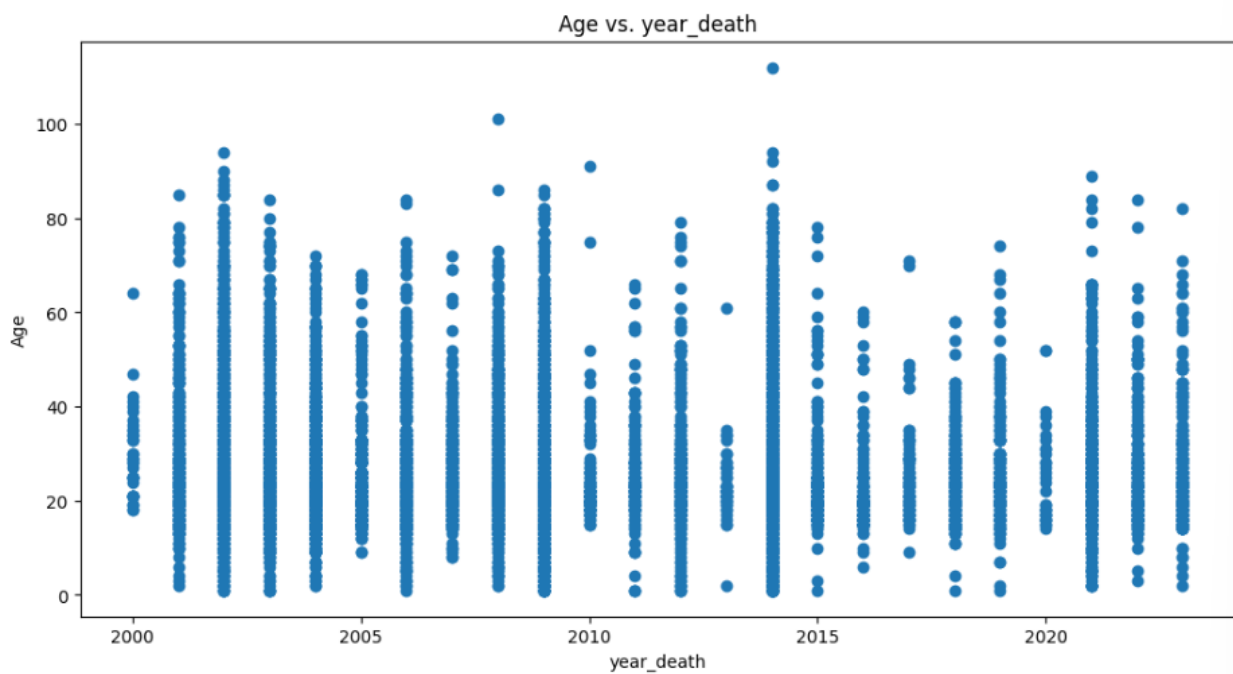
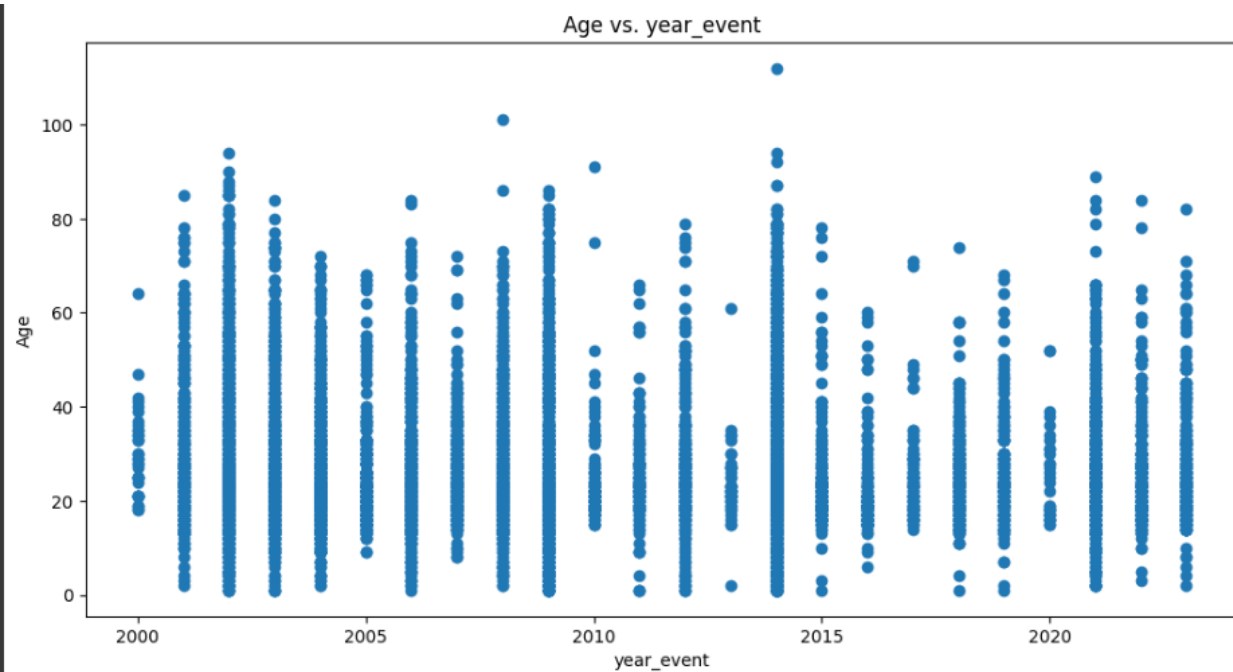
```
# Specify the columns for plotting
columns_to_plot = ['year_event', 'year_death', 'citizenship',
'event_location_district', 'place_of_residence_district']
```

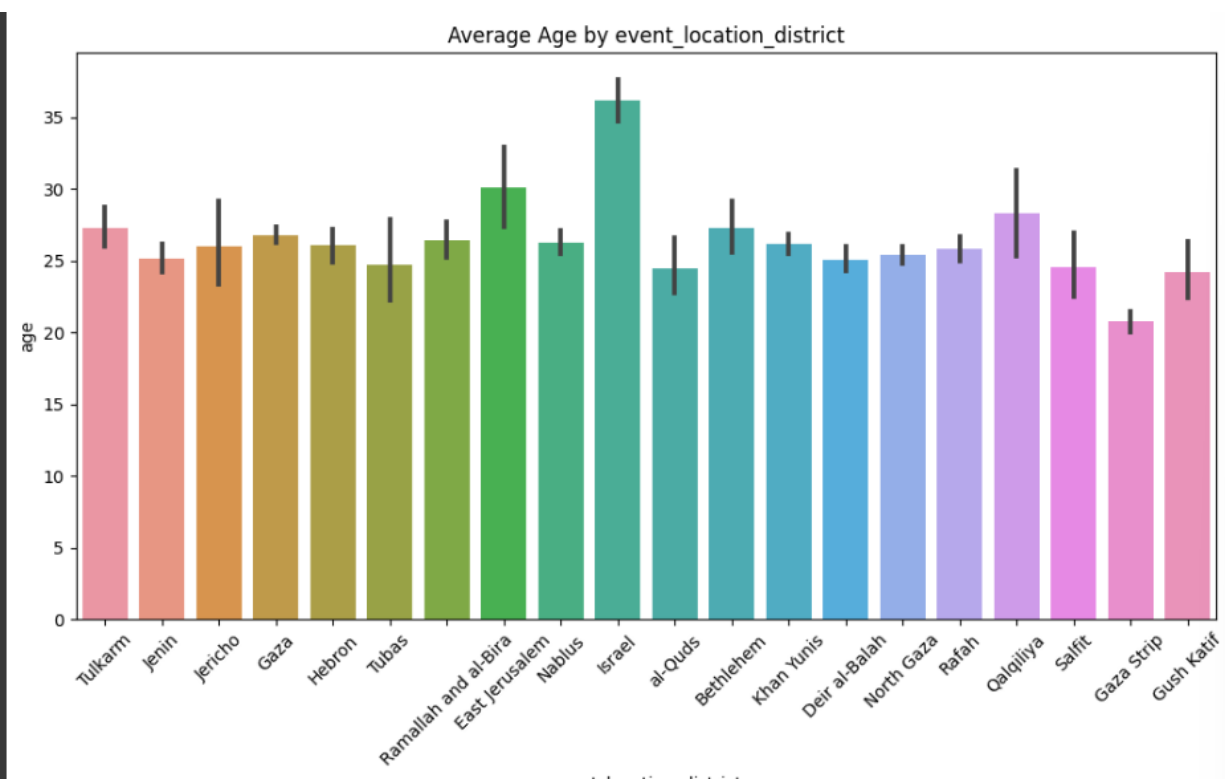
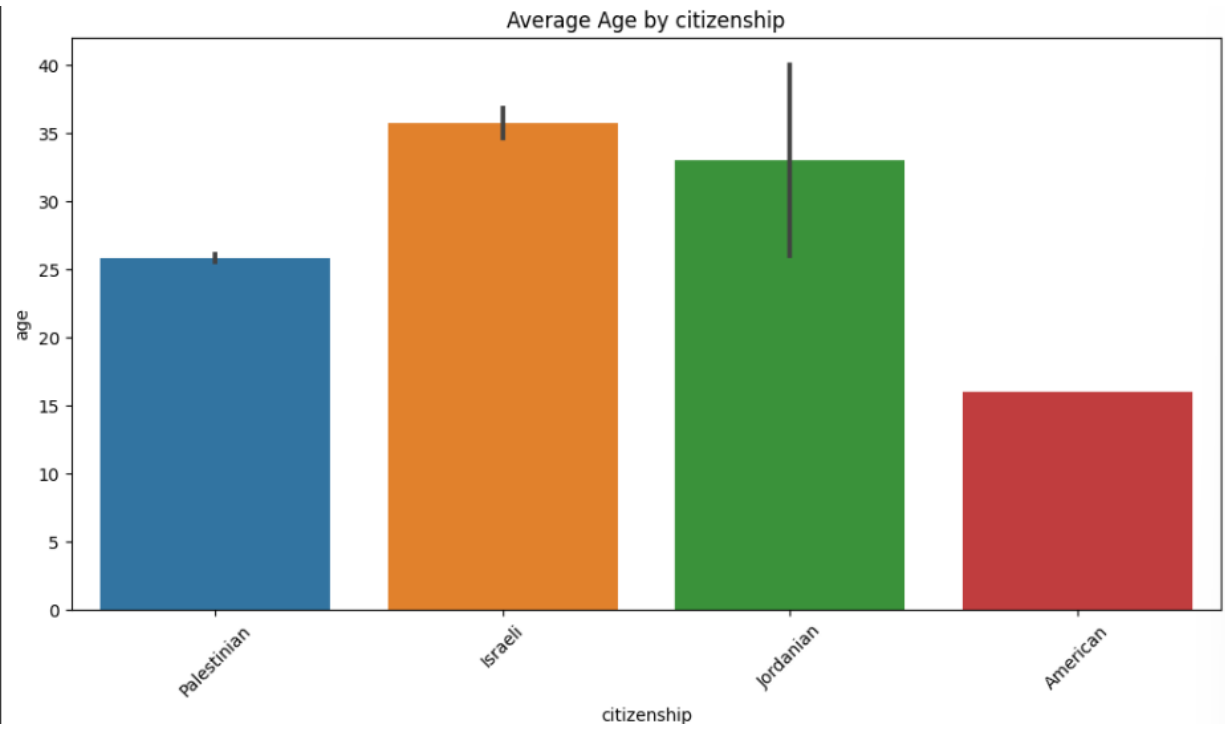
```
for column in columns_to_plot:
    plt.figure(figsize=(12, 6))
```

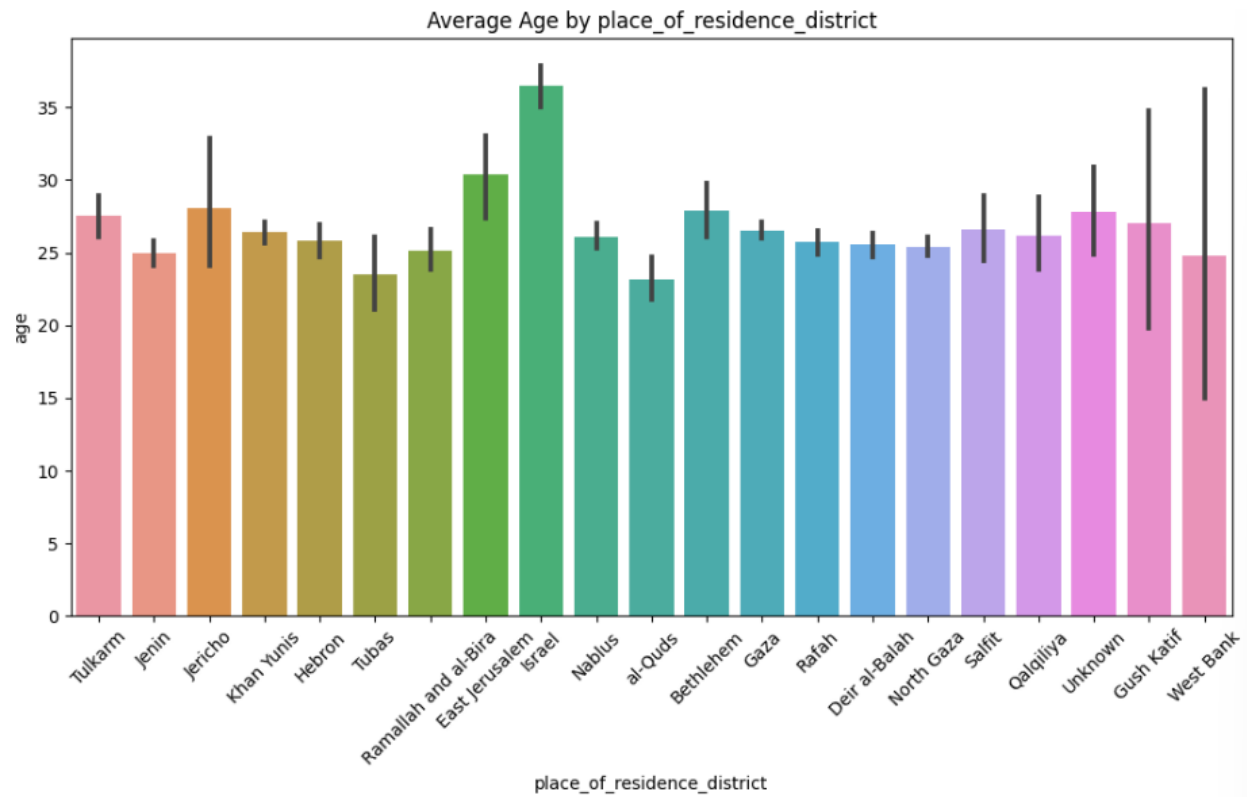
```
    if data[column].dtype == 'object':
        # For categorical data, create a bar chart
        sns.barplot(x=column, y='age', data=data)
        plt.xticks(rotation=45)
        plt.title(f'Average Age by {column}')
```

```
    else:
        # For numerical data, create a scatter plot
        plt.scatter(data[column], data['age'])
        plt.xlabel(column)
        plt.ylabel('Age')
        plt.title(f'Age vs. {column}')
```

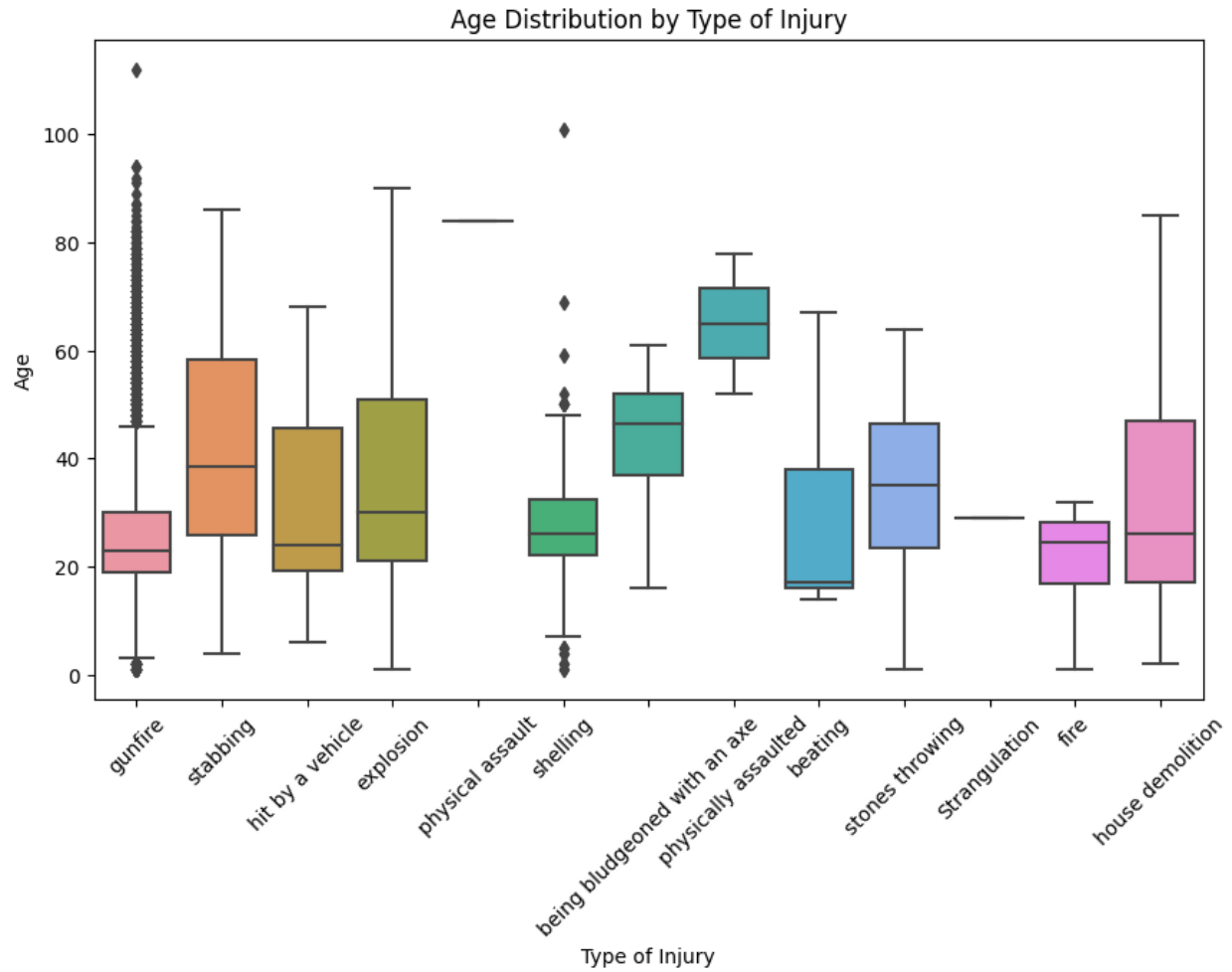
```
plt.show()
```



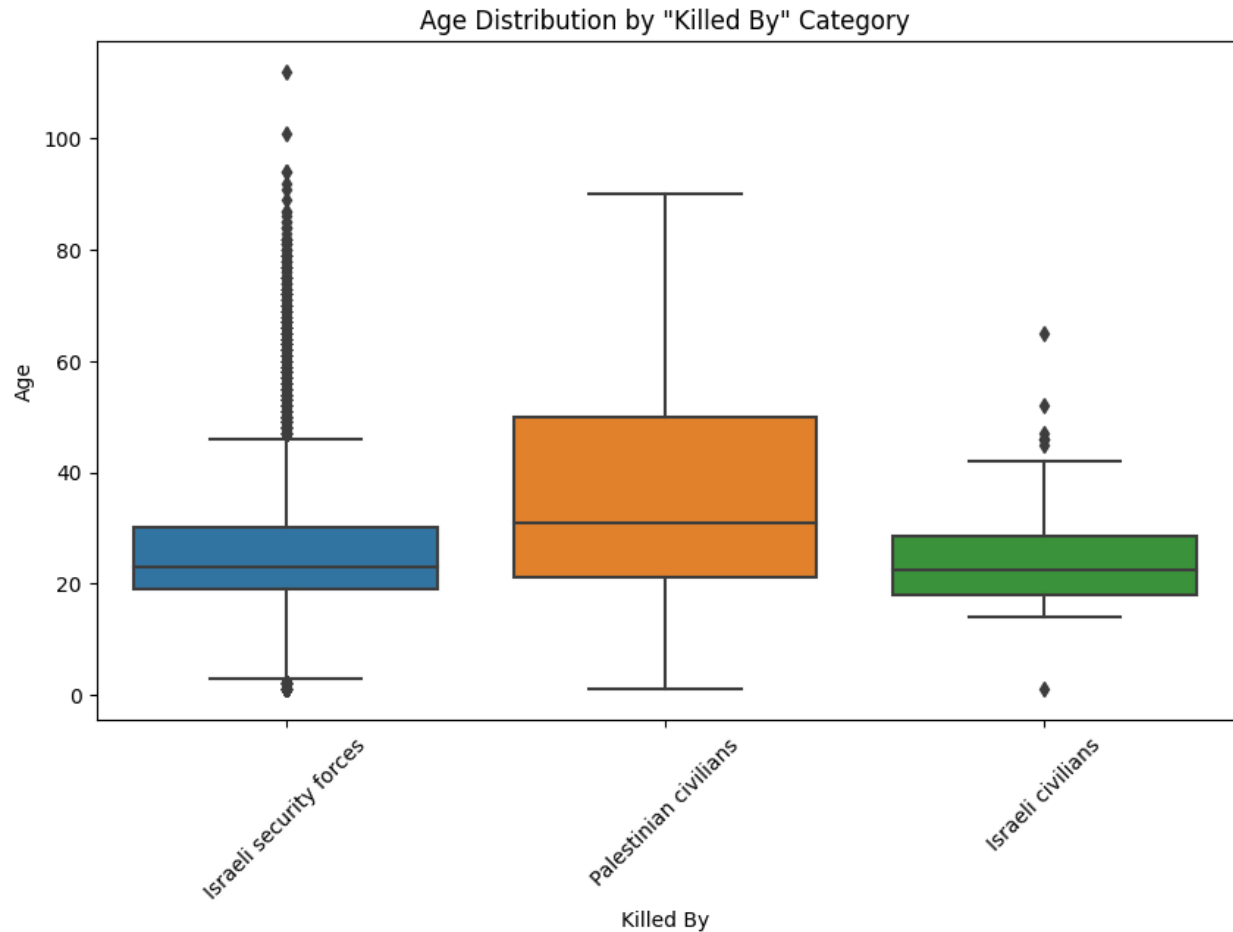




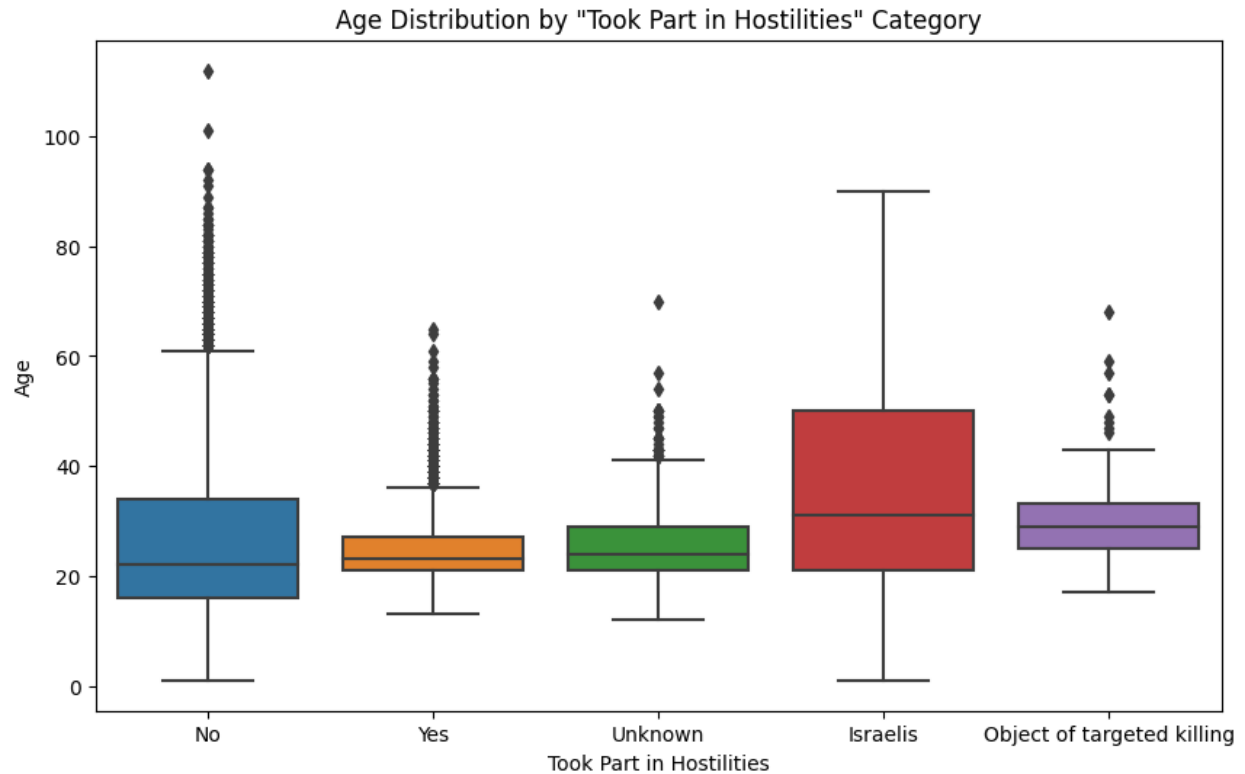
```
# box plot to visualize the age distribution by type of injury
plt.figure(figsize=(10, 6))
sns.boxplot(x='type_of_injury', y='age', data=data)
plt.title('Age Distribution by Type of Injury')
plt.xlabel('Type of Injury')
plt.ylabel('Age')
plt.xticks(rotation=45)
plt.show()
```



```
# box plot to visualize the age distribution by "killed by" category
plt.figure(figsize=(10, 6))
sns.boxplot(x='killed_by', y='age', data=data)
plt.title('Age Distribution by "Killed By" Category')
plt.xlabel('Killed By')
plt.ylabel('Age')
plt.xticks(rotation=45)
plt.show()
```



```
# box plot to visualize the age distribution by "took part in hostilities" category
plt.figure(figsize=(10, 6))
sns.boxplot(x='took_part_in_the_hostilities', y='age', data=data)
plt.title('Age Distribution by "Took Part in Hostilities" Category')
plt.xlabel('Took Part in Hostilities')
plt.ylabel('Age')
plt.show()
```

Citizenship

Examine the geographic distribution of events involving Palestinian and Israeli citizens

```
location_counts_by_citizenship =
```

```
data.groupby('citizenship')['event_location_region'].value_counts()
```

```
print(location_counts_by_citizenship)
```

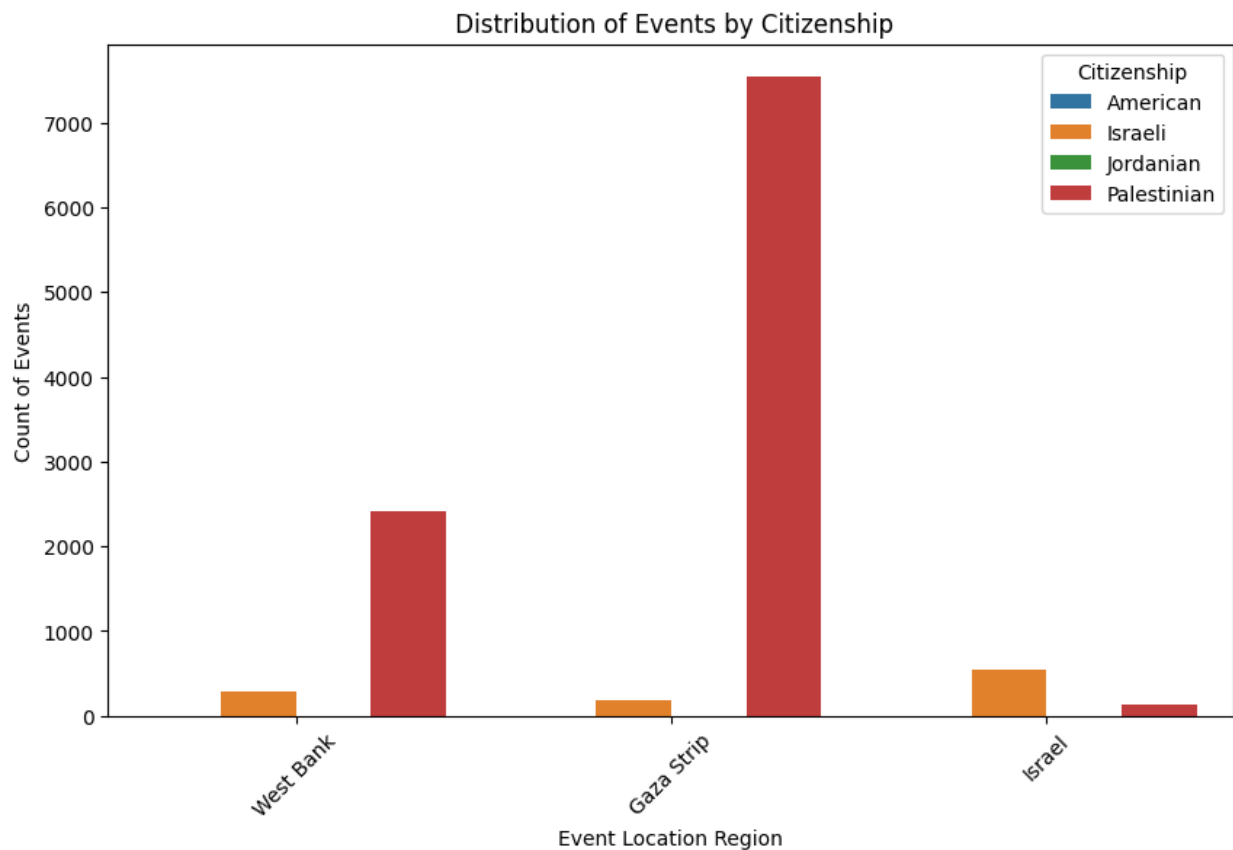
citizenship	event_location_region	count
American	West Bank	1
Israeli	Israel	551
	West Bank	292
	Gaza Strip	186
Jordanian	Gaza Strip	1
	Israel	1
Palestinian	Gaza Strip	7546
	West Bank	2419

Israel 127

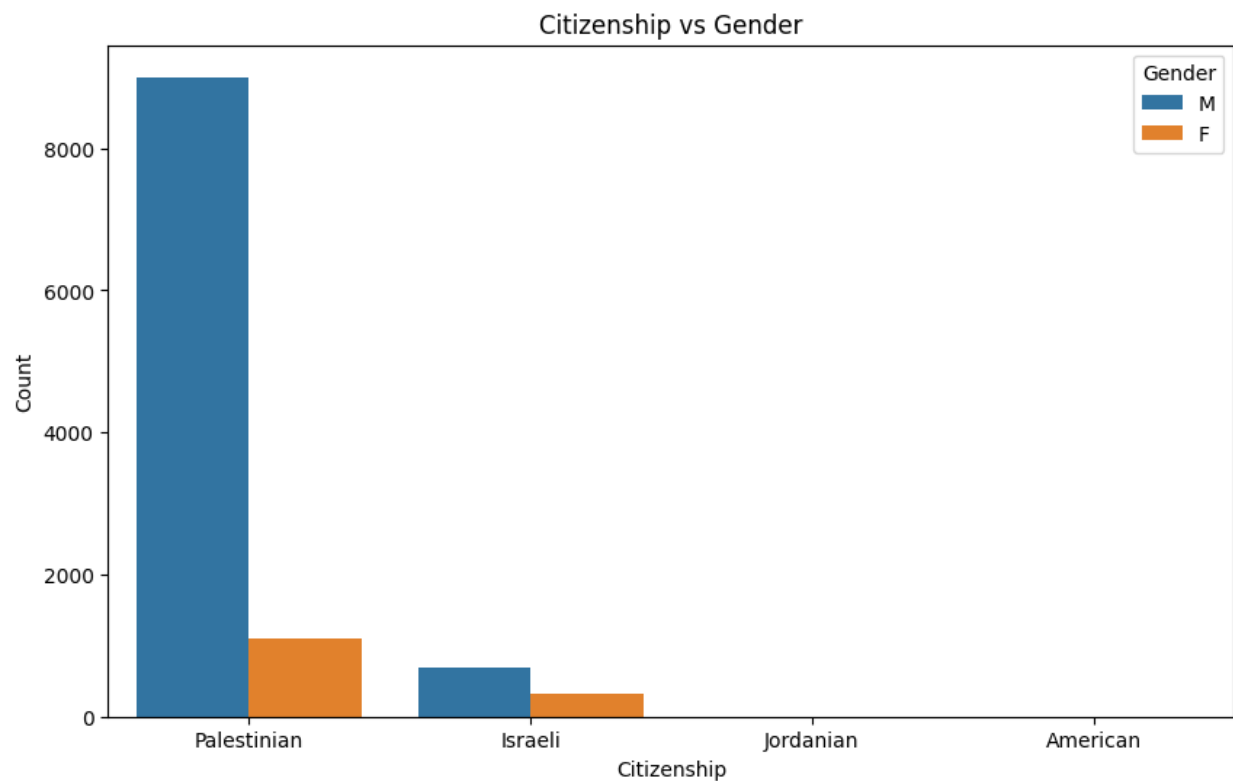
Name: event_location_region, dtype: int64

```
# Calculate the count of events in different regions by citizenship
location_counts_by_citizenship = data.groupby(['citizenship',
'event_location_region']).size().reset_index(name='count')
```

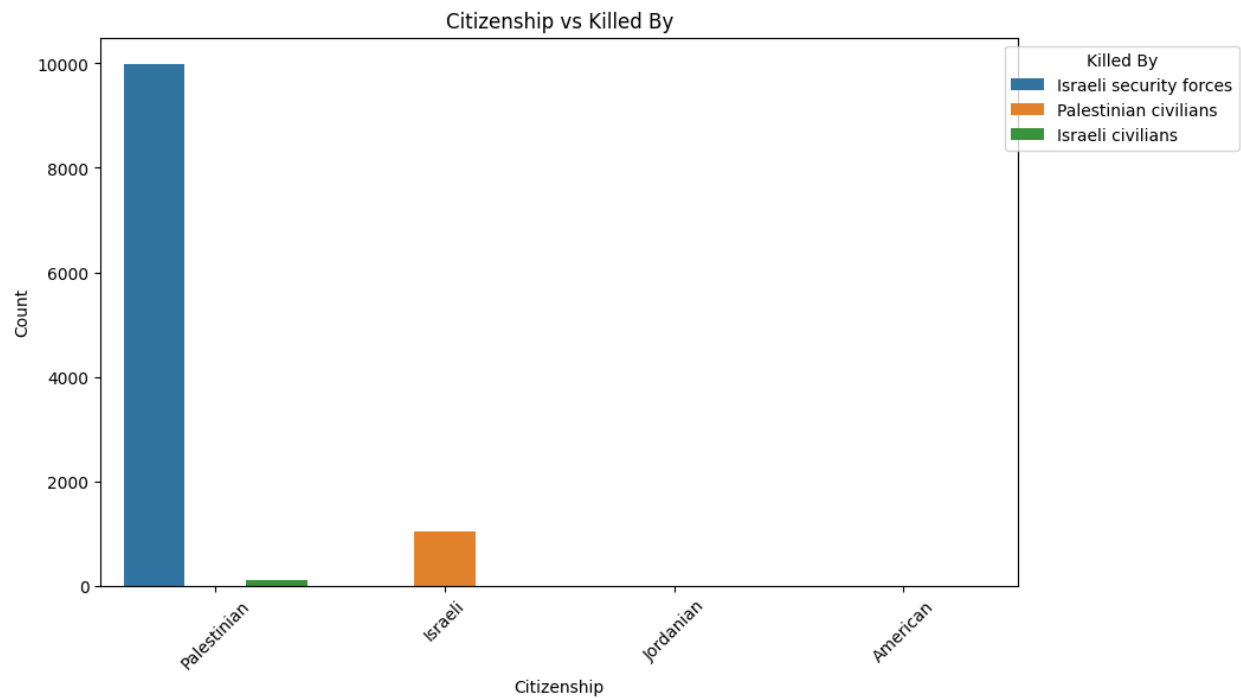
```
plt.figure(figsize=(10, 6))
sns.barplot(x='event_location_region', y='count', hue='citizenship',
data=location_counts_by_citizenship)
plt.title(' Distribution of Events by Citizenship')
plt.xlabel('Event Location Region')
plt.ylabel('Count of Events')
plt.xticks(rotation=45)
plt.legend(title='Citizenship')
plt.show()
```



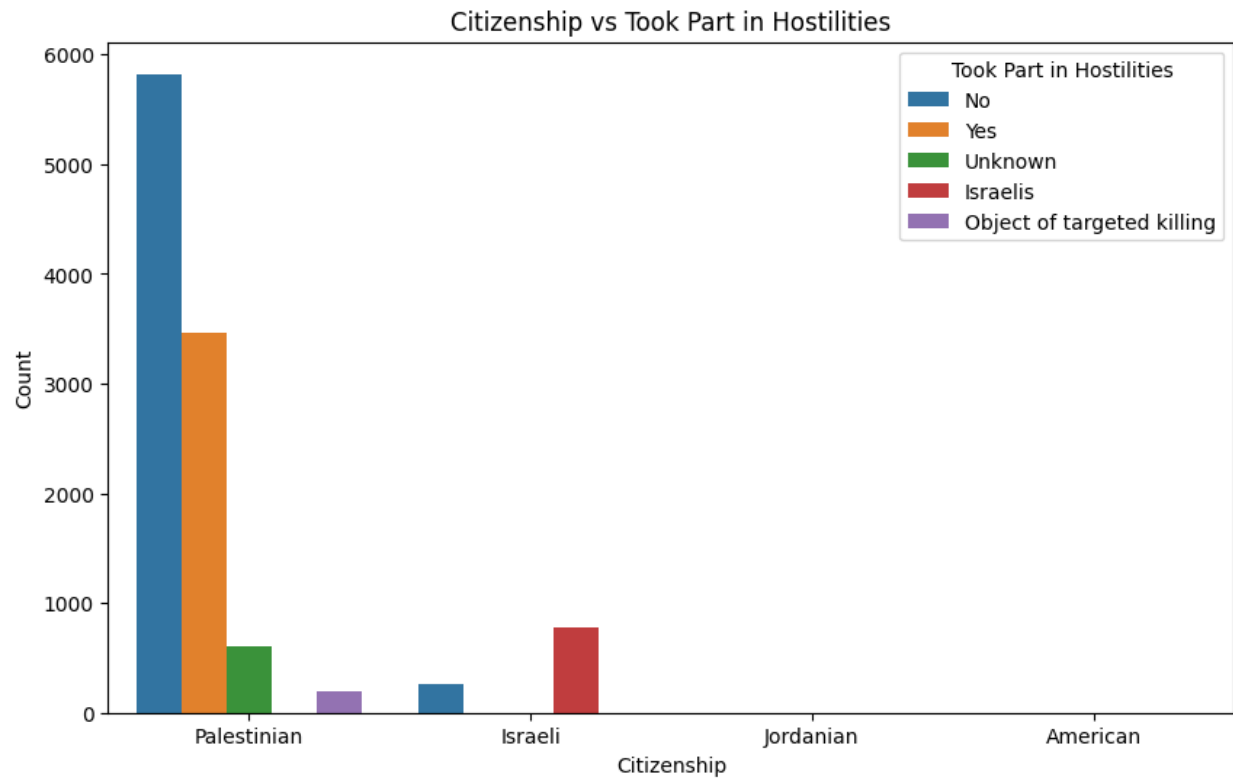
```
# Create a clustered bar plot to visualize "citizenship" vs "gender"
plt.figure(figsize=(10, 6))
sns.countplot(x='citizenship', hue='gender', data=data)
plt.title('Citizenship vs Gender')
plt.xlabel('Citizenship')
plt.ylabel('Count')
plt.legend(title='Gender')
plt.show()
```



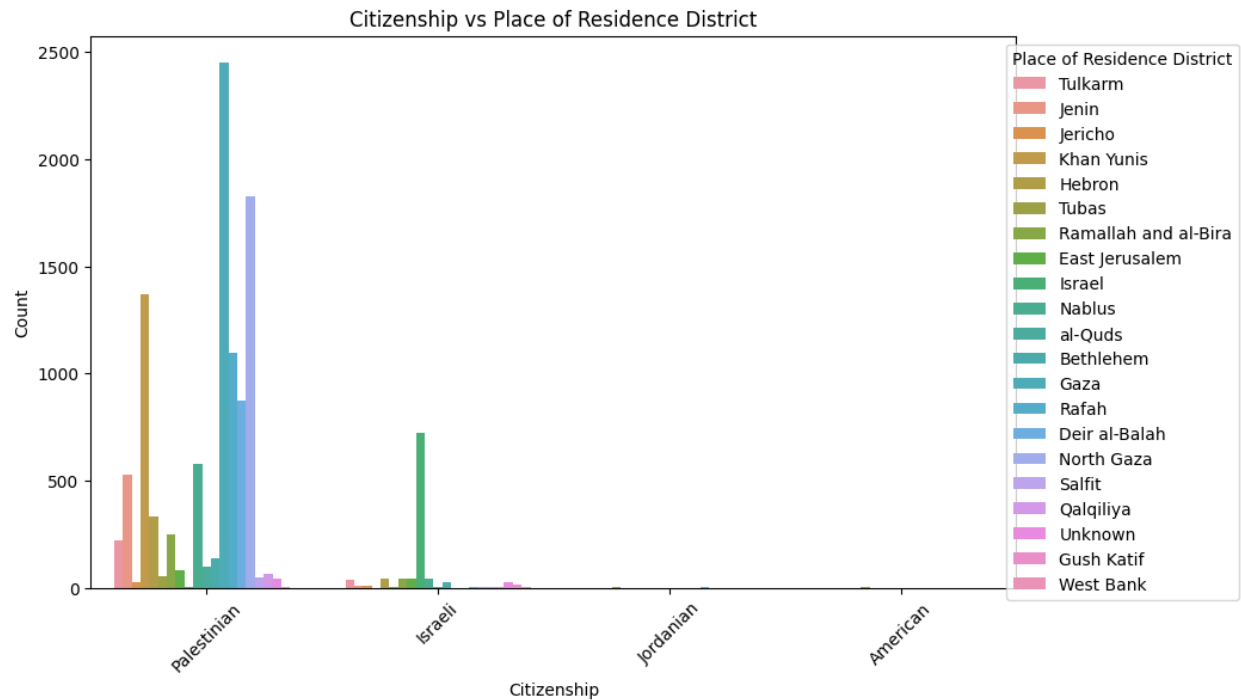
```
plt.figure(figsize=(10, 6))
sns.countplot(x='citizenship', hue='killed_by', data=data)
plt.title('Citizenship vs Killed By')
plt.xlabel('Citizenship')
plt.ylabel('Count')
plt.legend(title='Killed By', loc='upper right', bbox_to_anchor=(1.25, 1))
plt.xticks(rotation=45)
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.countplot(x='citizenship', hue='took_part_in_the_hostilities', data=data)
plt.title('Citizenship vs Took Part in Hostilities')
plt.xlabel('Citizenship')
plt.ylabel('Count')
plt.legend(title='Took Part in Hostilities')
plt.show()
```



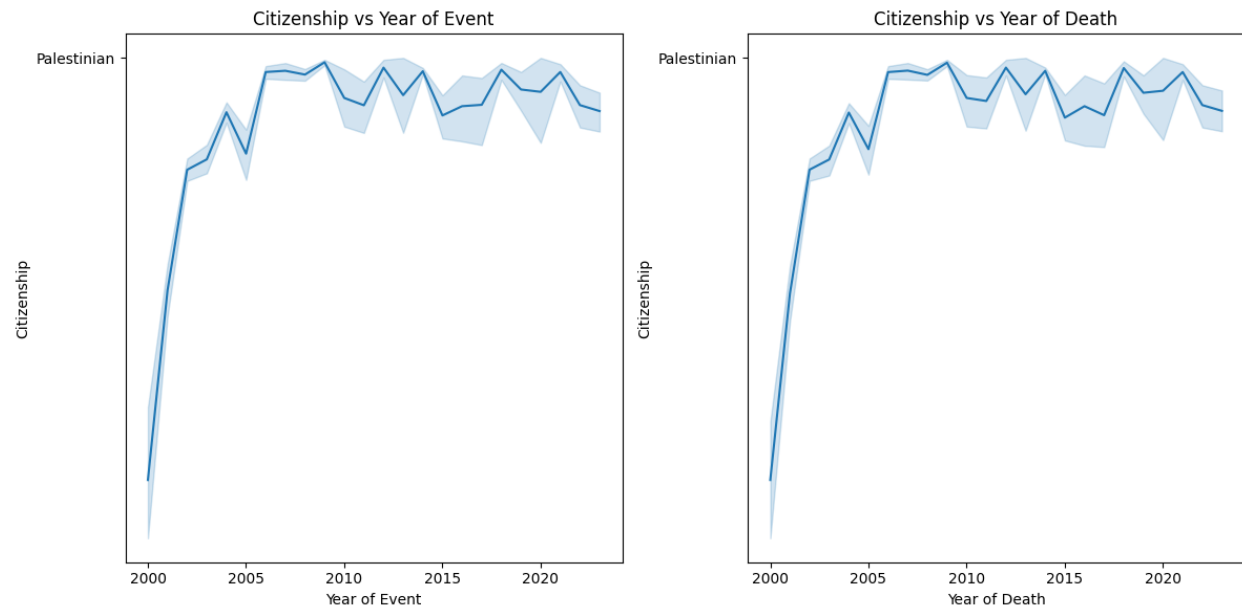
```
plt.figure(figsize=(10, 6))
sns.countplot(x='citizenship', hue='place_of_residence_district', data=data)
plt.title('Citizenship vs Place of Residence District')
plt.xlabel('Citizenship')
plt.ylabel('Count')
plt.legend(title='Place of Residence District', loc='upper right', bbox_to_anchor=(1.25, 1))
plt.xticks(rotation=45)
plt.show()
```



```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
sns.lineplot(x='year_event', y='citizenship', data=data)
plt.title('Citizenship vs Year of Event')
plt.xlabel('Year of Event')
plt.ylabel('Citizenship')

plt.subplot(1, 2, 2)
sns.lineplot(x='year_death', y='citizenship', data=data)
plt.title('Citizenship vs Year of Death')
plt.xlabel('Year of Death')
plt.ylabel('Citizenship')

plt.tight_layout()
plt.show()
```



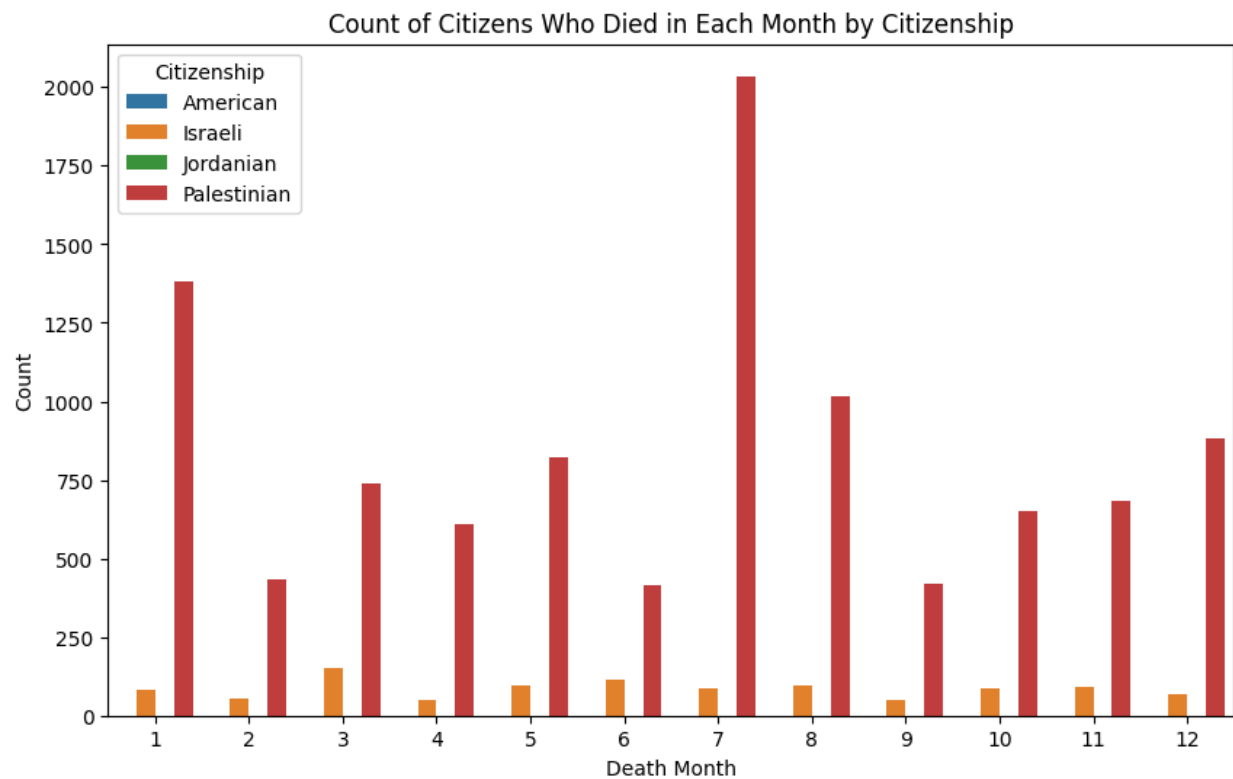
```
death_counts = data.groupby(['citizenship',
'deathevent_month']).size().reset_index(name='count')
```

```
print(death_counts)
```

```
plt.figure(figsize=(10, 6))
sns.barplot(x='deathevent_month', y='count', hue='citizenship', data=death_counts)
plt.title('Count of Citizens Who Died in Each Month by Citizenship')
plt.xlabel('Death Month')
plt.ylabel('Count')
plt.legend(title='Citizenship')
plt.show()
```

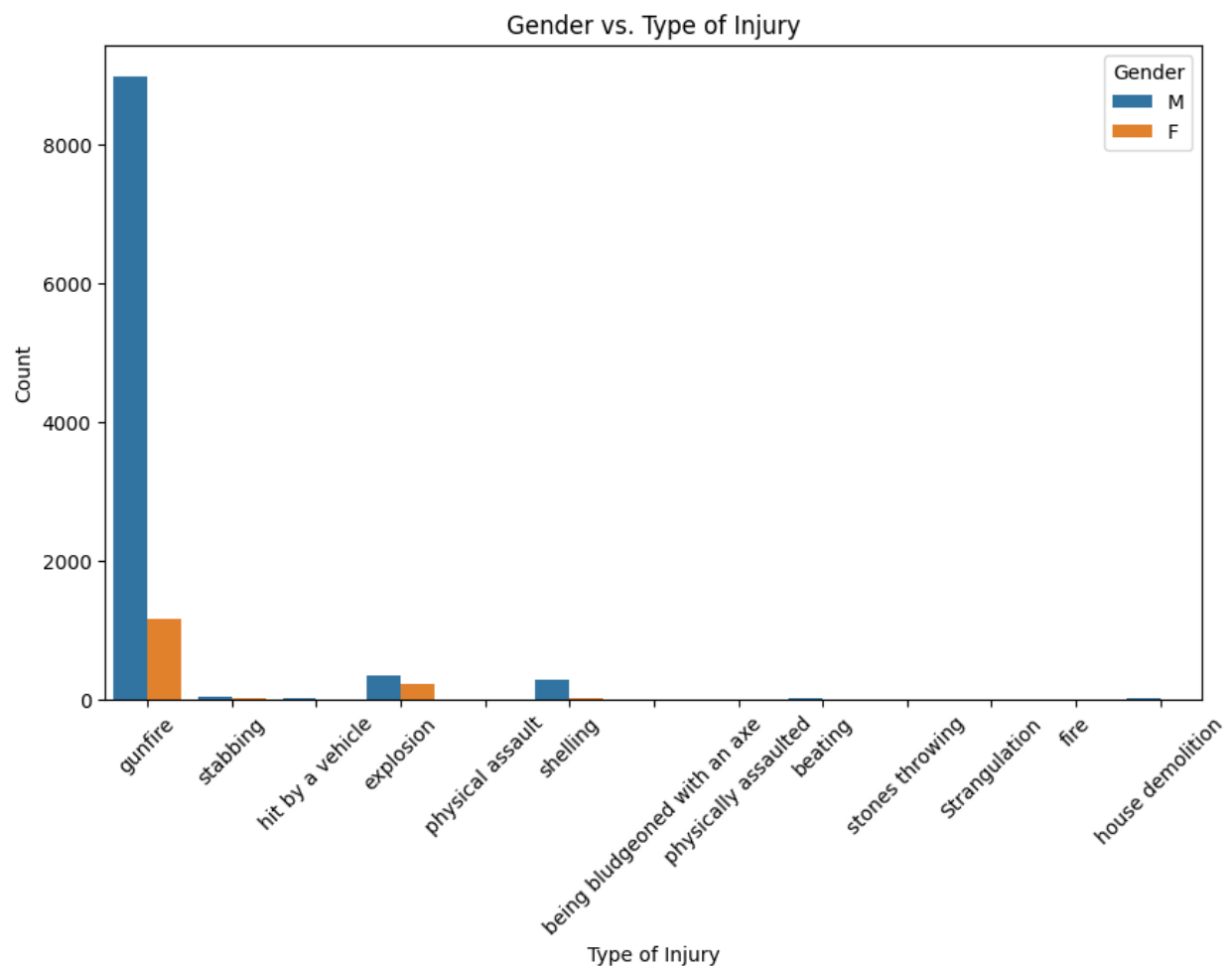
	citizenship	deathevent_month	count
0	American	2	1
1	Israeli	1	81
2	Israeli	2	54
3	Israeli	3	153
4	Israeli	4	51
5	Israeli	5	98
6	Israeli	6	115
7	Israeli	7	87

8	Israeli	8	95
9	Israeli	9	51
10	Israeli	10	85
11	Israeli	11	89
12	Israeli	12	70
13	Jordanian	4	1
14	Jordanian	7	1
15	Palestinian	1	1381
16	Palestinian	2	434
17	Palestinian	3	741
18	Palestinian	4	611
19	Palestinian	5	823
20	Palestinian	6	413
21	Palestinian	7	2035
22	Palestinian	8	1018
23	Palestinian	9	419
24	Palestinian	10	651
25	Palestinian	11	682
26	Palestinian	12	884



Type of Hostilities

```
plt.figure(figsize=(10, 6))
sns.countplot(x='type_of_injury', hue='gender', data=data)
plt.title('Gender vs. Type of Injury')
plt.xlabel('Type of Injury')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='Gender')
plt.show()
```

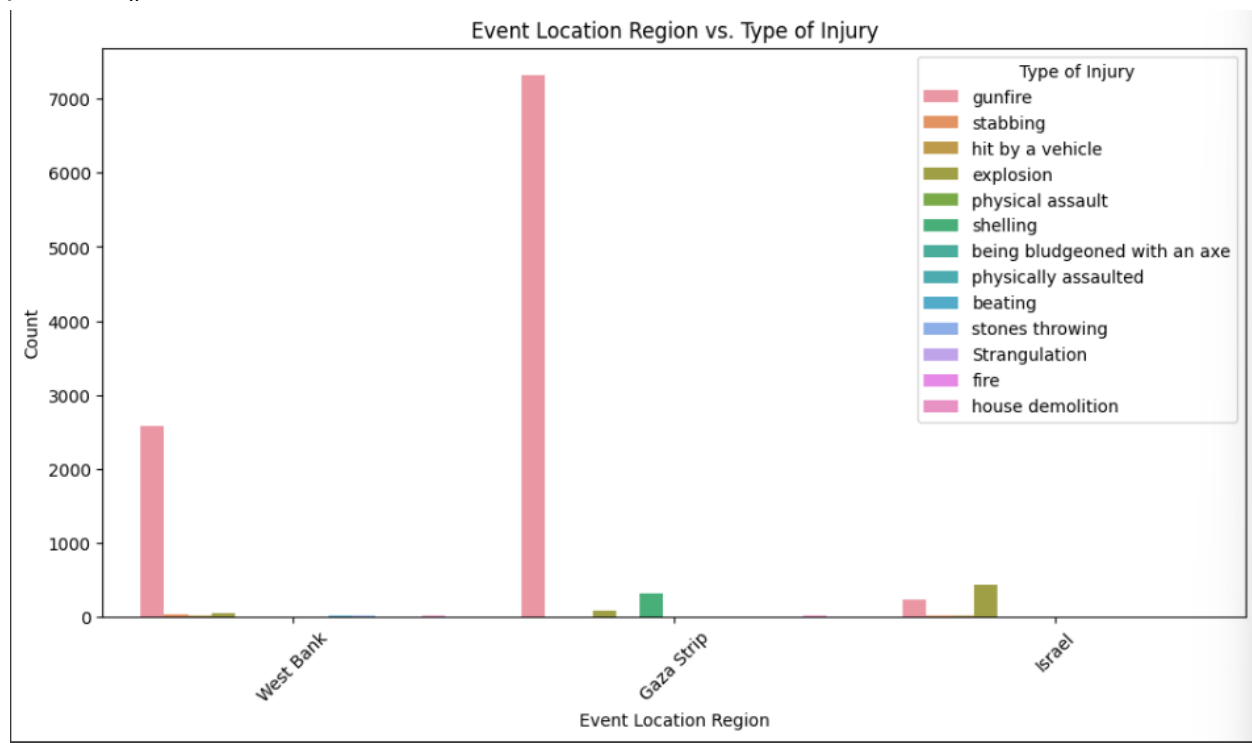


```
plt.figure(figsize=(12, 6))
```

```

sns.countplot(x='event_location_region', hue='type_of_injury', data=data)
plt.title('Event Location Region vs. Type of Injury')
plt.xlabel('Event Location Region')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='Type of Injury')
plt.show()

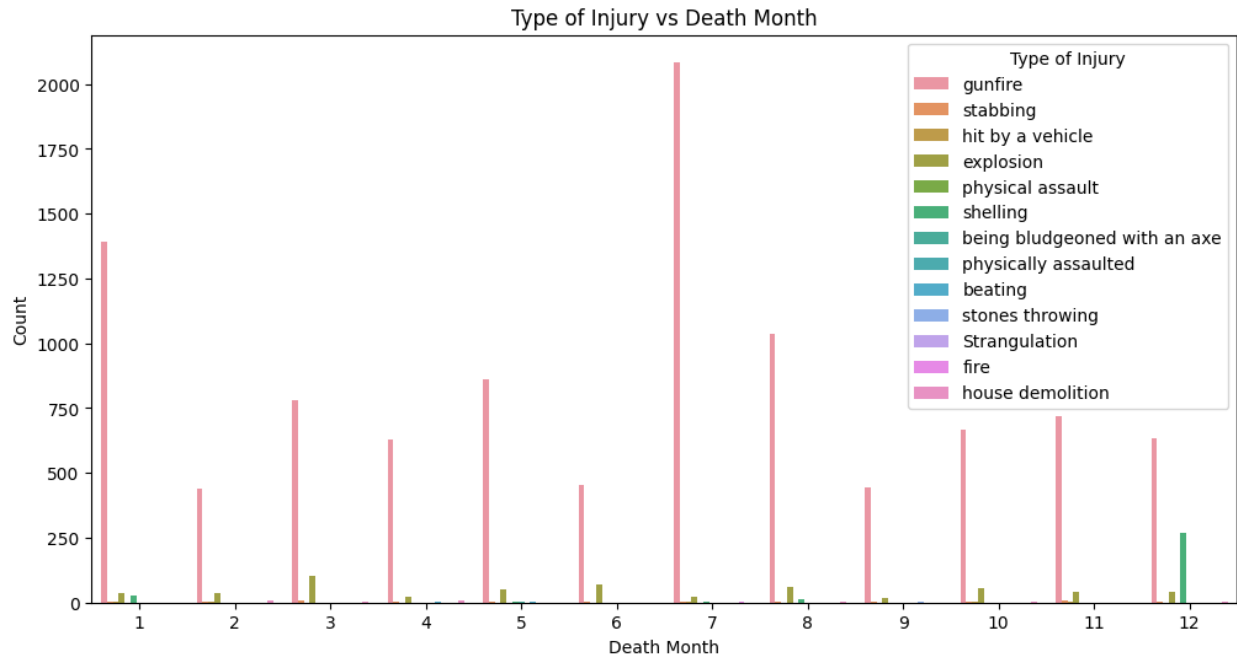
```



```

# Create a grouped bar plot to visualize "type_of_injury" vs "death_month"
plt.figure(figsize=(12, 6))
sns.countplot(x='death_month', hue='type_of_injury', data=data)
plt.title('Type of Injury vs Death Month')
plt.xlabel('Death Month')
plt.ylabel('Count')
plt.legend(title='Type of Injury')
plt.show()

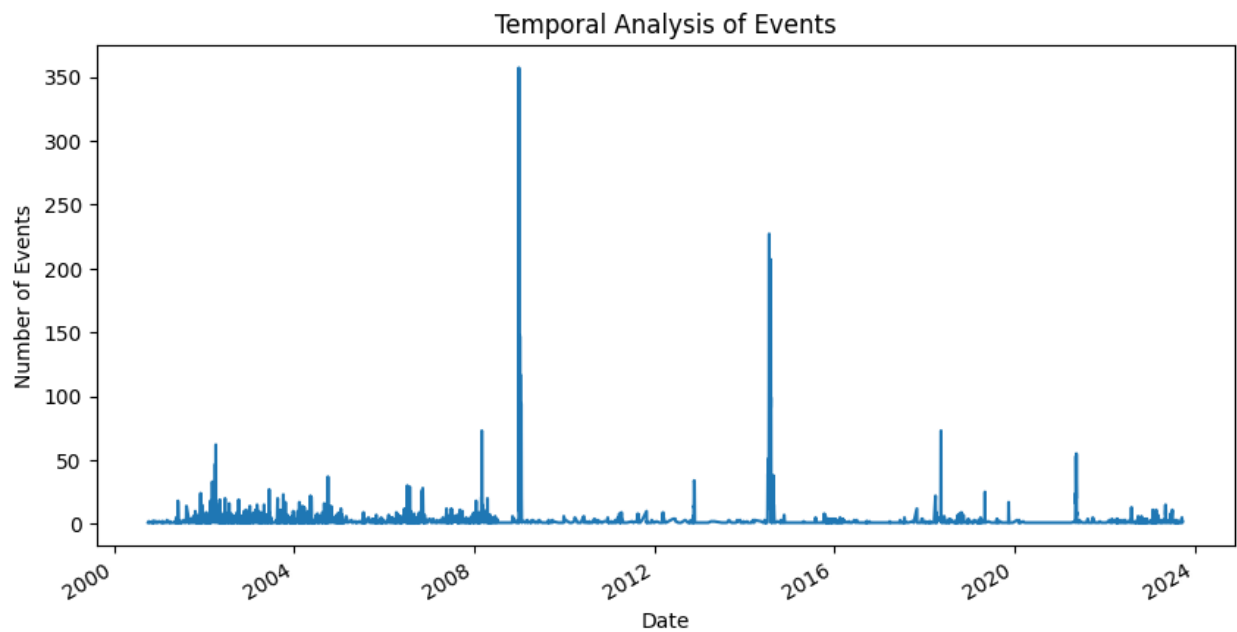
```



```

event_counts.plot(kind='line', figsize=(10, 5))
plt.xlabel('Date')
plt.ylabel('Number of Events')
plt.title('Temporal Analysis of Events')
plt.show()

```



```

data['date_of_death'] = pd.to_datetime(data['date_of_death'])
data['date_of_event'] = pd.to_datetime(data['date_of_event'])

deaths_on_event_day = data[data['date_of_death'].dt.date ==
data['date_of_event'].dt.date]

count_deaths_on_event_day = len(deaths_on_event_day)

print(f"Number of people who died on the same day as the event:
{count_deaths_on_event_day}")

count_no_deaths_on_event_day = len(data)- count_deaths_on_event_day

print(f"Number of people who did NOT die on the same day as the event:
{count_no_deaths_on_event_day}")

Number of people who died on the same day as the event: 10184
Number of people who did NOT die on the same day as the event: 940

```

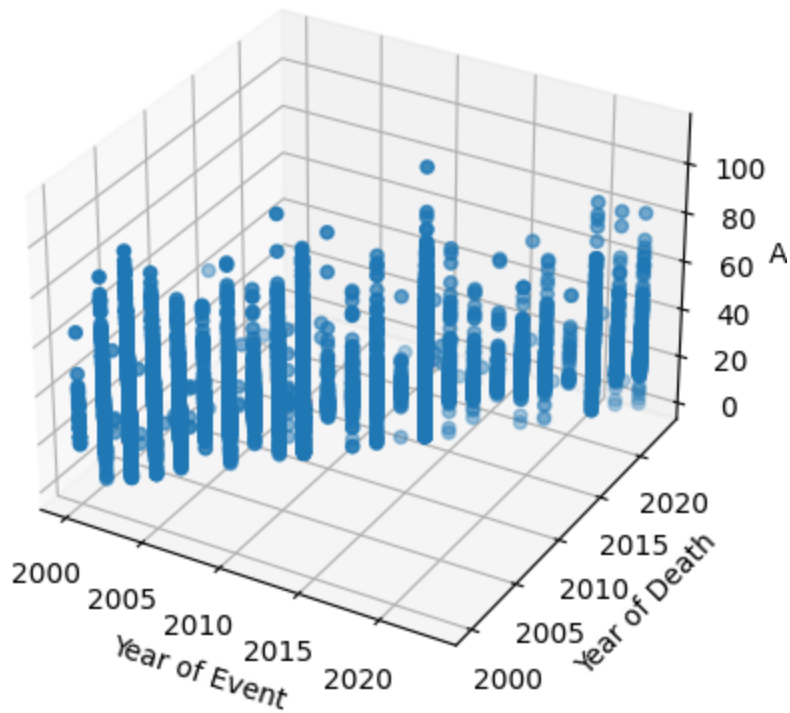
MULTIVARIATE ANALYSIS

```

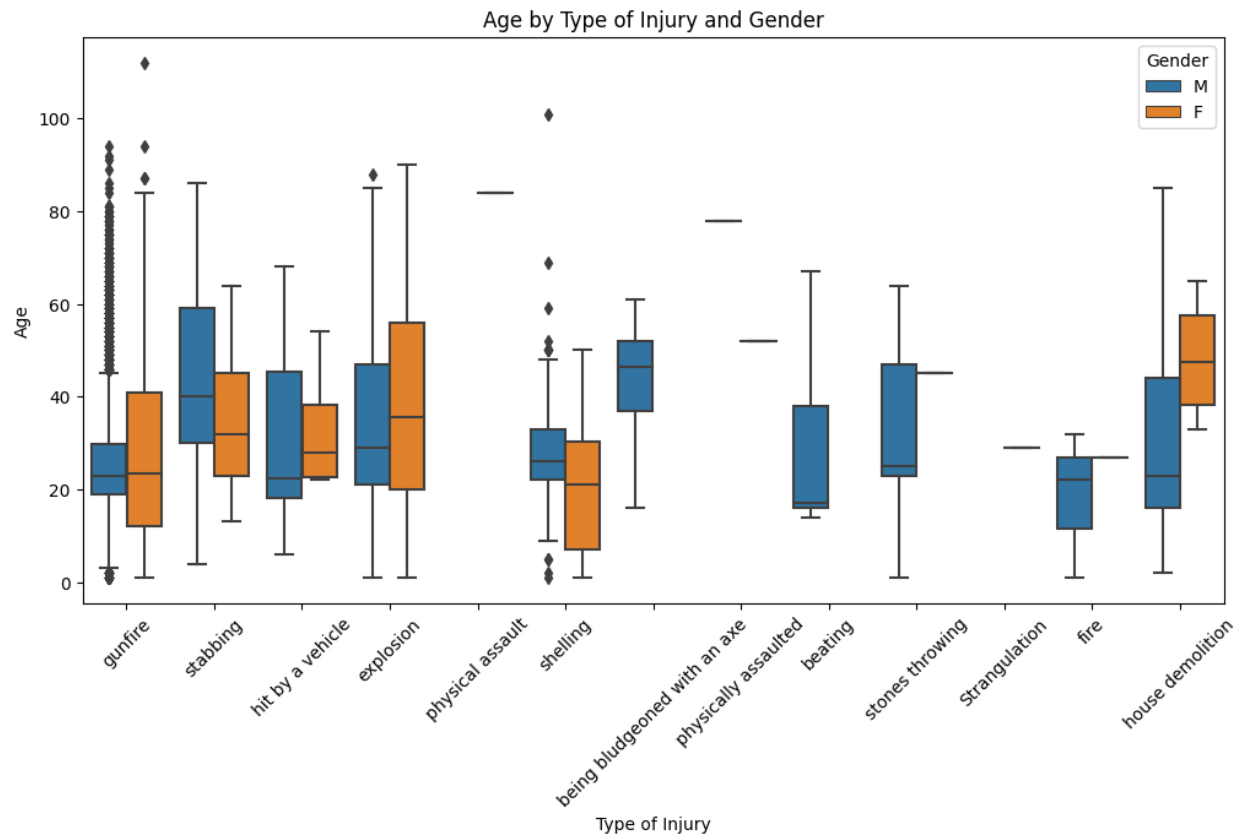
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(data['year_event'], data['year_death'], data['age'])
ax.set_xlabel('Year of Event')
ax.set_ylabel('Year of Death')
ax.set_zlabel('Age')
plt.show()

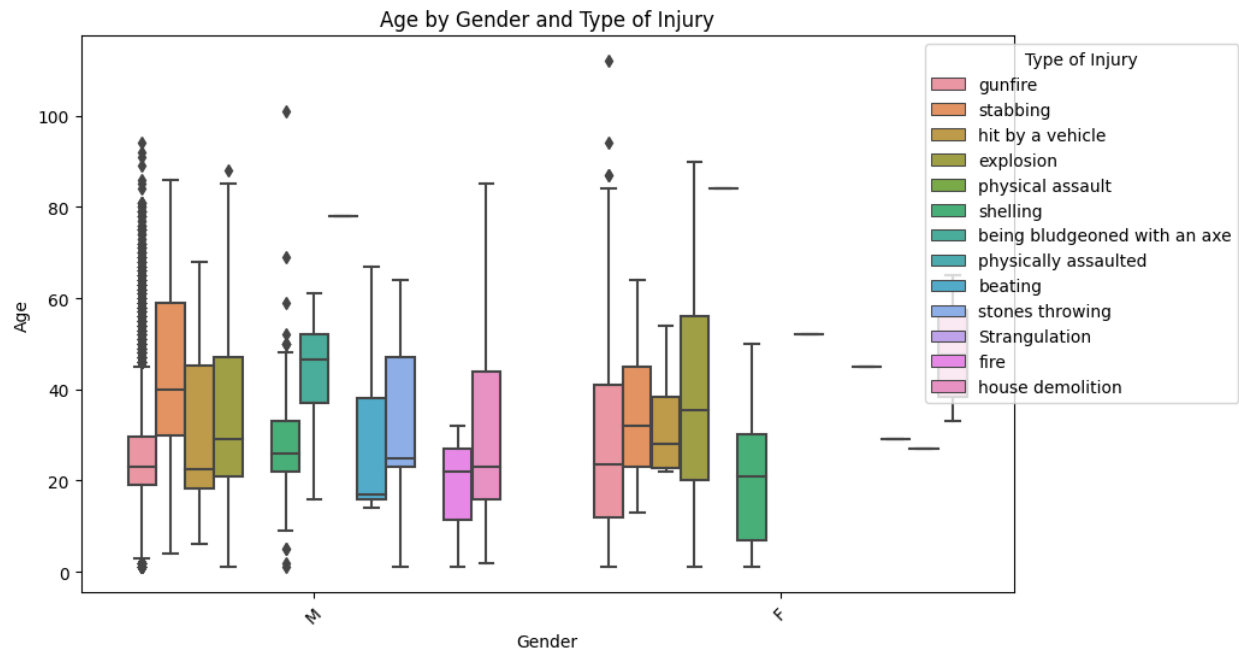
```



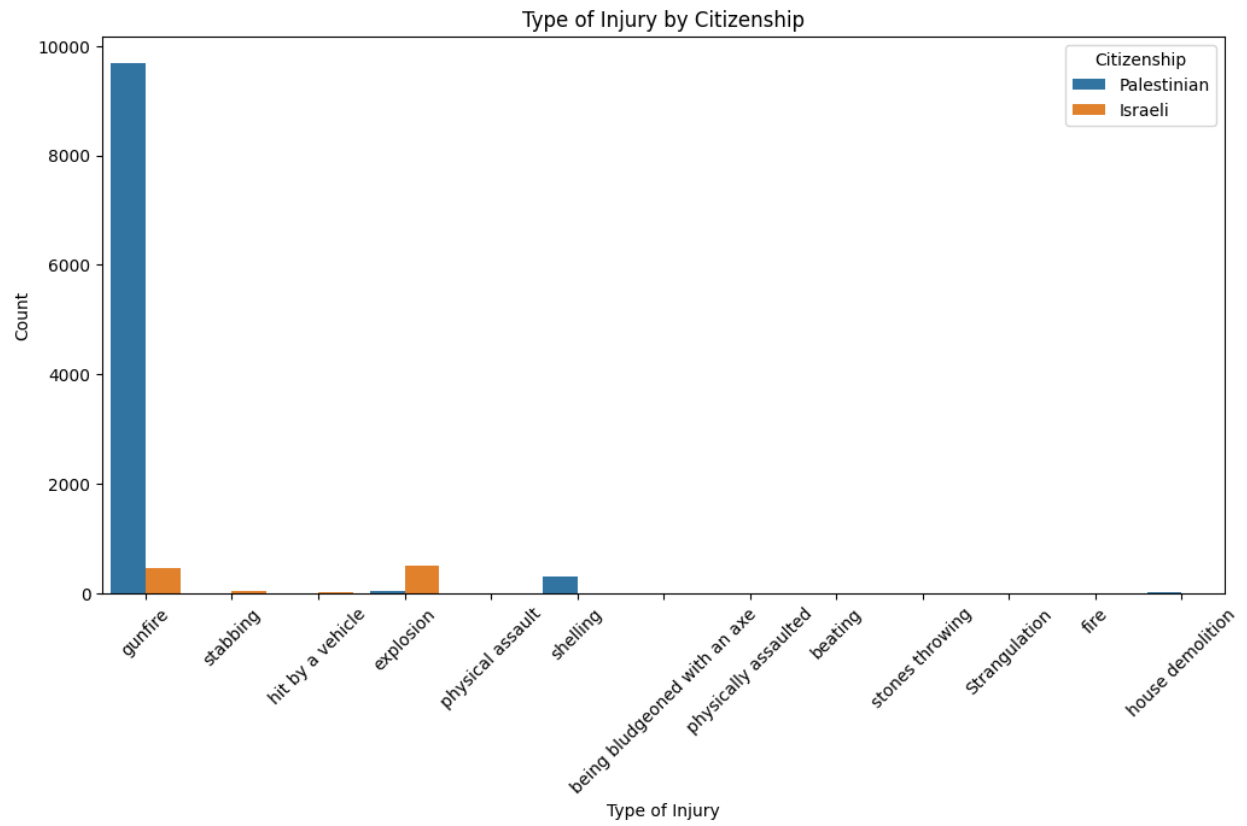
```
plt.figure(figsize=(12, 6))
sns.boxplot(x='type_of_injury', y='age', hue='gender', data=data)
plt.title('Age by Type of Injury and Gender')
plt.xlabel('Type of Injury')
plt.ylabel('Age')
plt.xticks(rotation=45)
plt.legend(title='Gender')
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.boxplot(x='gender', y='age', hue='type_of_injury', data=data)
plt.title('Age by Gender and Type of Injury')
plt.xlabel('Gender')
plt.ylabel('Age')
plt.xticks(rotation=45)
plt.legend(title='Type of Injury', loc='upper right', bbox_to_anchor=(1.25, 1))
plt.show()
```



```
plt.figure(figsize=(12, 6))
sns.countplot(x='type_of_injury', hue='citizenship', data=data, hue_order=['Palestinian',
'Israeli'])
plt.title('Type of Injury by Citizenship')
plt.xlabel('Type of Injury')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='Citizenship')
plt.show()
```



SIMPSON'S PARADOX

```
from scipy.stats import chi2_contingency

observed_data = pd.crosstab(index=data['type_of_injury'], columns=data['ammunition'])
initial_chi2, initial_p, _, _ = chi2_contingency(observed_data)

killed_by_groups = data['killed_by'].unique()

results = []

for killed_by in killed_by_groups:
    subgroup_data = data[data['killed_by'] == killed_by]

    if len(subgroup_data) >= 2:

        observed_data_subgroup = pd.crosstab(index=subgroup_data['type_of_injury'],
        columns=subgroup_data['ammunition'])
        chi2, p, _, _ = chi2_contingency(observed_data_subgroup)

        results.append({'Killed By': killed_by, 'Chi-Squared Statistic': chi2, 'P-value': p})

results_df = pd.DataFrame(results)

print("Initial Chi-Squared Statistic:", initial_chi2)
print("Chi-Squared Statistics within 'Killed By' Subgroups:")
print(results_df)

if all(results_df['P-value'] >= 0.05):
    print("No Simpson's Paradox detected. Associations are consistent across 'Killed By'
    subgroups.")
else:
    print("Simpson's Paradox may be present. Associations change within 'Killed By'
    subgroups.")

Initial Chi-Squared Statistic: 23182.11758722398
Chi-Squared Statistics within 'Killed By' Subgroups:
      Killed By  Chi-Squared Statistic    P-value
```

```
0 Israeli security forces      2111.398476  0.000000e+00
1 Palestinian civilians      2744.385421  0.000000e+00
2 Israeli civilians      102.139535  1.558741e-18
```

Simpson's Paradox may be present. Associations change within 'Killed By' subgroups.

```
from scipy.stats import chi2_contingency

observed_data = pd.crosstab(index=data['year_death'], columns=data['killed_by'])
initial_chi2, initial_p, _, _ = chi2_contingency(observed_data)

print("Initial Chi-Squared Statistic:", initial_chi2)

event_location_groups = data['event_location'].unique()

results = []

for event_location in event_location_groups:
    subgroup_data = data[data['event_location'] == event_location]

    observed_data_subgroup = pd.crosstab(index=[subgroup_data['year_death'],
subgroup_data['killed_by']], columns=subgroup_data['event_location'])

    chi2_subgroup, p_subgroup, _, _ = chi2_contingency(observed_data_subgroup)

    results.append({'Event Location': event_location, 'Chi-Squared Statistic':
chi2_subgroup, 'P-value': p_subgroup})

print("Chi-Squared Statistics within 'Event Location' Subgroups:")
for result in results:
    print(f"Event Location: {result['Event Location']}, Chi-Squared Statistic:
{result['Chi-Squared Statistic']}, P-value: {result['P-value']}")

alpha = 0.05
if all(result['P-value'] >= alpha for result in results):
    print("No Simpson's Paradox detected. Associations are consistent across 'Event
Location' subgroups.")
else:
```

```
print("Simpson's Paradox may be present. Associations change within 'Event Location' subgroups.")
```

No Simpson's Paradox detected. Associations are consistent across 'Event Location' subgroups.

FEATURE SELECTION

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import SelectFromModel
from mlxtend.feature_selection import SequentialFeatureSelector
import numpy as np
from sklearn import metrics
from sklearn.model_selection import train_test_split
```

```
deathevent_month_mode = data['deathevent_month'].mode()[0]
day_of_week_death_mode = data['day_of_week_death'].mode()[0]
year_death_mode = data['year_death'].mode()[0]
```

```
data['deathevent_month'].fillna(deathevent_month_mode, inplace=True)
data['day_of_week_death'].fillna(day_of_week_death_mode, inplace=True)
data['year_death'].fillna(year_death_mode, inplace=True)
```

```
df=
data[['age','citizenship','event_location_district','gender','took_part_in_the_hostilities','place_of_residence_district','type_of_injury','ammunition','killed_by','event_month','day_of_week','deathevent_month','day_of_week_death','year_death','year_event']]
```

```
from sklearn.preprocessing import LabelEncoder
```

```
# Initialize LabelEncoder
```

```
label_encoder = LabelEncoder()
```

```
# Apply LabelEncoder to each categorical column
```

```
df['age'] = label_encoder.fit_transform(df['age'])
```

```
df['citizenship'] = label_encoder.fit_transform(df['citizenship'])
```

```
df['event_location_district'] =
```

```
label_encoder.fit_transform(df['event_location_district'])
```

```
df['gender'] = label_encoder.fit_transform(df['gender'])
```

```
df['took_part_in_the_hostilities'] =
```

```
label_encoder.fit_transform(df['took_part_in_the_hostilities'])
```

```
df['place_of_residence_district'] =
```

```
label_encoder.fit_transform(df['place_of_residence_district'])
```

```
df['type_of_injury'] = label_encoder.fit_transform(df['type_of_injury'])
```

```
df['ammunition'] = label_encoder.fit_transform(df['ammunition'])
```

```
df['killed_by'] = label_encoder.fit_transform(df['killed_by'])
```

```
df['event_month'] = label_encoder.fit_transform(df['event_month'])
```

```
df['day_of_week'] = label_encoder.fit_transform(df['day_of_week'])
```

```
df['deathevent_month'] = label_encoder.fit_transform(df['deathevent_month'])
```

```
df['day_of_week_death'] = label_encoder.fit_transform(df['day_of_week_death'])
```

```
df['year_death'] = label_encoder.fit_transform(df['year_death'])
```

```
df['year_event'] = label_encoder.fit_transform(df['year_event'])
```

RANDOM SELECTION

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
X=df[['citizenship','killed_by','place_of_residence_district','year_death','type_of_injury','ammunition']]
```

```
y = df['event_location_district']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

```
model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Accuracy: 0.86

FILTER METHOD - INFORMATION GAIN

```
import pandas as pd
from sklearn.feature_selection import mutual_info_classif
import matplotlib.pyplot as plt

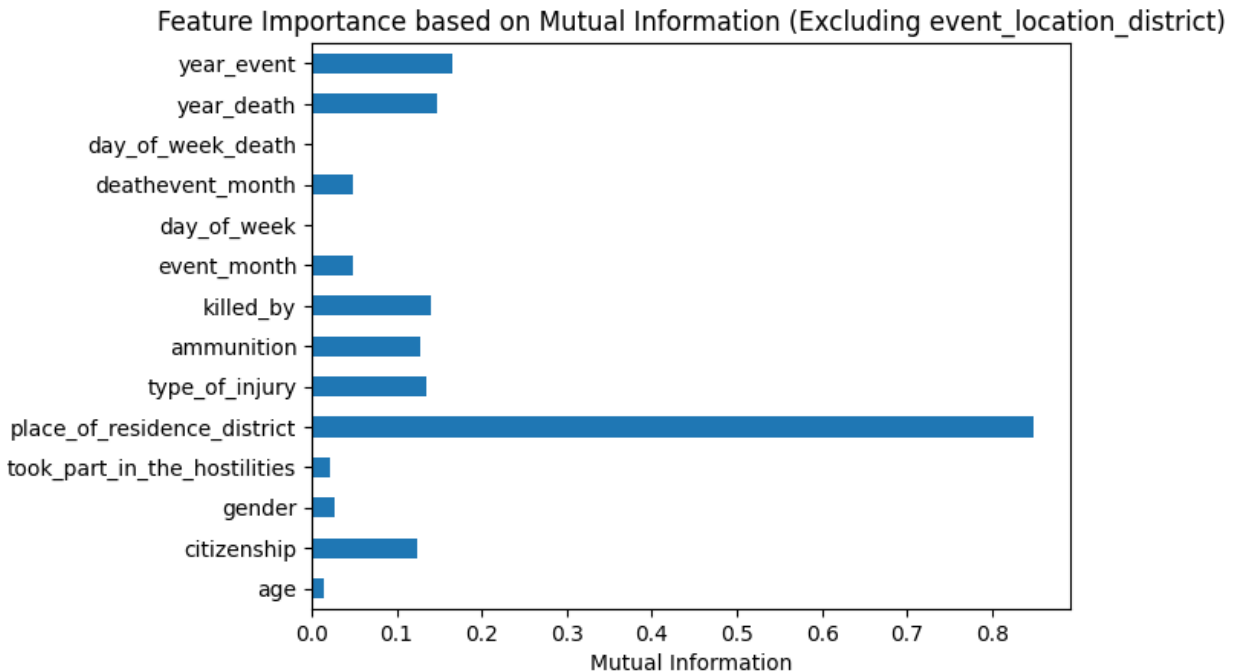
columns_to_exclude = ['event_location_district']

X = df.drop(columns=columns_to_exclude)
y = df['event_location_district']

importance = mutual_info_classif(X, y)
feat_importance = pd.Series(importance, index=X.columns)
feat_importance.plot(kind='barh')
plt.xlabel('Mutual Information')
plt.title('Feature Importance based on Mutual Information (Excluding
event_location_district)')
plt.show()

sorted_feat_importance = feat_importance.sort_values(ascending=False)

print("Feature Importance based on Mutual Information (Excluding
'event_location_district'):")
print(sorted_feat_importance)
```



Feature Importance based on Mutual Information (Excluding 'event_location_district'):

place_of_residence_district	0.849354
year_event	0.165826
year_death	0.146807
killed_by	0.139930
type_of_injury	0.135267
ammunition	0.127299
citizenship	0.124343
event_month	0.048881
deathevent_month	0.048828
gender	0.027890
took_part_in_the_hostilities	0.020820
age	0.013683
day_of_week	0.001216
day_of_week_death	0.000000

dtype: float64

```
X=df[['place_of_residence_district','year_event','year_death','killed_by','type_of_injury','ammunition']]
```

```
y = df['event_location_district']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

```
model = RandomForestClassifier()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)  
accuracy = accuracy_score(y_test, y_pred)  
print(f"Accuracy: {accuracy:.2f}")
```

Accuracy: 0.86

EMBEDDED METHOD - LASSO REGRESSION

```
X2 =  
df[['age', 'citizenship', 'gender', 'took_part_in_the_hostilities', 'place_of_residence_di  
strict', 'type_of_injury', 'ammunition', 'killed_by', 'event_month', 'day_of_week', 'death  
event_month', 'day_of_week_death', 'year_death', 'year_event']]  
Y2 = data['event_location_district']  
logistic = LogisticRegression(C=1, penalty="l1", solver="liblinear", random_state  
=7).fit(X2, Y2)  
model1 = SelectFromModel(logistic, prefit=True)  
X_new = model1.transform(X2)  
feature_idx = model1.get_support()  
feature_name = X2.columns[feature_idx]  
print(feature_name)
```

```
Index(['age', 'citizenship', 'gender', 'took_part_in_the_hostilities',  
      'place_of_residence_district', 'type_of_injury', 'ammunition',  
      'killed_by', 'event_month', 'day_of_week', 'death event_month',  
      'day_of_week_death', 'year_death', 'year_event'],  
      dtype='object')
```

```
x2 = df[['age', 'citizenship', 'gender', 'took_part_in_the_hostilities',  
        'place_of_residence_district', 'type_of_injury', 'ammunition',
```

```

        'killed_by', 'event_month', 'day_of_week', 'death_event_month',
        'day_of_week_death', 'year_death', 'year_event']]
y2 = df['event_location_district']
model2 = RandomForestClassifier(n_estimators=600)
x_train, x_test, y_train, y_test= train_test_split(x2, y2, test_size= 0.25,
random_state=0)
model2.fit(x_train, y_train)
y_pred=model2.predict(x_test)
print("ACCURACY OF THE MODEL WITH EMBEDDED METHOD:
",metrics.accuracy_score(y_test, y_pred))
importances2 = model.feature_importances_
print(importances2)

```

ACCURACY OF THE MODEL WITH EMBEDDED METHOD:

0.8504135203164329

[0.75058534 0.06375437 0.05784774 0.03295037 0.03575283 0.05910934]

WRAPPER METHOD - FORWARD FEATURE SELECTION

```

from mlxtend.feature_selection import ExhaustiveFeatureSelector
from sklearn.ensemble import RandomForestClassifier

```

```

X_efs =
df[['age','citizenship','gender','took_part_in_the_hostilities','place_of_residence_di
strict','type_of_injury','ammunition','killed_by','event_month','day_of_week','death
event_month','day_of_week_death','year_death','year_event']]
Y_efs = data['event_location_district']
efs =ExhaustiveFeatureSelector (RandomForestClassifier(), min_features=4,
max_features=8, scoring ='roc_auc',cv=2)
efs= efs.fit(X_efs, Y_efs)

selected_features =X_train.columns [list (efs.best_idx_)]
print(selected_features)
print (efs.best_score_)

```



```

X_filter = df[features]
Y_filter = df['event_location_district']
X_train, X_test, y_train, y_test = train_test_split(X_filter, y, test_size=0.25,
                                                    random_state=42)
model4 = RandomForestClassifier(n_estimators=600)
model4.fit(X_train,y_train)

y_pred = model4.predict(X_test)
accuracy = metrics.accuracy_score(y_test, y_pred)
print('ACCURACY OF THE MODEL WITH WRAPPER METHOD:',accuracy)

```

TIME SERIES ANALYSIS

```

from pandas import datetime
from matplotlib import pyplot
from pandas.plotting import autocorrelation_plot
from statsmodels.tsa.arima_model import ARIMA
from datetime import datetime

def parser(x):
    date_obj = datetime.strptime(x, '%m/%d/%Y')
    return date_obj.strftime('%Y%m%d')

to_use_series = data[["date_of_death", "ammunition"]]

to_use_series.isnull().sum()

date_of_death    0
ammunition       0
dtype: int64

to_use_series.head()

      date_of_death    ammunition
0    2023-09-24  live ammunition

```

```

1    2023-09-24  live ammunition
2    2023-09-22  live ammunition
3    2023-09-20  live ammunition
4    2023-09-19  live ammunition

```

```
to_use_series['Year'] = to_use_series['date_of_death'].dt.year
```

```
# Group the data by year, ammunition type, and count the occurrences
```

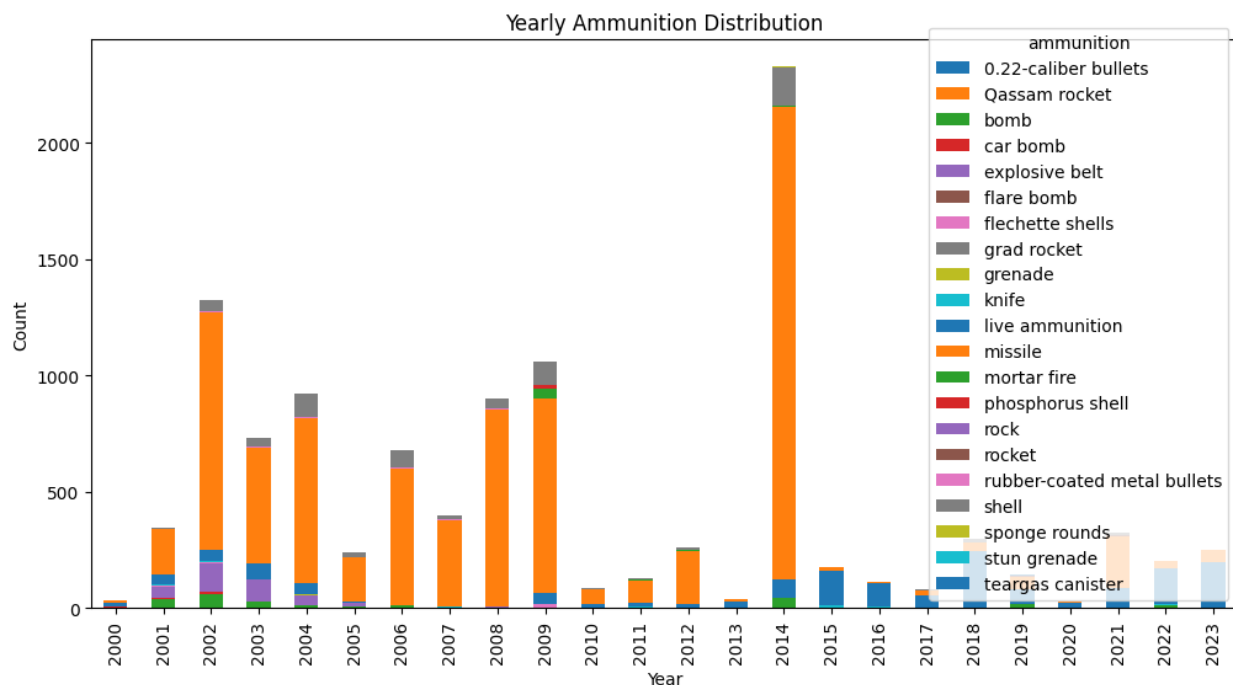
```
yearly_ammunition_counts = to_use_series.groupby(['Year',
'ammunition']).size().unstack().fillna(0)
```

```
yearly_ammunition_counts.plot(kind='bar', stacked=True, figsize=(12, 6), title='Yearly
Ammunition Distribution')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Count')
```

```
plt.show()
```



```
top_2_ammunitions = ammunition_counts.head(2).index
```

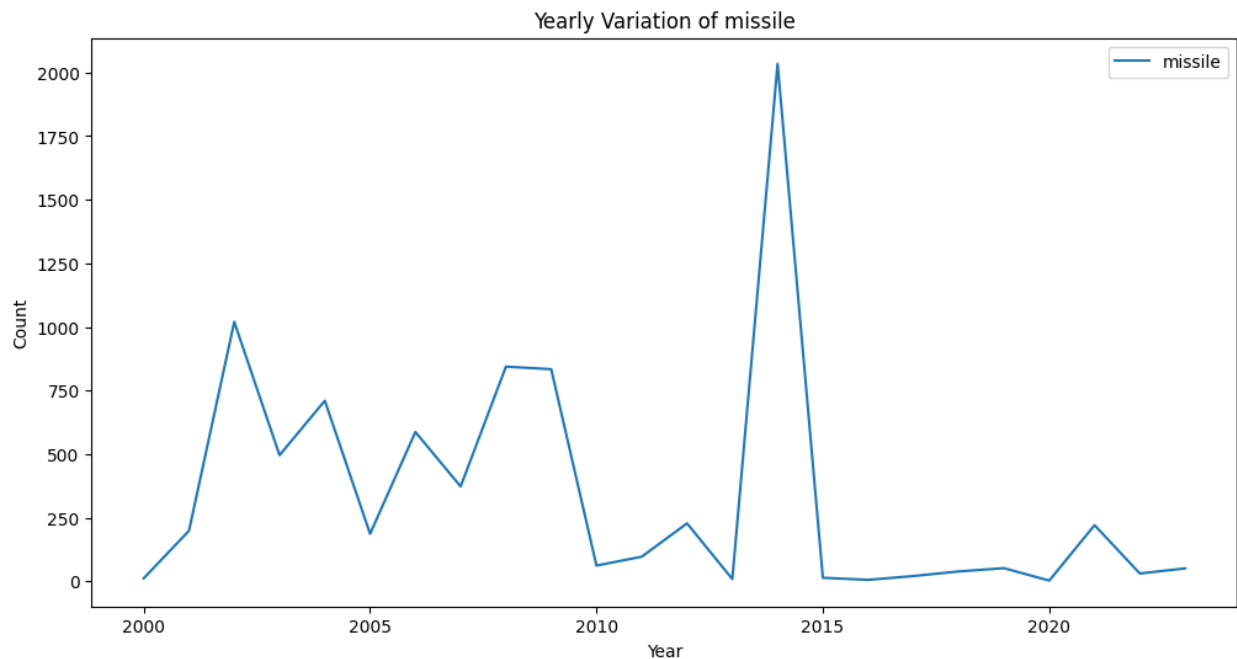
```

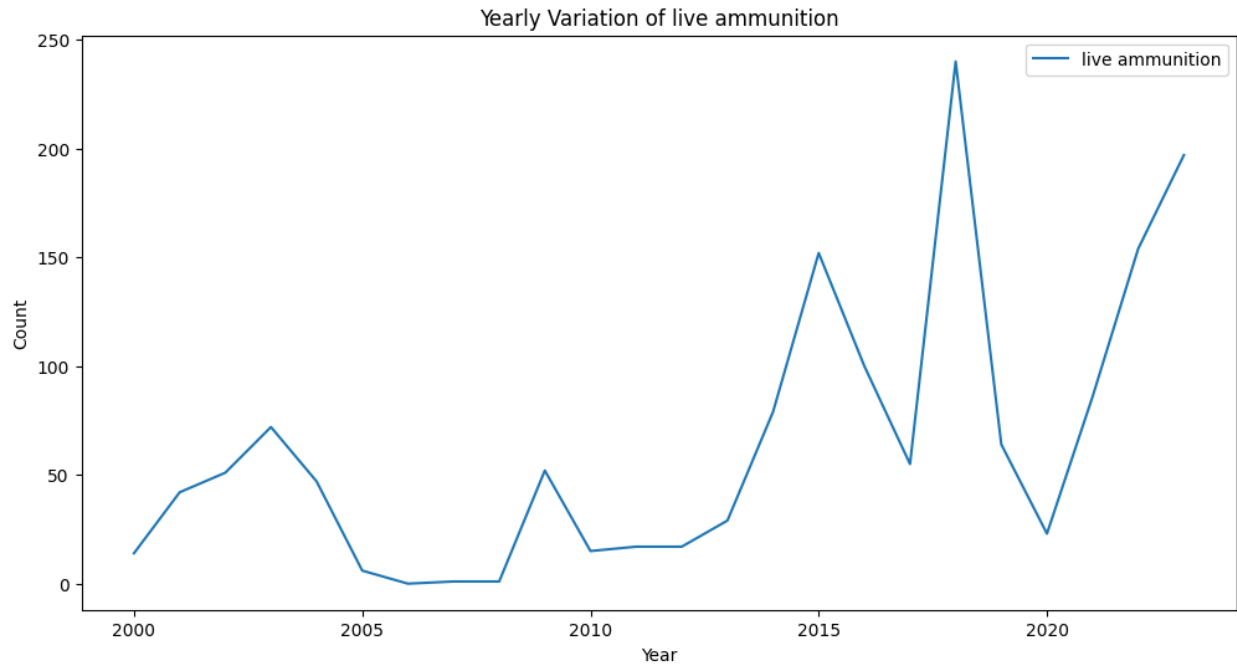
filtered_to_use_series =
to_use_series[to_use_series['ammunition'].isin(top_2_ammunitions)]

yearly_ammunition_counts = filtered_to_use_series.groupby(['Year',
'ammunition']).size().unstack().fillna(0)

for ammunition_type in top_2_ammunitions:
    plt.figure(figsize=(12, 6))
    plt.plot(yearly_ammunition_counts.index,
yearly_ammunition_counts[ammunition_type], label=ammunition_type)
    plt.title(f'Yearly Variation of {ammunition_type}')
    plt.xlabel('Year')
    plt.ylabel('Count')
    plt.legend()
    plt.show()

```



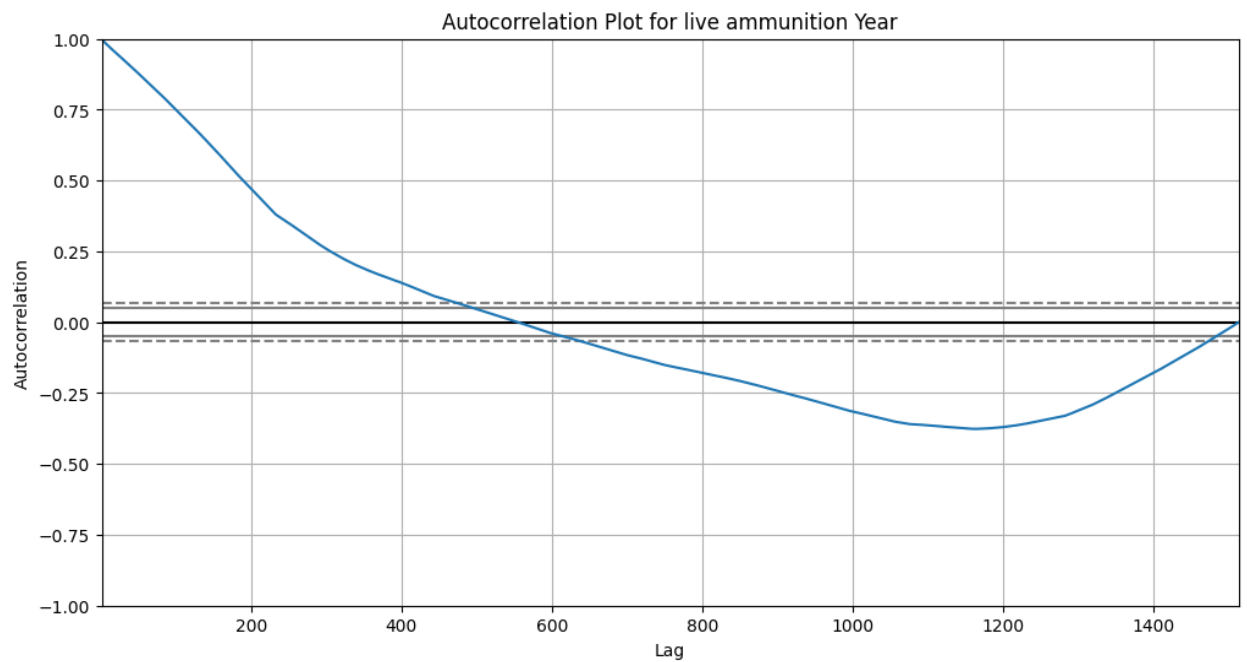
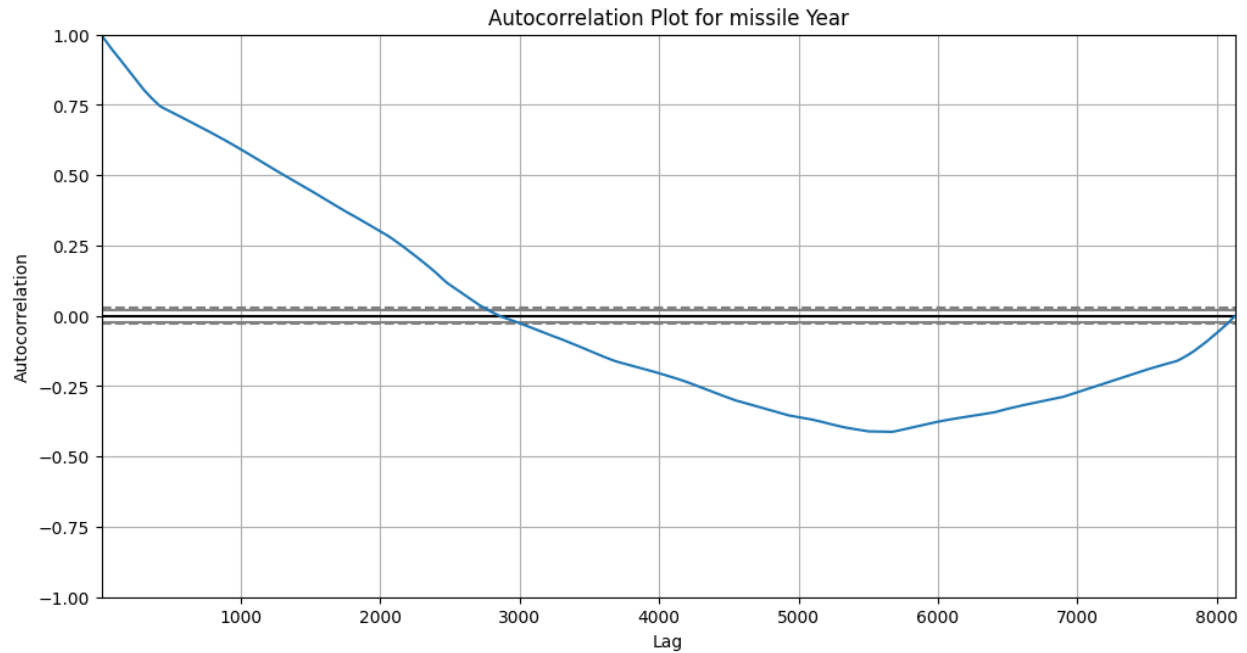


```
to_use_series["Year"] = to_use_series["date_of_death"].dt.year
```

```
from pandas.plotting import autocorrelation_plot
```

```
top_2_ammunitions = ammunition_counts.head(2).index
```

```
for ammunition_type in top_2_ammunitions:  
    filtered_data = to_use_series[to_use_series["ammunition"] == ammunition_type]  
    plt.figure(figsize=(12, 6))  
    autocorrelation_plot(filtered_data["Year"])  
    plt.title(f'Autocorrelation Plot for {ammunition_type} Year')  
    plt.show()
```



```
import statsmodels.api as sm
```

```
# Group data by year and ammunition type and count occurrences
```

```
count_data = to_use_series.groupby(['Year', 'ammunition']).size().unstack().fillna(0)
```

```
# Perform Poisson regression
for ammunition_type in top_2_ammunitions:
    count_series = count_data[ammunition_type]
    poisson_model = sm.GLM(count_series, sm.add_constant(count_series.index),
family=sm.families.Poisson())
    poisson_results = poisson_model.fit()

    print(f"Summary for {ammunition_type}:")
    print(poisson_results.summary())
```

Summary for missile:

Generalized Linear Model Regression Results

```
=====
=====
Dep. Variable:          missile  No. Observations:           24
Model:                  GLM    Df Residuals:                22
Model Family:           Poisson  Df Model:                  1
Link Function:           Log    Scale:                     1.0000
Method:                  IRLS   Log-Likelihood:            -5515.6
Date:                   Wed, 08 Nov 2023  Deviance:          10876.
Time:                   16:07:23  Pearson chi2:         1.65e+04
No. Iterations:          5  Pseudo R-squ. (CS):             1.000
Covariance Type:         nonrobust
=====
=====
               coef    std err          z      P>|z|     [0.025    0.975]
-----
const      132.5894    3.402    38.976    0.000    125.922    139.257
x1        -0.0631    0.002   -37.237    0.000    -0.066    -0.060
=====
=====
```

Summary for live ammunition:

Generalized Linear Model Regression Results

```
=====
=====
Dep. Variable:          live ammunition  No. Observations:           24
Model:                  GLM    Df Residuals:                22
Model Family:           Poisson  Df Model:                  1
Link Function:           Log    Scale:                     1.0000
```

Method: IRLS Log-Likelihood: -488.54
 Date: Wed, 08 Nov 2023 Deviance: 852.90
 Time: 16:07:23 Pearson chi2: 836.
 No. Iterations: 5 Pseudo R-squ. (CS): 1.000
 Covariance Type: nonrobust

```
=====
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
const    -181.9057     8.395    -21.669     0.000   -198.359   -165.452
x1         0.0924     0.004     22.184     0.000     0.084     0.101
=====
=====
```

```
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt
```

```
top_2_ammunitions = ammunition_counts.head(2).index
```

```
for ammunition_type in top_2_ammunitions:
    filtered_data = to_use_series[to_use_series['ammunition'] == ammunition_type]
```

```
yearly_counts = filtered_data.groupby('Year').size()
```

```
# Fit ARIMA model
```

```
model = ARIMA(yearly_counts, order=(5, 1, 0))
```

```
model_fit = model.fit()
```

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(yearly_counts, label='Observed')
```

```
plt.plot(model_fit.fittedvalues, color='red', label='Predicted')
```

```
plt.title(f'ARIMA Model for {ammunition_type}')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Count')
```

```
plt.legend()
```

```
plt.show()
```

```

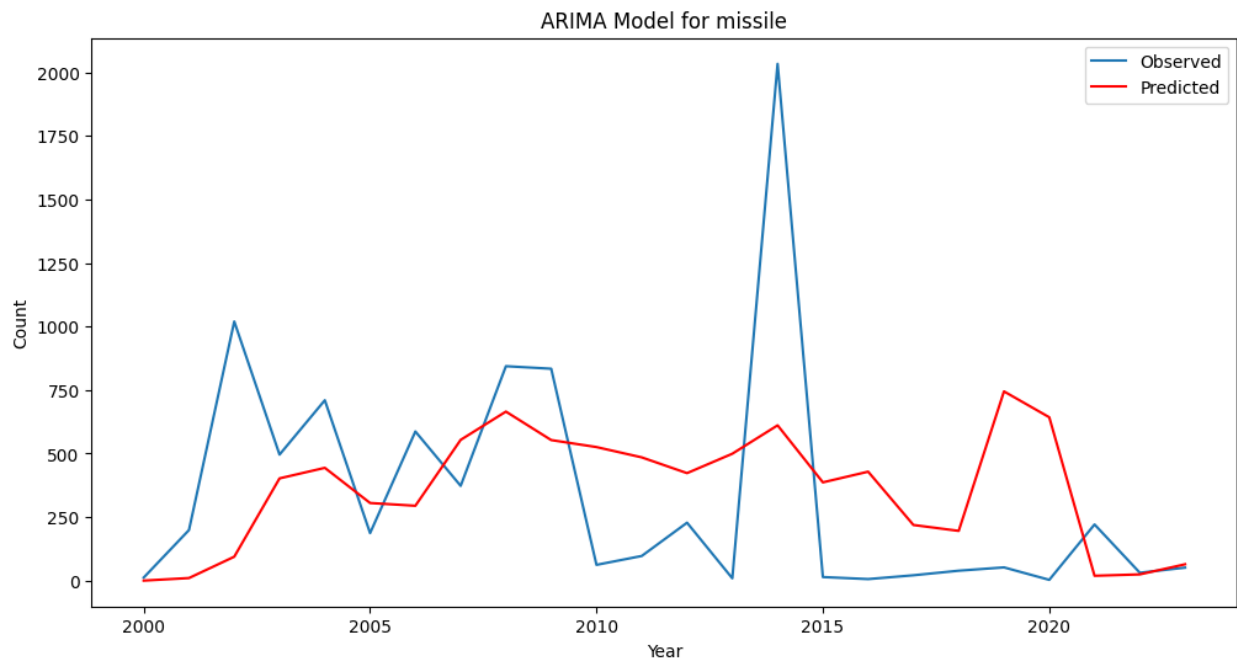
print(f"Summary for {ammunition_type}:")
print(model_fit.summary())

# Plot the residuals
residuals = pd.DataFrame(model_fit.resid)
residuals.plot()
plt.title(f'Residuals for {ammunition_type}')
plt.xlabel('Year')
plt.ylabel('Residual Value')
plt.show()

# Plot the kernel density estimate (KDE) of residuals
residuals.plot(kind='kde')
plt.title(f'KDE of Residuals for {ammunition_type}')
plt.xlabel('Residual Value')
plt.show()

# Print a summary of residuals
print(f'Residuals summary for {ammunition_type}:')
print(residuals.describe())

```



Summary for missile:

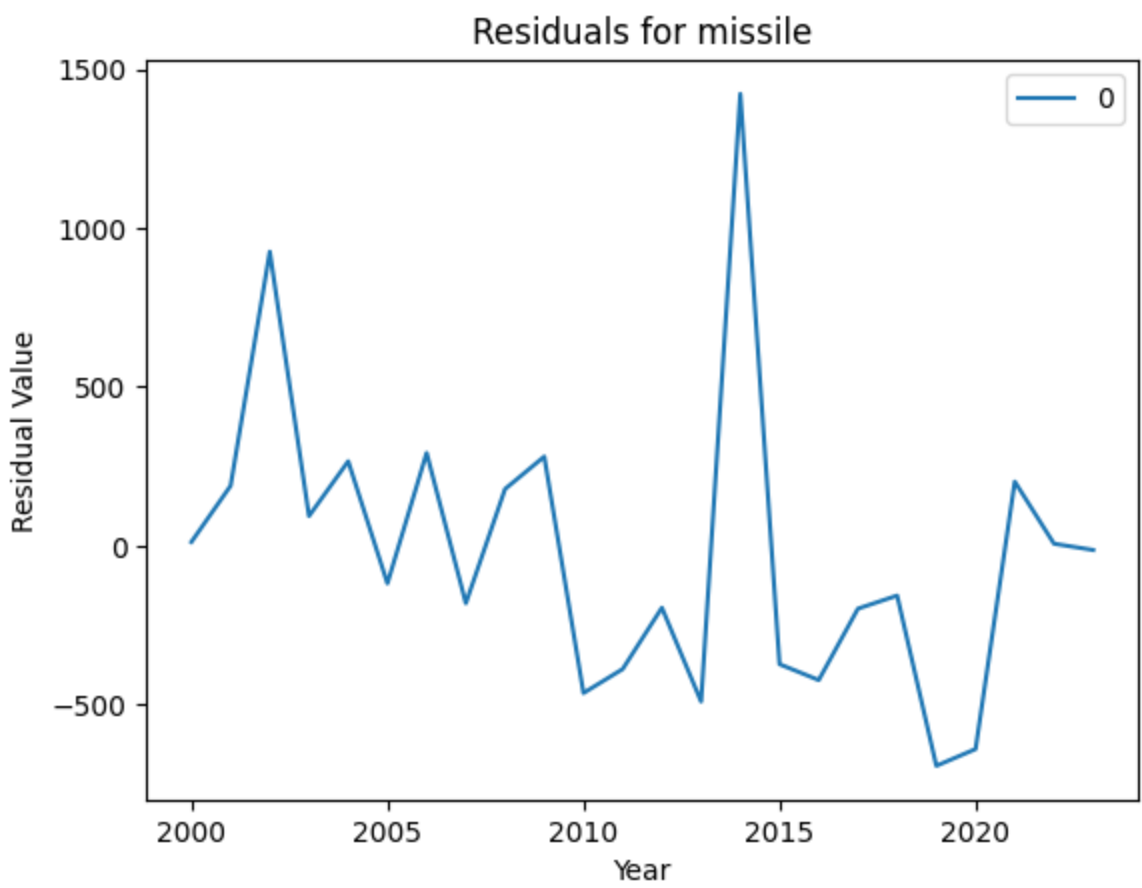
SARIMAX Results

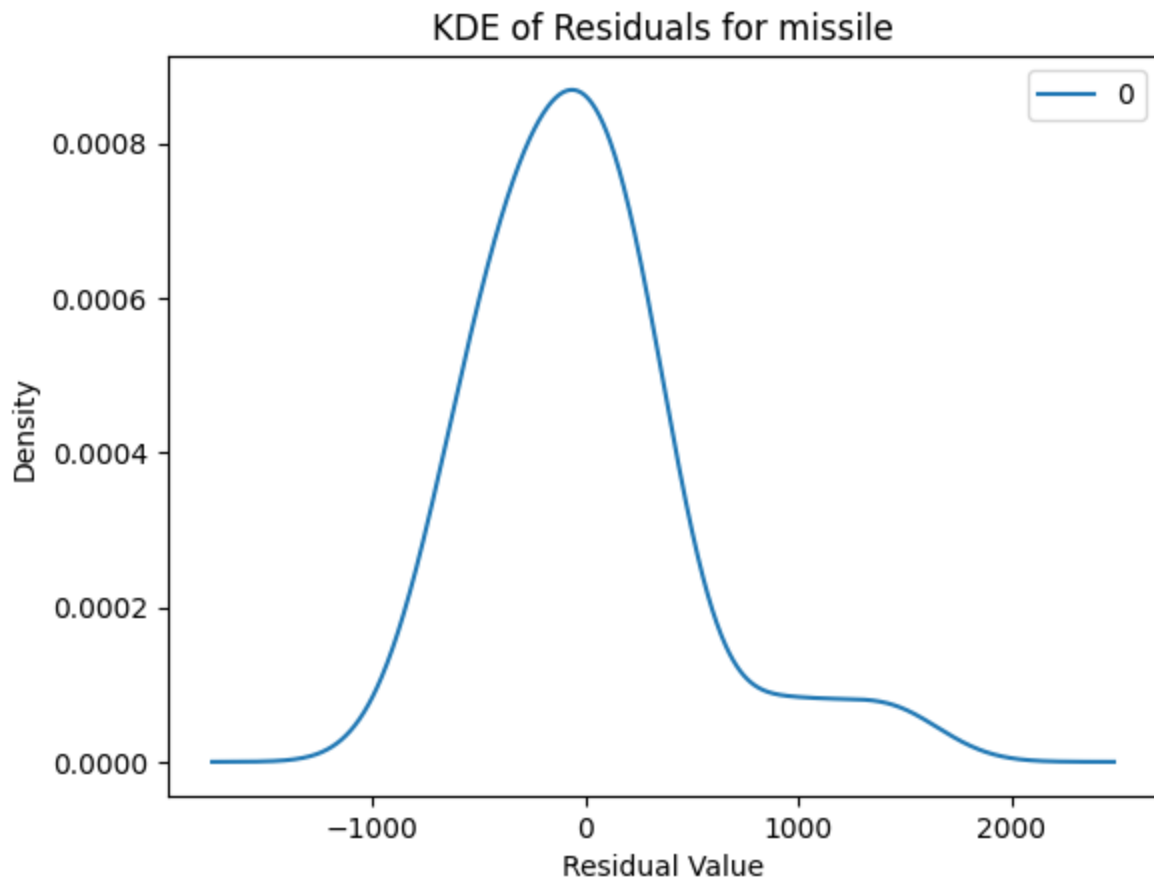
```
=====
Dep. Variable:          y  No. Observations:          24
Model:                ARIMA(5, 1, 0)  Log Likelihood      -174.532
Date:                Wed, 08 Nov 2023  AIC                361.065
Time:                16:09:43  BIC                367.878
Sample:                0  HQIC                362.778
                        - 24
```

Covariance Type: opg

```
=====
              coef  std err          z      P>|z|    [0.025    0.975]
-----
ar.L1        -0.9566    0.261     -3.662    0.000    -1.469    -0.445
ar.L2        -0.7795    0.347     -2.244    0.025    -1.461    -0.099
ar.L3        -0.7288    0.357     -2.041    0.041    -1.429    -0.029
ar.L4        -0.6700    0.177     -3.781    0.000    -1.017    -0.323
ar.L5        -0.3084    0.149     -2.073    0.038    -0.600    -0.017
sigma2       2.12e+05  5.66e+04    3.745    0.000    1.01e+05  3.23e+05
=====
```

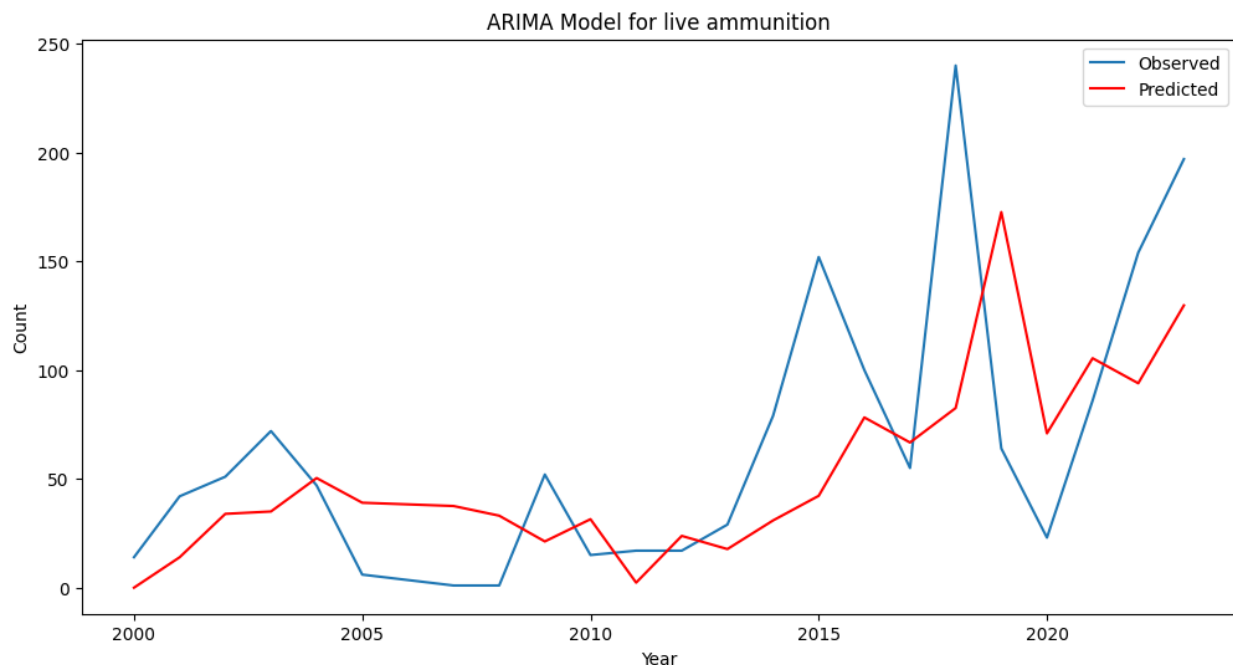
```
=====
Ljung-Box (L1) (Q):          0.07  Jarque-Bera (JB):          11.01
Prob(Q):                    0.79  Prob(JB):              0.00
Heteroskedasticity (H):        1.44  Skew:              1.27
Prob(H) (two-sided):          0.62  Kurtosis:          5.24
=====
```





Residuals summary for missile:

```
0
count  24.000000
mean   -19.298720
std    474.906934
min    -692.790405
25%    -376.579701
50%    -65.865387
75%     192.186207
max     1422.806373
```



Summary for live ammunition:

SARIMAX Results

```

=====
Dep. Variable:          y  No. Observations:          23
Model:                 ARIMA(5, 1, 0)  Log Likelihood    -120.262
Date:                 Wed, 08 Nov 2023  AIC              252.523
Time:                 16:09:45  BIC                   259.070
Sample:               0  HQIC                   254.065
                        - 23
  
```

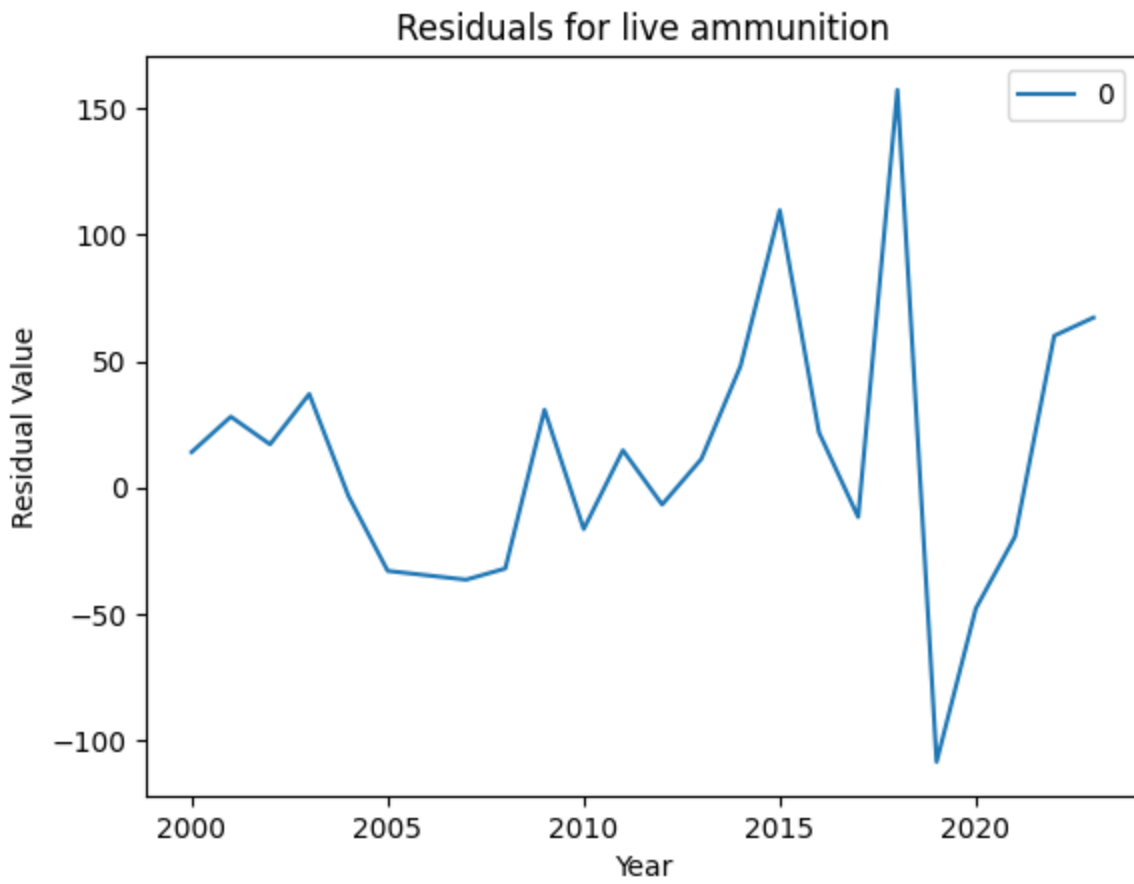
Covariance Type: opg

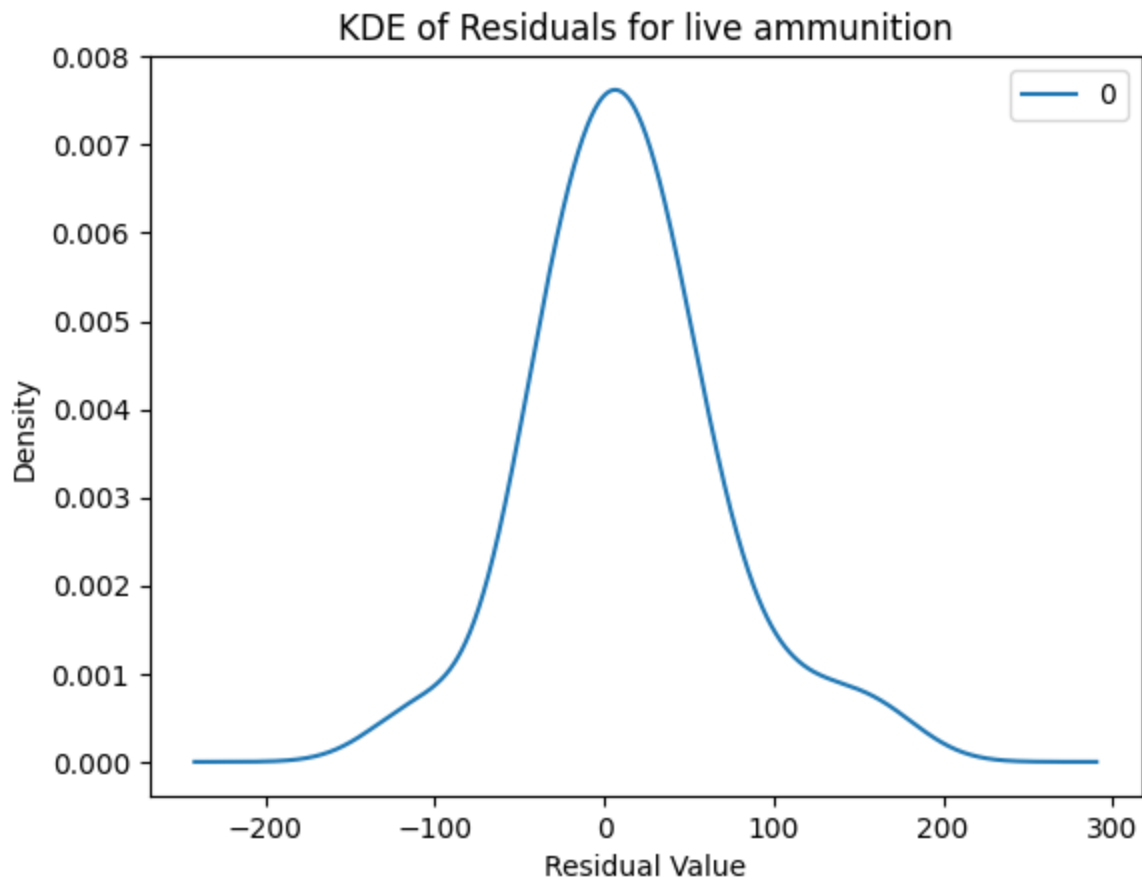
```

=====
      coef  std err      z  P>|z|  [0.025   0.975]
-----
ar.L1    -0.5420    0.213   -2.545   0.011   -0.959   -0.125
ar.L2    -0.6168    0.439   -1.405   0.160   -1.477    0.244
ar.L3    -0.2878    0.347   -0.829   0.407   -0.968    0.392
ar.L4    -0.1702    0.297   -0.572   0.567   -0.753    0.413
ar.L5     0.0524    0.260    0.202   0.840   -0.456    0.561
sigma2   3156.0981  1072.943    2.942   0.003  1053.168  5259.028
  
```

```

=====
Ljung-Box (L1) (Q):      0.06  Jarque-Bera (JB):      1.93
Prob(Q):                 0.81  Prob(JB):            0.38
Heteroskedasticity (H):   8.71  Skew:              0.52
Prob(H) (two-sided):      0.01  Kurtosis:          4.00
=====
  
```





Residuals summary for live ammunition:

```
0
count    23.000000
mean     13.092110
std      54.831459
min     -108.646903
25%     -17.962349
50%      14.000000
75%      33.874325
max      157.432065
```

```
import matplotlib.pyplot as plt
```

```
top_2_ammunitions = ammunition_counts.head(2).index
```

```

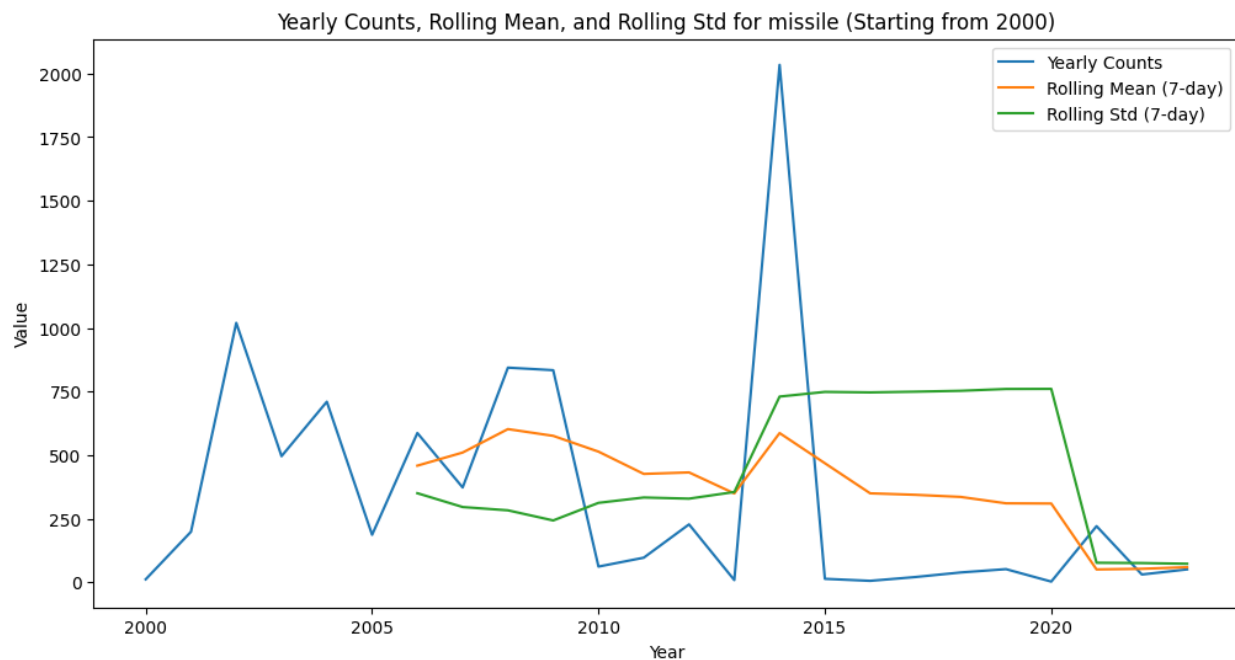
for ammunition_type in top_2_ammunitions:
    filtered_data = to_use_series[(to_use_series['ammunition'] == ammunition_type) &
    (to_use_series['Year'] >= 2000)]

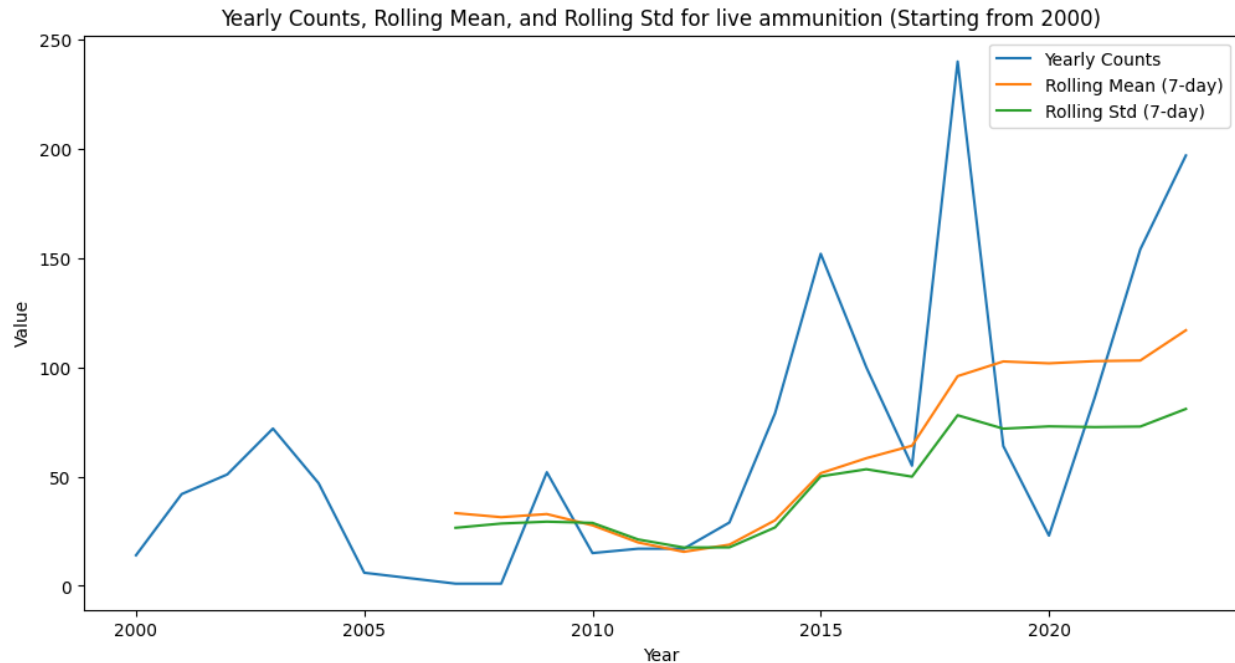
    yearly_counts = filtered_data.groupby('Year').size()

    # Calculate the rolling mean and rolling standard deviation with a 7-day window
    rolling_mean = yearly_counts.rolling(7).mean()
    rolling_std = yearly_counts.rolling(7).std()

    plt.figure(figsize=(12, 6))
    plt.plot(yearly_counts, label='Yearly Counts')
    plt.plot(rolling_mean, label='Rolling Mean (7-day)')
    plt.plot(rolling_std, label='Rolling Std (7-day)')
    plt.title(f'Yearly Counts, Rolling Mean, and Rolling Std for {ammunition_type} (Starting
from 2000)')
    plt.xlabel('Year')
    plt.ylabel('Value')
    plt.legend()
    plt.show()

```





```
from statsmodels.tsa.stattools import adfuller
```

```
top_2_ammunitions = ammunition_counts.head(2).index
```

```
for ammunition_type in top_2_ammunitions:
```

```
    filtered_data = to_use_series[to_use_series['ammunition'] == ammunition_type]
```

```
    # Perform the ADF test
```

```
    adft = adfuller(filtered_data['Year'], autolag="AIC")
```

```
    output_df = pd.DataFrame({
```

```
        "Values": [adft[0], adft[1], adft[2], adft[3], adft[4]['1%'], adft[4]['5%'], adft[4]['10%']],
```

```
        "Metric": ["Test Statistics", "p-value", "No. of lags used", "Number of observations used",
```

```
                  "Critical value (1%)", "Critical value (5%)", "Critical value (10%)"]
```

```
    })
```

```
    print(f"ADF Test Results for {ammunition_type}:")
```

```
    print(output_df)
```


ADF Test Results for missile:

	Values	Metric
0	-2.920472	Test Statistics
1	0.043019	p-value
2	10.000000	No. of lags used
3	8119.000000	Number of observations used
4	-3.431156	Critical value (1%)
5	-2.861896	Critical value (5%)
6	-2.566960	Critical value (10%)

ADF Test Results for live ammunition:

	Values	Metric
0	2.056150	Test Statistics
1	0.998742	p-value
2	1.000000	No. of lags used
3	1512.000000	Number of observations used
4	-3.434682	Critical value (1%)
5	-2.863453	Critical value (5%)
6	-2.567789	Critical value (10%)

```
top_2_ammunitions = ammunition_counts.head(2).index
```

```
\lags = [1, 3, 6, 9]
```

```
for ammunition_type in top_2_ammunitions:
```

```
    filtered_data = to_use_series[to_use_series['ammunition'] == ammunition_type]
```

```
    print(f"Autocorrelation for {ammunition_type}:")
```

```
    for lag in lags:
```

```
        autocorrelation = filtered_data['Year'].autocorr(lag=lag)
```

```
        print(f"{lag}-Month Lag: {autocorrelation}")
```

Autocorrelation for missile:

1-Month Lag: 0.9985966535306842

3-Month Lag: 0.9984961928048004

6-Month Lag: 0.9983274099894497

9-Month Lag: 0.9981386419678294

Autocorrelation for live ammunition:
1-Month Lag: 0.9996339396168304
3-Month Lag: 0.9991614883218808
6-Month Lag: 0.9984550264940032
9-Month Lag: 0.9976997449785631

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
top_2_ammunitions = ammunition_counts.head(2).index
```

```
for ammunition_type in top_2_ammunitions:
```

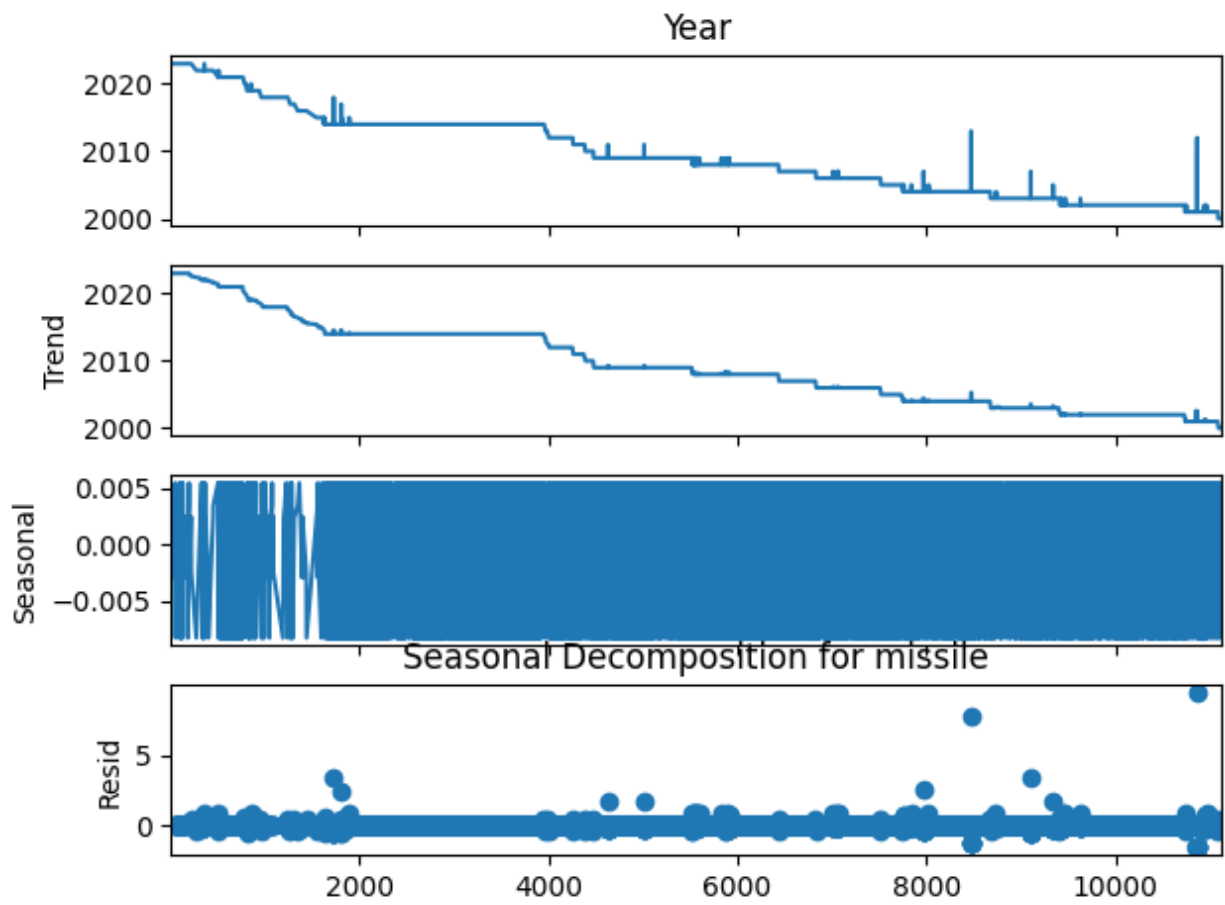
```
    filtered_data = to_use_series[to_use_series['ammunition'] == ammunition_type]
```

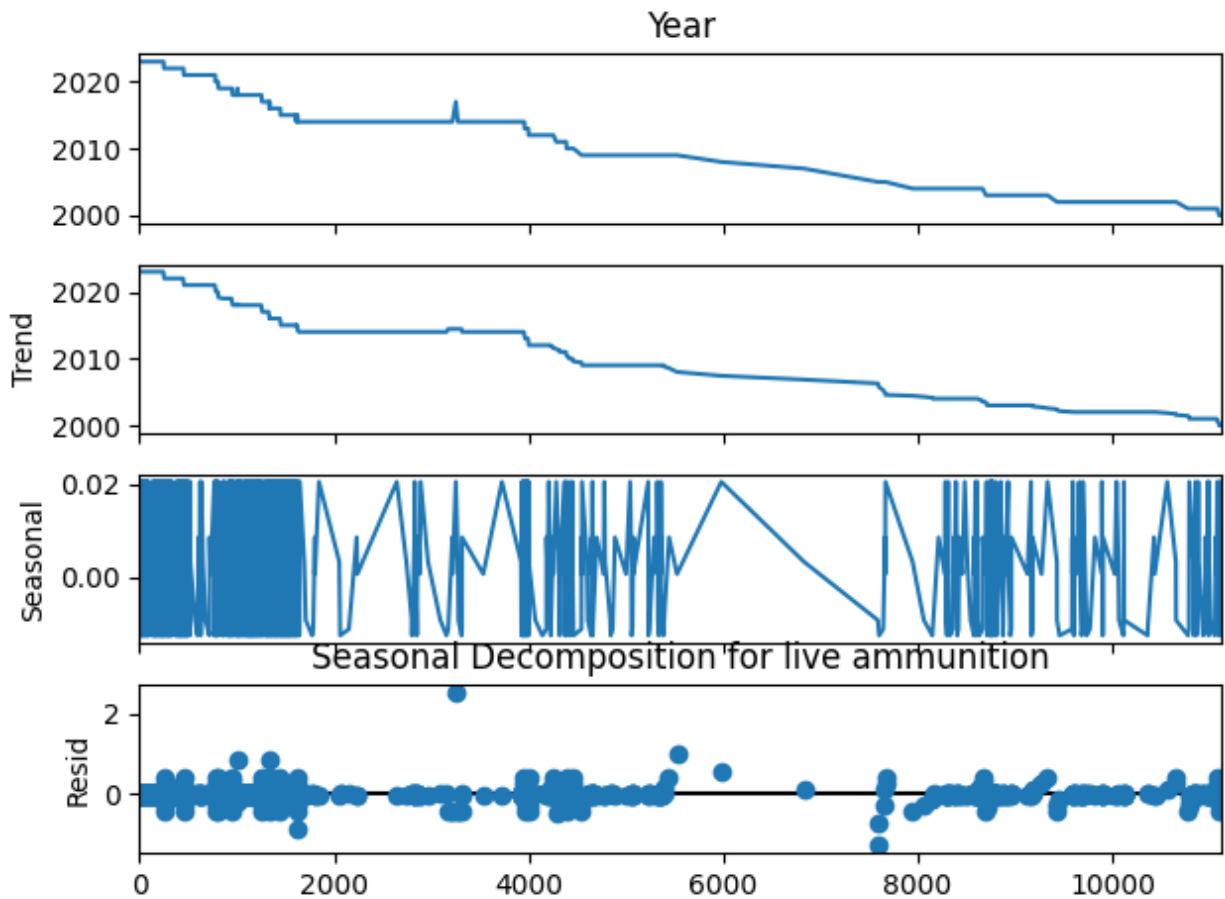
```
    decompose = seasonal_decompose(filtered_data['Year'], model='additive', period=7)
```

```
    decompose.plot()
```

```
    plt.title(f'Seasonal Decomposition for {ammunition_type}')
```

```
    plt.show()
```





HYPOTHESIS TESTING

HYPOTHESIS 1

```
contingency_table = pd.crosstab(data['event_location_region'],
data['took_part_in_the_hostilities'])
```

```
chi2, p, _, _ = stats.chi2_contingency(contingency_table)
```

```
alpha = 0.05
```

```
print("Chi-Square Statistic:", chi2)
```

```
print("P-value:", p)
```

```
if p < alpha:
    print("Reject the null hypothesis: There is a significant association between event
location region and taking part in hostilities.")
else:
    print("Accept the null hypothesis: There is no significant association between event
location region and taking part in hostilities.")
```

Chi-Square Statistic: 4513.45835536826

P-value: 0.0

Reject the null hypothesis: There is a significant association between event location region and taking part in hostilities.

HYPOTHESIS 2

```
hostilities = data[data['took_part_in_the_hostilities'] == 'Yes']['age']
no_hostilities = data[data['took_part_in_the_hostilities'] == 'No']['age']
```

```
# Perform a t-test for independent samples
```

```
t_statistic, p_value = stats.ttest_ind(hostilities, no_hostilities, equal_var=False)
```

```
alpha = 0.05
```

```
print("T-statistic:", t_statistic)
print("P-value:", p_value)
```

```
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference in the age between
those who took part in hostilities and those who did not.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in the age
between those who took part in hostilities and those who did not.")
```

T-statistic: -7.525687017651027

P-value: 5.765293976149714e-14

Reject the null hypothesis: There is a significant difference in the age between those who took part in hostilities and those who did not.

HYPOTHESIS 3

```
result = stats.kruskal(  
    *[group['event_location_region'] for name, group in data.groupby(['type_of_injury',  
    'ammunition'])]  
)
```

alpha = 0.05

```
print("Kruskal-Wallis Statistic:", result.statistic)  
print("P-value:", result.pvalue)
```

if result.pvalue < alpha:

```
    print("Reject the null hypothesis: There is a significant difference in  
    event_location_region among different combinations of type_of_injury and  
    ammunition.")
```

else:

```
    print("Fail to reject the null hypothesis: There is no significant difference in  
    event_location_region among different combinations of type_of_injury and  
    ammunition.")
```

Kruskal-Wallis Statistic: 2533.7692340686654

P-value: 0.0

Reject the null hypothesis: There is a significant difference in event_location_region among different combinations of type_of_injury and ammunition.

HYPOTHESIS 4

```
contingency_table = pd.crosstab(data['killed_by'], data['citizenship'])
```

```
chi2, p, _, _ = stats.chi2_contingency(contingency_table)
```

alpha = 0.05

```
print("Chi-Square Statistic:", chi2)
```

```
print("P-value:", p)
```

```
if p < alpha:
```

```
    print("Reject the null hypothesis: There is a significant association between killed_by  
and citizenship.")
```

```
else:
```

```
    print("Fail to reject the null hypothesis: There is no significant association between  
killed_by and citizenship.")
```

Chi-Square Statistic: 11112.120512176605

P-value: 0.0

Reject the null hypothesis: There is a significant association between killed_by and citizenship.

HYPOTHESIS 5

```
contingency_table = pd.crosstab(data['day_of_week_death'], data['citizenship'])
```

```
chi2, p, _, _ = stats.chi2_contingency(contingency_table)
```

alpha = 0.05

```
print("Chi-Square Statistic:", chi2)
```

```
print("P-value:", p)
```

```
if p < alpha:
```

```
    print("Reject the null hypothesis: There is a significant association between  
day_of_week_death and citizenship.")
```

```
else:
```

```
    print("Fail to reject the null hypothesis: There is no significant association between  
day_of_week_death and citizenship.")
```

Chi-Square Statistic: 90.5207538285631

P-value: 1.1641669086312365e-11

Reject the null hypothesis: There is a significant association between day_of_week_death and citizenship.

HYPOTHESIS 6

```
contingency_table = pd.crosstab(data['ammunition'], data['killed_by'])
```

```
chi2, p, _, _ = stats.chi2_contingency(contingency_table)
```

```
alpha = 0.05
```

```
print("Chi-Square Statistic:", chi2)
```

```
print("P-value:", p)
```

```
if p < alpha:
```

```
    print("Reject the null hypothesis: There is a significant association between  
ammunition and killed_by.")
```

```
else:
```

```
    print("Fail to reject the null hypothesis: There is no significant association between  
ammunition and killed_by.")
```

Chi-Square Statistic: 5013.59697093408

P-value: 0.0

Reject the null hypothesis: There is a significant association between ammunition and killed_by.

CONCLUSIONS

FEATURE SELECTION:

Filter Method - Information Gain:

Test Objective: To identify the importance of features in predicting the "event_location_district."

Conclusion: The feature importance analysis showed that "place_of_residence_district" has the highest mutual information with the target variable, followed by "year_event" and "year_death." As a result, these features were retained, while the others were excluded. This implies that the location of residence and event years are the most informative features for predicting the event location.

Embedded Method:

Test Objective: To select features based on a model's intrinsic feature importance.

Conclusion: By using the embedded method, features such as "age," "citizenship," "gender," "place_of_residence_district," "type_of_injury," "ammunition," and others were identified as important for predicting "event_location_district." A RandomForestClassifier model was used for feature selection.

Wrapper Method:

Test Objective: Exhaustively select features using a RandomForestClassifier in a wrapper method.

Conclusion: Features were selected using an exhaustive feature selector based on the RandomForestClassifier model. The accuracy of the model trained on the selected features was observed to be approximately 85%. The most relevant features include "age," "citizenship," "gender," "place_of_residence_district," and others.

In conclusion, the statistical tests provided evidence for significant relationships between various variables. Additionally, feature selection methods helped identify the most important features for predicting the "event_location_district." These conclusions are essential for further analysis and modeling in understanding the factors affecting event locations.

Simpson Paradox:

Part 1: 'Killed By' Subgroups

Initial Chi-Squared Statistic: 23182.12

Subgroups: Israeli security forces, Palestinian civilians, Israeli civilians

The Chi-Squared Statistics and p-values for each subgroup indicate that there is a significant association between "type_of_injury" and "ammunition" in each subgroup, with all p-values being less than 0.05.

The conclusion states that Simpson's Paradox may be present because the associations between "type_of_injury" and "ammunition" change within different "Killed By" subgroups.

Part 2: 'Event Location' Subgroups

Initial Chi-Squared Statistic: (not provided)

Subgroups: Various event locations

The Chi-Squared Statistics and p-values for each subgroup within different event locations suggest consistent associations between "type_of_injury" and "ammunition."

The conclusion is that no Simpson's Paradox is detected, as the associations between "type_of_injury" and "ammunition" remain consistent across the various "Event Location" subgroups.

In summary, the analysis indicates that there may be a presence of Simpson's Paradox within the "Killed By" subgroups, where the associations between "type_of_injury" and "ammunition" change, while no Simpson's Paradox is detected within the "Event Location" subgroups, where the associations remain consistent across different event locations. This difference in findings highlights

the importance of considering subgroup analysis when interpreting statistical associations within the dataset.

TIME SERIES

Analyzing yearly variation in the counts of top ammunition types

Autocorrelation Analysis, ARIMA (AutoRegressive Integrated Moving Average), residual analysis, and the Ljung-Box test for autocorrelation in residuals, seasonal decomposition, ADF test (Augmented Dickey-Fuller Test)

The ARIMA models are used for time series forecasting and provide parameter summaries for both ammunition types, 'missile' and 'live ammunition.'

The seasonal decomposition plots reveal any seasonal patterns or trends in the data. The ADF tests indicate whether the time series data is stationary.

Hypothesis Testing:

1 - T-Test for Independent Samples:

Test Objective: To determine if there is a significant difference in age between individuals who took part in hostilities and those who did not.

Conclusion: The t-test yielded a statistically significant result with a very low p-value ($p < 0.05$). Therefore, we reject the null hypothesis, concluding that there is a significant difference in age between those who participated in hostilities and those who did not. This suggests that age is associated with participation in hostilities.

2 - Kruskal-Wallis Test:

Test Objective: To investigate if there is a significant difference in event location regions among different combinations of type of injury and ammunition.

Conclusion: The Kruskal-Wallis test resulted in a very low p-value ($p < 0.05$). Thus, we reject the null hypothesis, indicating a significant difference in event location regions among different combinations of type of injury and ammunition. It

suggests that the type of injury and ammunition used have an impact on the event location.

3 - Chi-Square Test 1:

Test Objective: To determine if there is a significant association between the variable "killed_by" and "citizenship."

Conclusion: The chi-square test produced a very low p-value ($p < 0.05$), leading us to reject the null hypothesis. This means there is a significant association between the variable "killed_by" and "citizenship." It suggests that the identity of the perpetrator ("killed_by") is linked to the citizenship of the victims.

4 - Chi-Square Test 2:

Test Objective: To investigate if there is a significant association between the "day_of_week_death" and "citizenship."

Conclusion: The chi-square test yielded a very low p-value ($p < 0.05$). Therefore, we reject the null hypothesis, concluding that there is a significant association between the "day_of_week_death" and "citizenship." This indicates that the day of the week on which deaths occur is related to the citizenship of the individuals.

5 - Chi-Square Test 3:

Test Objective: To explore if there is a significant association between "ammunition" and "killed_by."

Conclusion: The chi-square test resulted in a very low p-value ($p < 0.05$). Thus, we reject the null hypothesis, indicating a significant association between "ammunition" and "killed_by." It implies that the type of ammunition used is related to the identity of the perpetrator.